Advanced Platform Managment Link (APML) Library

Generated by Doxygen 1.9.1

1 Advanced Platform Management Link (APML) Library 1
1.1 (formerly known as EPYC™ System Management Interface (E-SMI) Out-of-band Library) . . . 1
1.2 Important note about Versioning and Backward Compatibility . . . 1
1.3 Building APML Library . . . 1
1.4 Usage Basics . . . 3
1.5 Usage . . . 3
1.5.1 Tool Usage . . . 3
1.5.2 | PowerLimitMax (Watts) | 400.000 | . . . 4

2 Module Index 7
2.1 Modules . . . 7

3 Data Structure Index 9
3.1 Data Structures . . . 9

4 File Index 13
4.1 File List . . . 13

5 Module Documentation 15
5.1 Auxiliary functions . . . 15
5.1.1 Detailed Description . . . 15
5.1.2 Function Documentation . . . 15
5.1.2.1 errno_to_oob_status() . . . 15
5.1.2.2 esmi_get_err_msg() . . . 15
5.2 SB-RMI Mailbox Service . . . 16
5.2.1 Detailed Description . . . 16
5.3 Power Monitor . . . 16
5.3.1 Detailed Description . . . 16
5.3.2 Function Documentation . . . 16
5.3.2.1 read_socket_power() . . . 16
5.3.2.2 read_socket_power_limit() . . . 17
5.3.2.3 read_max_socket_power_limit() . . . 17
5.4 Platforms . . . 18
5.4.1 Fam-19h_Mod-00h-0Fh . . . 18
5.4.2 Fam-19h_Mod-10h-1Fh . . . 18
5.4.3 Fam-19h_Mod-90h-9Fh . . . 18
5.4.4 Fam-1Ah_Mod-00h-0Fh . . . 18
5.4.5 Fam-1Ah_Mod-50h-57h . . . 18
5.5 Power Control . . . 18
5.5.1 Detailed Description . . . 18
5.5.2 Function Documentation . . . 18
5.5.2.1 write_socket_power_limit() . . . 18
5.6 Performance (Boost limit) Monitor . . . 19
5.6.1 Detailed Description . . . 19

5.6.2 Function Documentation . . . 19
5.6.2.1 read_esb_boost_limit() . . . 19
5.6.2.2 read_bios_boost_fmax() . . . 20
5.7 Out-of-band Performance (Boost limit) Control . . . 20
5.7.1 Detailed Description . . . 20
5.7.2 Function Documentation . . . 20
5.7.2.1 write_esb_boost_limit() . . . 20
5.7.2.2 write_esb_boost_limit_allcores() . . . 21
5.8 Current, Min, Max TDP . . . 21
5.8.1 Detailed Description . . . 21
5.8.2 Function Documentation . . . 21
5.8.2.1 read_tdp() . . . 21
5.8.2.2 read_max_tdp() . . . 22
5.8.2.3 read_min_tdp() . . . 22
5.9 Prochot . . . 23
5.9.1 Detailed Description . . . 23
5.9.2 Function Documentation . . . 23
5.9.2.1 read_prochot_status() . . . 23
5.9.2.2 read_prochot_residency() . . . 23
5.10 Dram and other features Query . . . 24
5.10.1 Detailed Description . . . 24
5.10.2 Function Documentation . . . 24
5.10.2.1 read_dram_throttle() . . . 24
5.10.2.2 write_dram_throttle() . . . 26
5.10.2.3 read_vddio_mem_power() . . . 26
5.10.2.4 read_nbio_error_logging_register() . . . 27
5.10.2.5 read_iod_bist() . . . 27
5.10.2.6 read_ccd_bist_result() . . . 27
5.10.2.7 read_ccx_bist_result() . . . 28
5.10.2.8 read_cclk_freq_limit() . . . 28
5.10.2.9 read_socket_c0_residency() . . . 29
5.10.2.10 read_ddr_bandwidth() . . . 29
5.11 using CPUID Register Access . . . 30
5.11.1 Detailed Description . . . 30
5.11.2 Function Documentation . . . 30
5.11.2.1 esmi_get_vendor_id() . . . 30
5.11.2.2 esmi_get_processor_info() . . . 30
5.11.2.3 esmi_get_logical_cores_per_socket() . . . 31
5.11.2.4 esmi_get_threads_per_core() . . . 31
5.12 SB_RMI Read Processor Register Access . . . 31
5.12.1 Detailed Description . . . 32
5.12.2 Function Documentation . . . 32

5.12.2.1 esmi_oob_read_msr() . . . 32
5.13 SB-RMI CPUID Register Access . . . 32
5.13.1 Detailed Description . . . 33
5.13.2 Function Documentation . . . 33
5.13.2.1 esmi_oob_cpuid() . . . 33
5.13.2.2 esmi_oob_cpuid_eax() . . . 33
5.13.2.3 esmi_oob_cpuid_ebx() . . . 34
5.13.2.4 esmi_oob_cpuid_ecx() . . . 34
5.13.2.5 esmi_oob_cpuid_edx() . . . 35
5.13.2.6 read_max_threads_per_l3() . . . 35
5.14 SB-RMI Register Read Byte Protocol . . . 36
5.14.1 Detailed Description . . . 37
5.14.2 Function Documentation . . . 38
5.14.2.1 read_sbrmi_alert_status() . . . 38
5.14.2.2 read_sbrmi_alert_mask() . . . 38
5.14.2.3 clear_sbrmi_ras_status() . . . 39
5.14.2.4 esmi_get_threads_per_socket() . . . 39
5.14.2.5 clear_sw_async_alert_status() . . . 39
5.14.2.6 clear_MP0_alert_status() . . . 40
5.14.2.7 set_sw_async_alert_mask() . . . 40
5.14.2.8 read_sbrmi_ready_status() . . . 40
5.14.2.9 clear_shut_down_err() . . . 41
5.14.2.10 gpio_assertion_on_mailbox() . . . 41
5.14.2.11 gpio_assertion_for_async_alerts() . . . 41
5.14.3 Variable Documentation . . . 42
5.14.3.1 thread_en_reg_v10 . . . 42
5.15 SBTSI Register Read Byte Protocol . . . 42
5.15.1 Detailed Description . . . 45
5.15.2 Function Documentation . . . 45
5.15.2.1 read_sbtsi_cpuinttemp() . . . 45
5.15.2.2 read_sbtsi_status() . . . 45
5.15.2.3 read_sbtsi_config() . . . 46
5.15.2.4 read_sbtsi_updaterate() . . . 46
5.15.2.5 write_sbtsi_updaterate() . . . 46
5.15.2.6 read_sbtsi_hitempint() . . . 47
5.15.2.7 read_sbtsi_lotempint() . . . 47
5.15.2.8 read_sbtsi_configwrite() . . . 48
5.15.2.9 read_sbtsi_cputempdecimal() . . . 48
5.15.2.10 read_sbtsi_cputempoffint() . . . 48
5.15.2.11 read_sbtsi_cputempoffdec() . . . 49
5.15.2.12 read_sbtsi_hitempdecimal() . . . 49
5.15.2.13 read_sbtsi_lotempdecimal() . . . 50

5.15.2.14 read_sbtsi_timeoutconfig() . . . 50
5.15.2.15 read_sbtsi_alertthreshold() . . . 50
5.15.2.16 read_sbtsi_alertconfig() . . . 51
5.15.2.17 read_sbtsi_manufid() . . . 51
5.15.2.18 read_sbtsi_revision() . . . 51
5.15.2.19 sbtsi_get_cputemp() . . . 52
5.15.2.20 sbtsi_get_temp_status() . . . 52
5.15.2.21 sbtsi_get_config() . . . 53
5.15.2.22 sbtsi_set_configwr() . . . 53
5.15.2.23 sbtsi_get_timeout() . . . 54
5.15.2.24 sbtsi_set_timeout_config() . . . 54
5.15.2.25 sbtsi_set_hitemp_threshold() . . . 55
5.15.2.26 sbtsi_set_lotemp_threshold() . . . 55
5.15.2.27 sbtsi_get_hitemp_threshold() . . . 55
5.15.2.28 sbtsi_get_lotemp_threshold() . . . 56
5.15.2.29 read_sbtsi_cputempoffset() . . . 56
5.15.2.30 write_sbtsi_cputempoffset() . . . 56
5.15.2.31 sbtsi_set_alert_threshold() . . . 57
5.15.2.32 sbtsi_set_alert_config() . . . 57

6 Data Structure Documentation **59**

6.1 apml_encodings Struct Reference . . . 59
6.1.1 Detailed Description . . . 59
6.2 bmc_ras_runtime_din Struct Reference . . . 59
6.2.1 Detailed Description . . . 60
6.2.2 Field Documentation . . . 60
6.2.2.1 err_category . . . 60
6.2.2.2 number_of_instance . . . 60
6.2.2.3 number_of_offsets . . . 60
6.3 core_floor_limit_din Struct Reference . . . 60
6.3.1 Detailed Description . . . 61
6.4 dimm_pow_din Struct Reference . . . 61
6.4.1 Detailed Description . . . 61
6.5 dimm_pow_dout Struct Reference . . . 61
6.5.1 Detailed Description . . . 61
6.6 dimm_power Struct Reference . . . 62
6.6.1 Detailed Description . . . 62
6.7 dimm_sb_reg_d_in Struct Reference . . . 62
6.7.1 Detailed Description . . . 62
6.8 dimm_sb_reg_write Struct Reference . . . 62
6.8.1 Detailed Description . . . 63
6.9 dimm_spd_d_in Struct Reference . . . 63

6.9.1 Detailed Description . . . 63
6.10 dimm_spd_write Struct Reference . . . 63
6.10.1 Detailed Description . . . 64
6.11 dimm_thermal Struct Reference . . . 64
6.11.1 Detailed Description . . . 64
6.12 dimm_thermal_din Struct Reference . . . 64
6.12.1 Detailed Description . . . 65
6.13 dpm_level Struct Reference . . . 65
6.13.1 Detailed Description . . . 65
6.14 dram_thr_with_status Struct Reference . . . 65
6.14.1 Detailed Description . . . 65
6.15 freq_limits Struct Reference . . . 66
6.15.1 Detailed Description . . . 66
6.16 get_hsmp_cmds_din Struct Reference . . . 66
6.16.1 Detailed Description . . . 66
6.17 get_hsmp_cmds_dout Struct Reference . . . 66
6.17.1 Detailed Description . . . 67
6.18 hbm_device_info Struct Reference . . . 67
6.18.1 Detailed Description . . . 67
6.19 host_status Struct Reference . . . 67
6.19.1 Detailed Description . . . 67
6.20 hottest_dimm_temp_refresh_rate Struct Reference . . . 68
6.20.1 Detailed Description . . . 68
6.21 lclk_dpm_level_range Struct Reference . . . 68
6.21.1 Detailed Description . . . 68
6.22 link_id_bw_type Struct Reference . . . 68
6.22.1 Detailed Description . . . 69
6.22.2 Field Documentation . . . 69
6.22.2.1 link_id . . . 69
6.23 max_ddr_bw Struct Reference . . . 69
6.23.1 Detailed Description . . . 69
6.24 max_mem_bw Struct Reference . . . 69
6.24.1 Detailed Description . . . 70
6.25 mca_bank Struct Reference . . . 70
6.25.1 Detailed Description . . . 70
6.26 mclk_fclk_pstates Struct Reference . . . 70
6.26.1 Detailed Description . . . 70
6.27 nbio_err_log Struct Reference . . . 70
6.27.1 Detailed Description . . . 71
6.28 oob_config_d_in Struct Reference . . . 71
6.28.1 Detailed Description . . . 71
6.28.2 Field Documentation . . . 71

6.28.2.1 mca_oob_misc0_ec_enable . . . 71
6.28.2.2 dram_cecc_oob_ec_mode . . . 71
6.28.2.3 dram_cecc_leak_rate . . . 71
6.28.2.4 pcie_err_reporting_en . . . 71
6.28.2.5 core_mca_err_reporting_en . . . 72
6.29 oob_err_inj_types Struct Reference . . . 72
6.29.1 Detailed Description . . . 72
6.29.2 Field Documentation . . . 72
6.29.2.1 err_types_vendor . . . 72
6.29.2.2 err_size . . . 72
6.30 pci_address Struct Reference . . . 72
6.30.1 Detailed Description . . . 73
6.31 pcie_link_training Struct Reference . . . 73
6.31.1 Detailed Description . . . 73
6.32 pow_eff_mode Struct Reference . . . 73
6.32.1 Detailed Description . . . 74
6.33 processor_info Struct Reference . . . 74
6.33.1 Detailed Description . . . 74
6.34 pstate_freq Struct Reference . . . 74
6.34.1 Detailed Description . . . 74
6.35 ras_df_err_chk Struct Reference . . . 74
6.35.1 Detailed Description . . . 75
6.36 ras_df_err_dump Union Reference . . . 75
6.36.1 Detailed Description . . . 75
6.36.2 Field Documentation . . . 75
6.36.2.1 input . . . 75
6.37 ras_override_delay Struct Reference . . . 75
6.37.1 Detailed Description . . . 76
6.38 ras_rt_err_req_type Struct Reference . . . 76
6.38.1 Detailed Description . . . 76
6.39 ras_rt_valid_err_inst Struct Reference . . . 76
6.39.1 Detailed Description . . . 77
6.40 ras_runtime_response Struct Reference . . . 77
6.40.1 Detailed Description . . . 77
6.40.2 Field Documentation . . . 77
6.40.2.1 buffer_data . . . 77
6.40.2.2 num_of_instances . . . 77
6.40.2.3 buffer_size_per_instance . . . 77
6.40.2.4 op_buffer_size . . . 78
6.41 run_time_err_d_in Struct Reference . . . 78
6.41.1 Detailed Description . . . 78
6.42 run_time_threshold Struct Reference . . . 78

6.42.1 Detailed Description . . . 78
6.42.2 Field Documentation . . . 79
6.42.2.1 err_type . . . 79
6.43 set_hsmp_cmds_din Struct Reference . . . 79
6.43.1 Detailed Description . . . 79
6.44 statistics Struct Reference . . . 79
6.44.1 Detailed Description . . . 79
6.45 svi3_vr_cont_data_in Struct Reference . . . 80
6.45.1 Detailed Description . . . 80
6.45.2 Field Documentation . . . 80
6.45.2.1 rail_mode . . . 80
6.46 svi3_vr_cont_data_out Struct Reference . . . 80
6.46.1 Detailed Description . . . 80
6.47 svi_port_domain Struct Reference . . . 80
6.47.1 Detailed Description . . . 81
6.48 temp_refresh_rate Struct Reference . . . 81
6.48.1 Detailed Description . . . 81
6.49 xgmi_speed_rate_n_width Struct Reference . . . 81
6.49.1 Detailed Description . . . 81

7 File Documentation **83**
7.1 apml.h File Reference . . . 83
7.1.1 Detailed Description . . . 84
7.1.2 Enumeration Type Documentation . . . 84
7.1.2.1 sbrmi_inbnd_msg . . . 84
7.1.3 Function Documentation . . . 84
7.1.3.1 esmi_oob_read_byte() . . . 85
7.1.3.2 esmi_oob_write_byte() . . . 85
7.1.3.3 esmi_oob_rmi_read_byte() . . . 85
7.1.3.4 esmi_oob_tsi_read_byte() . . . 86
7.1.3.5 esmi_oob_rmi_write_byte() . . . 86
7.1.3.6 esmi_oob_tsi_write_byte() . . . 87
7.1.3.7 esmi_oob_read_mailbox() . . . 87
7.1.3.8 esmi_oob_write_mailbox() . . . 88
7.1.3.9 sbrmi_xfer_msg() . . . 88
7.1.3.10 sbtsi_xfer_msg() . . . 88
7.1.3.11 validate_sbtsi_module() . . . 89
7.1.3.12 validate_sbrmi_module() . . . 89
7.1.3.13 validate_apml_dependency() . . . 89
7.2 apml_err.h File Reference . . . 91
7.2.1 Detailed Description . . . 91
7.2.2 Enumeration Type Documentation . . . 92

7.2.2.1 oob_status_t . . . 92
7.3 apml_recovery.h File Reference . . . 92
7.3.1 Detailed Description . . . 93
7.3.2 Function Documentation . . . 93
7.3.2.1 apml_recover_dev() . . . 93
7.4 esmi_cpuid_msr.h File Reference . . . 93
7.4.1 Detailed Description . . . 94
7.4.2 Enumeration Type Documentation . . . 94
7.4.2.1 cpuid_reg . . . 94
7.5 esmi_mailbox.h File Reference . . . 94
7.5.1 Detailed Description . . . 103
7.5.2 Enumeration Type Documentation . . . 103
7.5.2.1 esb_mailbox_commmands . . . 104
7.5.3 Function Documentation . . . 104
7.5.3.1 write_bmc_report_dimm_power() . . . 104
7.5.3.2 write_bmc_report_dimm_thermal_sensor() . . . 104
7.5.3.3 read_bmc_ras_pcie_config_access() . . . 105
7.5.3.4 read_bmc_ras_mca_validity_check() . . . 105
7.5.3.5 read_bmc_ras_mca_msr_dump() . . . 106
7.5.3.6 read_bmc_ras_fch_reset_reason() . . . 106
7.5.3.7 read_dimm_temp_range_and_refresh_rate() . . . 106
7.5.3.8 read_dimm_power_consumption() . . . 107
7.5.3.9 read_dimm_thermal_sensor() . . . 107
7.5.3.10 read_pwr_current_active_freq_limit_socket() . . . 108
7.5.3.11 read_pwr_current_active_freq_limit_core() . . . 108
7.5.3.12 read_pwr_svi_telemetry_all_rails() . . . 109
7.5.3.13 read_socket_freq_range() . . . 109
7.5.3.14 read_current_io_bandwidth() . . . 109
7.5.3.15 read_current_xgmi_bandwidth() . . . 110
7.5.3.16 write_gmi3_link_width_range() . . . 111
7.5.3.17 write_xgmi_link_width_range() . . . 111
7.5.3.18 write_apb_disable() . . . 111
7.5.3.19 write_apb_enable() . . . 112
7.5.3.20 read_current_dfpstate_frequency() . . . 112
7.5.3.21 write_lclk_dpm_level_range() . . . 113
7.5.3.22 read_bmc_rapl_units() . . . 113
7.5.3.23 read_bmc_cpu_base_frequency() . . . 114
7.5.3.24 read_bmc_control_pcie_gen5_rate() . . . 114
7.5.3.25 read_rapl_core_energy_counters() . . . 114
7.5.3.26 read_rapl_pckg_energy_counters() . . . 115
7.5.3.27 write_pwr_efficiency_mode() . . . 115
7.5.3.28 write_df_pstate_range() . . . 116

7.5.3.29 read_lclk_dpm_level_range() . . . 116
7.5.3.30 read_ucode_revision() . . . 117
7.5.3.31 read_ras_df_err_validity_check() . . . 117
7.5.3.32 read_ras_df_err_dump() . . . 117
7.5.3.33 reset_on_sync_flood() . . . 118
7.5.3.34 override_delay_reset_on_sync_flood() . . . 118
7.5.3.35 get_post_code() . . . 119
7.5.3.36 get_bmc_ras_run_time_err_validity_ck() . . . 119
7.5.3.37 get_bmc_ras_run_time_error_info() . . . 120
7.5.3.38 set_bmc_ras_err_threshold() . . . 120
7.5.3.39 set_bmc_ras_oob_config() . . . 121
7.5.3.40 get_bmc_ras_oob_config() . . . 121
7.5.3.41 read_ppin_fuse() . . . 122
7.5.3.42 read_rtc() . . . 122
7.5.3.43 read_dimm_spd_register() . . . 122
7.5.3.44 get_dimm_sb_register() . . . 123
7.5.3.45 write_dimm_spd_register() . . . 123
7.5.3.46 set_dimm_sb_register_data() . . . 123
7.5.3.47 get_dimm_serial_num() . . . 123
7.5.3.48 read_smu_fw_ver() . . . 124
7.5.3.49 write_bmc_pcie_config() . . . 124
7.5.3.50 set_xgmi_pstate_range() . . . 125
7.5.3.51 set_cpu_rail_iso_freq_policy() . . . 125
7.5.3.52 get_cpu_rail_iso_freq_policy() . . . 125
7.5.3.53 set_dfc_enable() . . . 126
7.5.3.54 get_dfc_enable() . . . 126
7.5.3.55 get_ch_dram_thr_with_status() . . . 127
7.5.3.56 get_avg_dram_thr_with_status() . . . 127
7.5.3.57 get_avg_dram_throttle() . . . 128
7.5.3.58 get_ch_dram_throttle() . . . 128
7.5.3.59 get_pc6_enable() . . . 128
7.5.3.60 set_pc6_enable() . . . 129
7.5.3.61 get_cc6_enable() . . . 129
7.5.3.62 set_cc6_enable() . . . 130
7.5.3.63 get_xgmi_link_width_range() . . . 130
7.5.3.64 get_apb_state() . . . 131
7.5.3.65 get_df_pstate_range() . . . 131
7.5.3.66 set_pcie_link_training() . . . 131
7.5.3.67 get_xgmi_pstate_range() . . . 132
7.5.3.68 get_ccd_power_consumption() . . . 132
7.5.3.69 get_tdelta() . . . 133
7.5.3.70 get_svi3_vr_controller_temp_by_rail() . . . 133

7.5.3.71 get_supporting_error_types() . . . 134
7.5.3.72 get_bmc_ras_mca_run_time_error_info() . . . 134
7.5.3.73 get_bmc_ras_dram_ecc_run_time_error_info() . . . 135
7.5.3.74 get_bmc_ras_pcie_run_time_error_info() . . . 135
7.5.3.75 get_floor_core_limit() . . . 136
7.5.3.76 set_core_floor_limit() . . . 136
7.5.3.77 set_floor_limit_for_all_cores() . . . 136
7.5.3.78 set_power_efficiency_mode_selection() . . . 137
7.5.3.79 get_power_efficiency_mode_selection() . . . 137
7.5.3.80 get_effective_floor_freq_per_core() . . . 138
7.5.3.81 soc_dimm_power_limit() . . . 138
7.5.3.82 get_enabled_hsmp_commands() . . . 139
7.5.3.83 set_enabled_hsmp_commands() . . . 139
7.5.3.84 get_dimm_pow_data() . . . 140
7.5.3.85 get_dimm_thermal_sensor_data() . . . 140
7.5.3.86 get_hottest_dimm_temp_range_ref_rate() . . . 141
7.5.3.87 write_pcie_link_control() . . . 141
7.6 esmi_rmi.h File Reference . . . 142
7.6.1 Detailed Description . . . 144
7.7 esmi_tsi.h File Reference . . . 144
7.7.1 Detailed Description . . . 147
7.8 rmi_mailbox_mi300.h File Reference . . . 147
7.8.1 Detailed Description . . . 150
7.8.2 Function Documentation . . . 150
7.8.2.1 set_gfx_core_clock() . . . 150
7.8.2.2 set_mclk_fclk_max_pstate() . . . 150
7.8.2.3 get_mclk_fclk_pstates() . . . 151
7.8.2.4 set_xgmi_pstate() . . . 151
7.8.2.5 unset_xgmi_pstate() . . . 152
7.8.2.6 get_xgmi_pstates() . . . 152
7.8.2.7 get_xcc_idle_residency() . . . 152
7.8.2.8 get_energy_accum_with_timestamp() . . . 153
7.8.2.9 get_alarms() . . . 153
7.8.2.10 get_psn() . . . 154
7.8.2.11 get_link_info() . . . 154
7.8.2.12 get_max_min_gfx_freq() . . . 155
7.8.2.13 get_act_gfx_freq_cap() . . . 155
7.8.2.14 get_svi_rail_telemetry() . . . 155
7.8.2.15 get_die_hotspot_info() . . . 156
7.8.2.16 get_mem_hotspot_info() . . . 156
7.8.2.17 get_host_status() . . . 157
7.8.2.18 get_max_mem_bw_util() . . . 157

7.8.2.19 get_hbm_throttle() . . . 157
7.8.2.20 set_hbm_throttle() . . . 158
7.8.2.21 get_hbm_temperature() . . . 158
7.8.2.22 get_clk_freq_limits() . . . 159
7.8.2.23 get_sockets_in_system() . . . 159
7.8.2.24 get_bist_results() . . . 159
7.8.2.25 get_statistics() . . . 160
7.8.2.26 clear_statistics() . . . 160
7.8.2.27 get_die_type() . . . 161
7.8.2.28 get_curr_xgmi_pstate() . . . 161
7.8.2.29 get_max_operating_temp() . . . 162
7.8.2.30 get_slow_down_temp() . . . 162
7.8.2.31 get_hbm_dev_info() . . . 162
7.8.2.32 get_pciestats() . . . 163
7.9 tsi_mi300.h File Reference . . . 163
7.9.1 Detailed Description . . . 164
7.9.2 Function Documentation . . . 164
7.9.2.1 read_sbtsi_hbm_hi_temp_int_th() . . . 165
7.9.2.2 write_sbtsi_hbm_hi_temp_th() . . . 165
7.9.2.3 read_sbtsi_hbm_hi_temp_dec_th() . . . 165
7.9.2.4 read_sbtsi_hbm_hi_temp_th() . . . 166
7.9.2.5 read_sbtsi_hbm_lo_temp_int_th() . . . 166
7.9.2.6 read_sbtsi_hbm_lo_temp_dec_th() . . . 166
7.9.2.7 write_sbtsi_hbm_lo_temp_th() . . . 167
7.9.2.8 read_sbtsi_max_hbm_temp_int() . . . 167
7.9.2.9 read_sbtsi_max_hbm_temp_dec() . . . 168
7.9.2.10 read_sbtsi_hbm_temp_int() . . . 168
7.9.2.11 read_sbtsi_hbm_temp_dec() . . . 168
7.9.2.12 read_sbtsi_hbm_lo_temp_th() . . . 169
7.9.2.13 read_sbtsi_max_hbm_temp() . . . 169
7.9.2.14 read_sbtsi_hbm_temp() . . . 169
7.9.2.15 read_sbtsi_hbm_alertthreshold() . . . 170
7.9.2.16 sbtsi_set_hbm_alert_threshold() . . . 170
7.9.2.17 get_sbtsi_hbm_alertconfig() . . . 171
7.9.2.18 set_sbtsi_hbm_alertconfig() . . . 171

Index **173**

Chapter 1

Advanced Platform Management Link (APML) Library

1.1 (formerly known as EPYC™ System Management Interface (E-SMI) Out-of-band Library)

The Advanced Platform Management Link (APML) Library library, is part of the EPYC™ System Management Out-of-band software stack. It is a C library for Linux that provides a user space interface to monitor and control the CPU's Systems Management features.

1.2 Important note about Versioning and Backward Compatibility

The APML library is currently under development, and therefore subject to change at the API level. The intention is to keep the API as stable as possible while in development, but in some cases we may need to break backwards compatibility in order to achieve future stability and usability. Following Semantic Versioning rules, while the APML library is in a high state of change, the major version will remain 0, and achieving backward compatibility may not be possible.

Once new development has leveled off, the major version will become greater than 0, and backward compatibility will be enforced between major versions.

1.3 Building APML Library

1.3.0.0.1 Additional Required software for building In order to build the APML library, the following components are required. Note that the software versions listed are what is being used in development. Earlier versions are not guaranteed to work:

- CMake (v3.5.0)
- latex (pdfTeX 3.14159265-2.6-1.40.18)
- apml modules (apml_sbrmi and apml_sbtsi)
  - available at https://github.com/amd/apml_modules/

1.3.0.0.2 Dowloading the source The source code for APML library is available on `Github`.

1.3.0.0.3 Directory stucture of the source Once the APML library source has been cloned to a local Linux machine, the directory structure of source is as below:

- `$ docs/` Contains Doxygen configuration files and Library descriptions
- `$ tool/` Contains apml_tool based on the APML library
- `$ include/esmi_oob` Contains the header files used by the APML library
- `$ src/esmi_oob` Contains library APML source

1.3.0.0.4 Building the library is achieved by following the typical CMake build sequence for native build, as follows.

`$ mkdir -p build`

`$ mkdir -p install`

`$ cd build`

`$ cmake -DCMAKE_INSTALL_PREFIX=${PWD}/install <location of root of APML library CMakeLists.txt>`

`$ make` The built library will appear in the `build` folder.

1.3.0.0.5 Cross compile the library for Target systems Before installing the cross compiler verfiy the target architecture

`$ uname -m` Eg: To cross compile for ARM32 processor:

`$ sudo apt-get install gcc-arm-linux-gnueabihf` Eg: To cross compile for AARCH64 processor: use

`$ sudo apt-get install gcc-aarch64-linux-gnu` NOTE: For cross compilation, cross-$↩ ARCH.cmake file is provided for below Architectures:

- armhf
- aarch64

Compilation steps

$ mkdir -p build

$ cd build

$ cmake -DCMAKE_TOOLCHAIN_FILE=../cross-[arch..].cmake <location of root of APML library CMakeLists.txt>

$ make The built library will appear in the `build` folder. Copy the required binaries and the dynamic linked library to target board(BMC).

$ scp libapml64.so.0 root@10.x.x.x:/usr/lib

$ scp apml_tool root@10.x.x.x:/usr/bin

1.3.0.0.6 Disclaimer

- Input arguments passed by the user are not validated. It might result in unreliable system behavior

1.3.0.0.7 Building the Documentation The documentation PDF file can be built with the following steps (continued from the steps above):

$ make doc The reference manual (APML_Library_Manual.pdf), release notes (APML_Library_Release_↩ Notes.pdf) upon a successful build.

1.4 Usage Basics

Most of the APIs need socket index as the first argument. Refer tools/apml_tool.c

1.5 Usage

1.5.1 Tool Usage

APML tool is a C program based on the APML Library, the executable "apml_tool" will be generated in the build/ folder. This tool provides options to monitor and control System Management functionality.

In execution platform, user can cross-verfiy "apml_sbrmi" and apml_rmi" modules are loaded. The apml modules are open-sourced at https://github.com/amd/apml_modules.git

For detailed usage information, use -h or –help flag:

```
$ ./apml_tool -h
================================ APML System Management Interface ===================================
Usage: ./apml_tool [soc_num] [Option<s> / [--help] [module-name]
Where:  soc_num : socket number 0 or 1
Description:
```

```
./apml_tool -v                          - Displays tool version
./apml_tool [SOC_NUM] --showdependency  - Displays module dependency
./apml_tool --help <MODULE>             - Displays help on the options for the specified module
./apml_tool <option/s>                  - Runs the specified option/s.
Usage: ./apml_tool [soc_num] [Option] params
        MODULES:
        1. mailbox
        2. sbrmi
        3. sbtsi
        4. reg-access
        5. cpuid
        6. recovery
======================================== End of APML SMI ========================================
$ ./apml_tool -v
```

================================== APML System Management Interface ==================================

APML_tool version : X.Y.Z

== End of APML SMI ==

```
Below is a sample usage to get the individual library functionality API's over I2C.
User can pass arguments either any of the ways "./apml_tool [socket_num] -p" or "./apml_tool [socket_num]
       --showpower"
```

1. $ ./apml_tool 0 -p

   ================================== APML System Management Interface ==================================

Power (Watts)	65.029		PowerLimit (Watts)	210.000

1.5.2 | PowerLimitMax (Watts) | 400.000 |

== End of APML SMI ==

1. $ ./apml_tool 1 –setpowerlimit 200000

   ================================== APML System Management Interface ==================================

```
    Set power_limit :          200.000 Watts successfully

    ======================================== End of APML SMI ========================================
```

```
3. $ ./apml_tool 0 --showtsiregisters

    ================================ APML System Management Interface ================================
    --------------------------------------------------------------------------------

                    *** SB-TSI REGISTER SUMMARY ***
    ------------------------------------------------------------------------------
     FUNCTION/Reg Name      | Reg offset    | Hexa(0x)      | Value [Units]
    ------------------------------------------------------------------------------
    _PROCTEMP               |               |               | 55.125 °C
            PROC_INT        | 0x1           | 0x37          | 55 °C
            PROC_DEC        | 0x10          | 0x1           | 0.125 °C
    _STATUS                 | 0x2           |               |
            PROC Temp Alert |               |               | PROC No Temp Alert
            Mem Temp Alert  |               |               | HBM High Temp Alert
    _CONFIG                 | 0x3           |               |
            ALERT_L pin     |               |               | Enabled
            Runstop         |               |               | Comparison Enabled
            Atomic Rd order |               |               | Integer latches Decimal
    _TSI_UPDATERATE         | 0x4           |               | 16.000 Hz
    _HIGH_THRESHOLD_TEMP    |               |               | 70.000 °C
            HIGH_INT        | 0x7           | 0x46          | 70 °C
            HIGH_DEC        | 0x13          | 0x0           | 0.000 °C
    _LOW_THRESHOLD_TEMP     |               |               | 0.000 °C
            LOW_INT         | 0x8           | 0x0           | 0 °C
            LOW_DEC         | 0x14          | 0x0           | 0.000 °C
    _HBM_HIGH_THRESHOLD_TEMP|               |               | 0.000 °C
            HIGH_INT        | 0x40          | 0x0           | 0 °C
            HIGH_DEC        | 0x44          | 0x0           | 0.000 °C
    _HBM_LOW_THRESHOLD_TEMP |               |               | 0.000 °C
            LOW_INT         | 0x48          | 0x0           | 0 °C
```

```
        LOW_DEC         | 0x4c          | 0x0           | 0.000 °C
    _HBM_MAX_TEMP       |               |               | 49.000 °C
        MAX_INT         | 0x50          | 0x31          | 49 °C
        MAX_DEC         | 0x54          | 0x0           | 0.000 °C
    _HBM_TEMP           |               |               | 47.000 °C
        HBM_INT         | 0x5c          | 0x2f          | 47 °C
        HBM_DEC         | 0x60          | 0x0           | 0.000 °C
    _TEMP_OFFSET        |               |               | 0.000 °C
        OFF_INT         | 0x11          | 0x0           | 0 °C
        OFF_DEC         | 0x12          | 0x0           | 0.000 °C
    _THRESHOLD_SAMPLE   | 0x32          |               |
        PROC Alert TH   |               |               | 1
        HBM Alert TH    |               |               | 1
    _TSI_ALERT_CONFIG   | 0xbf          |               |
        PROC Alert CFG  |               |               | Enabled
        HBM Alert CFG   |               |               | Disabled
    _TSI_MANUFACTURE_ID | 0xfe          |               | 0
    _TSI_REVISION       | 0xff          |               | 0x4
    ---------------------------------------------------------------------------

    ========================================= End of APML SMI ==========================================
```

Chapter 2

Module Index

2.1 Modules

Here is a list of all modules:

- Auxiliary functions . . . 15
- SB-RMI Mailbox Service . . . 16
  - Power Monitor . . . 16
    - Platforms . . . 18
  - Power Control . . . 18
  - Performance (Boost limit) Monitor . . . 19
  - Out-of-band Performance (Boost limit) Control . . . 20
  - Current, Min, Max TDP . . . 21
  - Prochot . . . 23
  - Dram and other features Query . . . 24
- using CPUID Register Access . . . 30
- SB_RMI Read Processor Register Access . . . 31
- SB-RMI CPUID Register Access . . . 32
- SB-RMI Register Read Byte Protocol . . . 36
- SBTSI Register Read Byte Protocol . . . 42

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

apml_encodings
: APML link ID encodings for legacy platforms and MI300A. Structure contains the following members. Link ID encoding value and its name. MI300A APML Link ID Encoding: 00000011b: P2 00000100b: P3 00001000b: G0 00001001b: G1 00001010b: G2 00001011b: G3 00001100b↩ : G4 00001101b: G5 00001110b: G6 00001111b: G7 For other platforms the APML Link ID Encoding: 00000001b: P0 00000010b: P1 00000100b: P2 00001000b: P3 00010000b: G0 00100000b: G1 01000000b: G2 10000000b: G3 59

bmc_ras_runtime_din
: Structure representing BMC RAS runtime error information 59

core_floor_limit_din
: Structure for APML floor core limit input 60

dimm_pow_din
: Input for DIMM power, DIMM address and power reporting flag. pow_reporting_flag: Valid values are 0, 1, 2 61

dimm_pow_dout
: DIMM power(mW), update rate(ms) and DIMM address 61

dimm_power
: DIMM power(mW), update rate(ms) and dimm address 62

dimm_sb_reg_d_in
: Dimm_sb_reg_d_in: structure contains following members to be passed as input, to get 4 byte DIMM sb register data 62

dimm_sb_reg_write
: Dimm side band register write: strucutre contains dimm address, LID of device, Register offset, Register space, and data to write 62

dimm_spd_d_in
: Dimm_spd_d_in: strucutre contains following memebers to be passed as input, to get 4 byte DIMM spd register data 63

dimm_spd_write
: SPD side band write: strucutre contains dimm address, LID of device, Register offset, Register space, and data to write 63

dimm_thermal
: DIMM thermal sensor (degree C), update rate and dimm address 64

dimm_thermal_din
: DIMM address and thermal sensor reporting flag. thermal_flag: Valid values are 0 and 1 . . . 64

dpm_level
: Max and min LCK DPM level on a given NBIO ID. Valid Max and min DPM level values are 0 - 1 65

dram_thr_with_status
: Structure for dram throttle percentage with throttle status 65

freq_limits
: Struct containing max frequency and min frequencey limit 66

get_hsmp_cmds_din
Data in for get enabled HSMP commands. structure contains: rmask (read mask indicator : 1 = for read commands, 0 = for write commands), offset for the 24-bitmask segment 66
get_hsmp_cmds_dout
Data out for get enabled HSMP commands. structure contains: offset for the 24-bitmask segment, bitmask (24-bit bitmask in hexa decimal) 66
hbm_device_info
Struct containing device vendor, part number and total memory size in GBs 67
host_status
Struct containing power management controlling status, driving running status 67
hottest_dimm_temp_refresh_rate
Hottest DIMM address, temperature range and refresh rate 68
lclk_dpm_level_range
Max and Min Link frequency clock (LCLK) DPM level on a socket. 8 bit NBIO ID, dpm_level struct containing 8 bit max DPM level, 8 bit min DPM level 68
link_id_bw_type
APML LINK ID and Bandwidth type Information.It contains APML LINK ID Encoding. SP7 platforms valid Link ID encodings are 1(P0), 2(P1), 4(P2), 8(P3), 16(G0), 32(G1), 64(G2), 128(G3), 256(P4) and 512(P5) and rest of the Non-MI300 Platforms Valid Link ID encodings are 1(P0), 2(P1), 4(P2), 8(P3), 16(G0), 32(G1), 64(G2), 128(G3). Valid APML MI300 APML LINK ID Encoding. Valid Link ID encodings are 3(P2), 4(P3), 8(G0), 9(G1), 10(G2), 11(G3), 12(G4), 13(G5), 14(G6), 15(G7). IO Bandwidth types 1(Aggregate_BW), 2 (Read BW), 4 (Write BW) 68
max_ddr_bw
Structure for Max DDR bandwidth and utilization. It contains max bandwidth(12 bit data) in GBps, current utilization bandwidth(12 bit data) in GBps, current utilized bandwidth(8 bit data) in percentage 69
max_mem_bw
Structure for Max DDR/HBM bandwidth and utilization. It contains max bandwidth(16 bit data) in GBps, current utilization bandwidth(Read+Write)(16 bit data) in GBps 69
mca_bank
MCA bank information.It contains 16 bit Index for MCA Bank and 16 bit offset 70
mclk_fclk_pstates
Struct containing memory clock and fabric clock pstate mappings 70
nbio_err_log
NBIO quadrant(8 bit data) and NBIO register offset(24 bit) data 70
oob_config_d_in
Configure oob state infrastructure in SoC. structure consists of mca_oob_misc0_ec_enable, dram_cecc_oob_ec_mode, dram_cecc_leak_rate, pcie_err_reporting_en and mca_err_↩ reporting_en 71
oob_err_inj_types
Error injection types containing generic error types, vendor specfic error types, error size and vendor flag 72
pci_address
PCI address information .PCI address includes 4 bit segment, 12 bit aligned offset, 8 bit bus, 5 bit device info and 3 bit function 72
pcie_link_training
PCIe Halt Link Training information. It includes 10 bit link ID, 4 bit Pcie Port number range from 0 - 7 73
pow_eff_mode
Structure for Power Efficiency Mode selection 73
processor_info
Read Proccessor Info 74
pstate_freq
DF P-state frequency.It includes mem clock(16 bit data) frequency (DRAM memory clock), data fabric clock (12 bit data), UMC clock divider (UMC) (1 bit data) 74

ras_df_err_chk
RAS df err validity check output status. Structure contains the following members. df_block_↩ instances number of block instance with error log to report (0 - 256) err_log_len length of error log in bytes per instance (0 - 256) . . . 74
ras_df_err_dump
RAS df error dump input. Union contains the following members. input[0] 4 byte alligned offset in error log (0 - 255) input[1] DF block ID (0 - 255) input[2] Zero based index of DF block instance (0 - 255) input[3] Reserved data_in 32-bit data input . . . 75
ras_override_delay
BMC RAS override delay reset CPU on sync flood. The structure contains delay value override in mins [5 -120 mins], disable delay counter and stop delay counter. If disable delay counter is set ResetCpuOnSyncFlood response will NOT be delayed in the next SyncFlood regardless of the value specified in delay_val_override. If StopDelayCounter is set it stops the active delay countdown which extends the DelayResetCpuOnSyncFlood indefinitely and system will not reset 75
ras_rt_err_req_type
Get the Error type and request type. Supported Runtime error type[1:0]: . . . 76
ras_rt_valid_err_inst
Number of valid error instance per category. It consists of number of bytes per each category and number of instances of each category . . . 76
ras_runtime_response
Structure representing the response from a RAS runtime operation . . . 77
run_time_err_d_in
Get run time error information data_in. Runtime error data_in includes 4 byte aligned offset in instance, runtime error category and zero based index of valid instance number of bytes per each category and number of instances of each category . . . 78
run_time_threshold
Configure threshold counters for MCA, DRAM CECC and PCIE. structure consists of error type, error count threshold and max interrupt rate . . . 78
set_hsmp_cmds_din
Data in for set enabled HSMP commands. structure contains: rmask (read mask indicator : 1 = for read commands, 0 = for write commands), offset for the 24-bitmask segment. bitmask (24-bit bitmask in hexa decimal) . . . 79
statistics
Struct containing statistics parameter of interest and output control pstate mappings . . . 79
svi3_vr_cont_data_in
Data in for the SVI3 VR controller temperature by rail . strucutre contains rail mode and svi3 rail index . . . 80
svi3_vr_cont_data_out
Data in for the SVI3 VR controller temperature by rail . strucutre contains read svi3 temp data bit and svi3 rail index . . . 80
svi_port_domain
Struct containing port and slave address . . . 80
temp_refresh_rate
DIMM temperature range and refresh rate, temperature update flag . . . 81
xgmi_speed_rate_n_width
Struct containing xgmi speed rate in MHZ and link width in units of Gpbs. If link_width[0] = 1 then XGMI link X2 is supported. If link_width[1] = 1 then XGMI link X4 is supported. If Link_width[2] = 1 then XGMI link X4 is supported and similarly if link_width[3] = 1 then XGMI link X8 is supported 81

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

apml.h 83
apml_common.h . . . . . **??**
apml_err.h 91
apml_recovery.h 92
esmi_cpuid_msr.h 93
esmi_mailbox.h 94
esmi_rmi.h 142
esmi_tsi.h 144
rmi_mailbox_mi300.h 147
tsi_mi300.h 163

Chapter 5

Module Documentation

5.1 Auxiliary functions

Functions

- oob_status_t errno_to_oob_status (int err)

  convert linux error to esmi error.
- char * esmi_get_err_msg (oob_status_t oob_err)

  Get the error string message for esmi oob errors.

5.1.1 Detailed Description

Below functions provide interfaces to get the total number of cores, sockets and threads per core in the system.

5.1.2 Function Documentation

5.1.2.1 errno_to_oob_status()

```
oob_status_t errno_to_oob_status (
            int err )
```

convert linux error to esmi error.
Get the appropriate esmi error for linux error.

Parameters

in	*err*	a linux error number

Return values

oob_status_t	is returned upon particular esmi error

5.1.2.2 esmi_get_err_msg()

```
char* esmi_get_err_msg (
            oob_status_t oob_err )
```

Get the error string message for esmi oob errors.
Get the error message for the esmi oob error numbers

Parameters

in	*oob_err*	is a esmi oob error number

Return values

char∗	value returned upon successful call.

5.2 SB-RMI Mailbox Service

Modules

- Power Monitor
- Power Control
- Performance (Boost limit) Monitor
- Out-of-band Performance (Boost limit) Control
- Current, Min, Max TDP
- Prochot
- Dram and other features Query

5.2.1 Detailed Description

write, 'write and read' operations for a given socket.

5.3 Power Monitor

Modules

- Platforms

Functions

- oob_status_t read_socket_power (uint8_t soc_die_num, uint32_t ∗buffer)

  Get the power consumption of the socket.
- oob_status_t read_socket_power_limit (uint8_t soc_die_num, uint32_t ∗buffer)

  Get the current power cap/limit value for a given socket.
- oob_status_t read_max_socket_power_limit (uint8_t soc_die_num, uint32_t ∗buffer)

  Get the maximum value that can be assigned as a power cap/limit for a given socket.

5.3.1 Detailed Description

Below functions provide interfaces to get the current power usage and Power Limits for a given socket.

5.3.2 Function Documentation

5.3.2.1 read_socket_power()

```
oob_status_t read_socket_power (
            uint8_t soc_die_num,
            uint32_t * buffer )
```

Get the power consumption of the socket.

Given Socket Die number and a pointer to a uint32_t `buffer`, this function will get the current power consumption (in watts) to the uint32_t pointed to by `buffer`. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

`in`	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
`in,out`	*buffer*	a pointer to uint32_t value of power consumption

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.3.2.2 read_socket_power_limit()

```
oob_status_t read_socket_power_limit (
            uint8_t soc_die_num,
            uint32_t * buffer )
```

Get the current power cap/limit value for a given socket.
This function will return the valid power cap `buffer` for a given socket, this value will be used for the system to limit the power. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

`in`	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
`in,out`	*buffer*	a pointer to a uint32_t that indicates the valid possible power cap/limit, in watts

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.3.2.3 read_max_socket_power_limit()

```
oob_status_t read_max_socket_power_limit (
            uint8_t soc_die_num,
            uint32_t * buffer )
```

Get the maximum value that can be assigned as a power cap/limit for a given socket.
This function will return the maximum possible valid power cap/limit Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

`in`	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
`out`	*buffer*	a pointer to a uint32_t that indicates the maximum possible power cap/limit, in watts

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.4 Platforms

Supporting Platforms: Fam-19h_Mod-00h-0Fh Fam-19h_Mod-10h-1Fh Fam-19h_Mod-90h-9Fh Fam-1Ah_Mod-00h-0Fh Fam-1Ah_Mod-50h-57h

5.4.1 Fam-19h_Mod-00h-0Fh

Platform with Family 19h and model in range 00h - 0Fh

5.4.2 Fam-19h_Mod-10h-1Fh

Platform with Family 19h and model in range 10h - 1Fh

5.4.3 Fam-19h_Mod-90h-9Fh

Platform with Family 19h and model in range 90h - 9Fh

5.4.4 Fam-1Ah_Mod-00h-0Fh

Platform with Family 1Ah and model in range 00h - 0Fh

5.4.5 Fam-1Ah_Mod-50h-57h

Platform with Family 1Ah and model in range 50h - 5Fh

5.5 Power Control

Functions

- oob_status_t write_socket_power_limit (uint8_t soc_die_num, uint32_t limit)

  Set the power cap/limit value for a given socket.

5.5.1 Detailed Description

This function provides a way to control Power Limit.

5.5.2 Function Documentation

5.5.2.1 write_socket_power_limit()

```
oob_status_t write_socket_power_limit (
            uint8_t soc_die_num,
            uint32_t limit )
```

Set the power cap/limit value for a given socket.
This function will set the power cap/limit Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*limit*	uint32_t that indicates the desired power cap/limit, in milliwatts

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.6 Performance (Boost limit) Monitor

Functions

- oob_status_t read_esb_boost_limit (uint8_t soc_die_num, uint32_t value, uint32_t *buffer)

  Get the Out-of-band boostlimit value for a given core.
- oob_status_t read_bios_boost_fmax (uint8_t soc_die_num, uint32_t value, uint32_t *buffer)

  Get the In-band maximum boostlimit value for a given core.

5.6.1 Detailed Description

This function provides the current boostlimit value for a given core.

5.6.2 Function Documentation

5.6.2.1 read_esb_boost_limit()

```
oob_status_t read_esb_boost_limit (
            uint8_t soc_die_num,
            uint32_t value,
            uint32_t * buffer )
```

Get the Out-of-band boostlimit value for a given core.
This function will return the core's current Out-of-band boost limit `buffer` for a particular `value` Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*value*	a cpu index
in,out	*buffer*	pointer to a uint32_t that indicates the possible boost limit value

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.6.2.2 read_bios_boost_fmax()

```
oob_status_t read_bios_boost_fmax (
            uint8_t soc_die_num,
            uint32_t value,
            uint32_t * buffer )
```

Get the In-band maximum boostlimit value for a given core.
This function will return the core's current maximum In-band boost limit `buffer` for a particular `value` is cpu_ind Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.

Parameters

`in`	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
`in`	*value*	is a cpu index
`in,out`	*buffer*	a pointer to a uint32_t that indicates the maximum boost limit value set via In-band

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.7 Out-of-band Performance (Boost limit) Control

Functions

- oob_status_t write_esb_boost_limit (uint8_t soc_die_num, uint32_t cpu_ind, uint32_t limit)

  Set the Out-of-band boostlimit value for a given core.
- oob_status_t write_esb_boost_limit_allcores (uint8_t soc_die_num, uint32_t limit)

  Set the boostlimit value for the whole socket (whole system).

5.7.1 Detailed Description

Below functions provide ways to control the Out-of-band Boost limit values.

5.7.2 Function Documentation

5.7.2.1 write_esb_boost_limit()

```
oob_status_t write_esb_boost_limit (
            uint8_t soc_die_num,
            uint32_t cpu_ind,
            uint32_t limit )
```

Set the Out-of-band boostlimit value for a given core.
This function will set the boostlimit to the provided value `limit` for a given cpu. NOTE: Currently the limit is setting for all the cores instead of a particular cpu. Testing in Progress. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

`in`	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
`in`	*cpu_ind*	a cpu index is a given core to set the boostlimit
`in`	*limit*	a uint32_t that indicates the desired Out-of-band boostlimit value of a given core

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.7.2.2 write_esb_boost_limit_allcores()

```
oob_status_t write_esb_boost_limit_allcores (
            uint8_t soc_die_num,
            uint32_t limit )
```

Set the boostlimit value for the whole socket (whole system).
This function will set the boostlimit to the provided value `boostlimit` for the socket. Supported platforms : Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

`in`	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
`in`	*limit*	a uint32_t that indicates the desired boostlimit value of the socket

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.8 Current, Min, Max TDP

Functions

- oob_status_t read_tdp (uint8_t soc_die_num, uint32_t *buffer)

  Get the Thermal Design Power limit TDP of the socket with provided Socket Die index.
- oob_status_t read_max_tdp (uint8_t soc_die_num, uint32_t *buffer)

  Get the Maximum Thermal Design Power limit TDP of the socket with provided Socket Die index.
- oob_status_t read_min_tdp (uint8_t soc_die_num, uint32_t *buffer)

  Get the Minimum Thermal Design Power limit TDP of the socket.

5.8.1 Detailed Description

Below functions provide interfaces to get the current, Min and Max TDP, Prochot and Prochot Residency for a given socket.

5.8.2 Function Documentation

5.8.2.1 read_tdp()

```
oob_status_t read_tdp (
            uint8_t soc_die_num,
            uint32_t * buffer )
```

Get the Thermal Design Power limit TDP of the socket with provided Socket Die index.

Given a socket and a pointer to a uint32_t `buffer`, this function will get the current TDP (in milliwatts) Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

`in`	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
`in,out`	*buffer*	a pointer to uint32_t to which the Current TDP value will be copied

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.8.2.2 read_max_tdp()

```
oob_status_t read_max_tdp (
            uint8_t soc_die_num,
            uint32_t * buffer )
```

Get the Maximum Thermal Design Power limit TDP of the socket with provided Socket Die index.
Given a socket and a pointer, this function will get the Maximum TDP (watts) Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

`in`	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
`in,out`	*buffer*	a pointer to uint32_t to which the Maximum TDP value will be copied

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.8.2.3 read_min_tdp()

```
oob_status_t read_min_tdp (
            uint8_t soc_die_num,
            uint32_t * buffer )
```

Get the Minimum Thermal Design Power limit TDP of the socket.
Given a socket and a pointer to a uint32_t, this function will get the Minimum TDP (watts) Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

`in`	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
`in,out`	*buffer*	a pointer to uint32_t to which the Minimum TDP value will be copied

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.9 Prochot

Functions

- oob_status_t read_prochot_status (uint8_t soc_die_num, uint32_t *buffer)

  Get the Prochot Status of the socket with provided Socket Die index.
- oob_status_t read_prochot_residency (uint8_t soc_die_num, float *buffer)

  Get the Prochot Residency (since the boot time or last read of Prochot Residency) of the socket.

5.9.1 Detailed Description

Below functions provide interfaces to get Prochot and Prochot Residency for a given socket.

5.9.2 Function Documentation

5.9.2.1 read_prochot_status()

```
oob_status_t read_prochot_status (
            uint8_t soc_die_num,
            uint32_t * buffer )
```

Get the Prochot Status of the socket with provided Socket Die index.
Given a socket and a pointer to a uint32_t, this function will get the Prochot status as active/1 or inactive/0 Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*buffer*	a pointer to uint32_t to which the Prochot status will be copied

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.9.2.2 read_prochot_residency()

```
oob_status_t read_prochot_residency (
            uint8_t soc_die_num,
            float * buffer )
```

Get the Prochot Residency (since the boot time or last read of Prochot Residency) of the socket.
Given a socket and a pointer to a uint32_t, this function will get the Prochot residency as a percentage Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*buffer*	a pointer to float to which the Prochot residency will be copied

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.10 Dram and other features Query

Functions

- oob_status_t read_dram_throttle (uint8_t soc_die_num, uint32_t *buffer)

  Read Dram Throttle will always read the highest percentage value.
- oob_status_t write_dram_throttle (uint8_t soc_die_num, uint32_t limit)

  Set Dram Throttle value in terms of percentage.
- oob_status_t read_vddio_mem_power (uint8_t soc_die_num, uint32_t *buffer)

  Read VDDIOMem Power returns the estimated VDDIOMem power consumed within the socket.
- oob_status_t read_nbio_error_logging_register (uint8_t soc_die_num, struct nbio_err_log nbio, uint32_t *buffer)

  Read NBIO Error Logging Register.
- oob_status_t read_iod_bist (uint8_t soc_die_num, uint32_t *buffer)

  Read IOD Bist status.
- oob_status_t read_ccd_bist_result (uint8_t soc_die_num, uint32_t input, uint32_t *buffer)

  Read CCD Bist status. Results are read for each CCD present in the system.
- oob_status_t read_ccx_bist_result (uint8_t soc_die_num, uint32_t value, uint32_t *ccx_bist)

  Read CPU Core Complex Bist result. results are read for each Logical CCX instance number and returns a value which is the concatenation of L3 pass status and all cores in the complex(n:0).
- oob_status_t read_cclk_freq_limit (uint8_t soc_die_num, uint32_t *cclk_freq)

  Read CCLK frequency limit for the given socket.
- oob_status_t read_socket_c0_residency (uint8_t soc_die_num, uint32_t *c0_res)

  Read socket C0 residency.
- oob_status_t read_ddr_bandwidth (uint8_t soc_die_num, struct max_ddr_bw *max_ddr)

  Get the Theoretical maximum DDR Bandwidth of the system in GB/s, Current utilized DDR Bandwidth (Read + Write) in GB/s and Current utilized DDR Bandwidth as a percentage of theoretical maximum. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.

5.10.1 Detailed Description

5.10.2 Function Documentation

5.10.2.1 read_dram_throttle()

```
oob_status_t read_dram_throttle (
            uint8_t soc_die_num,
            uint32_t * buffer )
```

Read Dram Throttle will always read the highest percentage value.

This function will always read the highest percentage value as represented by PROCHOT throttle or write dram throttle. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	is to read the dram throttle in % (0 - 100).

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.10.2.2 write_dram_throttle()

```
oob_status_t write_dram_throttle (
            uint8_t soc_die_num,
            uint32_t limit )
```

Set Dram Throttle value in terms of percentage.

This function will set the dram throttle of the provided value limit for the given socket. Supported platforms↩ : Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*limit*	that indicates the desired limit as per SSP PPR write can be between 0 to 80% to for a given socket

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.10.2.3 read_vddio_mem_power()

```
oob_status_t read_vddio_mem_power (
            uint8_t soc_die_num,
            uint32_t * buffer )
```

Read VDDIOMem Power returns the estimated VDDIOMem power consumed within the socket.

This function will read VDDIOMem Power for the given socket Supported platforms: Fam-19h_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	is to read VDDIOMem Power.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.10.2.4 read_nbio_error_logging_register()

```
oob_status_t read_nbio_error_logging_register (
            uint8_t soc_die_num,
            struct nbio_err_log nbio,
            uint32_t * buffer )
```

Read NBIO Error Logging Register.

Given a socket, quadrant and register offset as `input`, this function will read NBIOErrorLogging↩ Register. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*nbio*	nbio_err_log Struct containing nbio quadrant and offset.
out	*buffer*	is to read NBIOErrorLoggingRegiter(register value).

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.10.2.5 read_iod_bist()

```
oob_status_t read_iod_bist (
            uint8_t soc_die_num,
            uint32_t * buffer )
```

Read IOD Bist status.

This function will read IOD Bist result for the given socket. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	is to read IODBistResult 0 = Bist pass, 1 = Bist fail

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.10.2.6 read_ccd_bist_result()

```
oob_status_t read_ccd_bist_result (
            uint8_t soc_die_num,
            uint32_t input,
            uint32_t * buffer )
```

Read CCD Bist status. Results are read for each CCD present in the system.

Given a socket bus number and address, Logical CCD instance number as `input`, this function will read CCDBistResult. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*input*	is a Logical CCD instance number.
out	*buffer*	is to read CCDBistResult 0 = Bist pass, 1 = Bist fail

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.10.2.7 read_ccx_bist_result()

```
oob_status_t read_ccx_bist_result (
            uint8_t soc_die_num,
            uint32_t value,
            uint32_t * ccx_bist )
```

Read CPU Core Complex Bist result. results are read for each Logical CCX instance number and returns a value which is the concatenation of L3 pass status and all cores in the complex(n:0).
Given a socket bus number, address, Logical CCX instance number as `input`, this function will read CCXBistResult. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh, Fam-1Ah_Mod-50h-57h and Fam-1Ah_Mod-50h-57h. For Fam-1Ah_Mod-50h-57h the Bit[31] of input will be 1 (to get the L3 bist) or 0 (to get core bist), Bits[8 - 30] are reserved and Bits [0 - 7] will be logical ccx instance. For the rest of the platforms the input is only CCX instance number.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*value*	For Fam-1Ah_Mod-50h-57h the Bit[31] of input will be 1 (to get the L3 bist) or 0 (to get core bist), Bits[8 - 30] are reserved and Bits [0 - 7] will be logical ccx instance number. For the rest of the platforms value is only CCX instance number. Valid values [0, (k -1)] where k is the number of logical CCX instances.
out	*ccx_bist*	result is concatenation of bist results for all cores[31:16] in the complex(n:0) L3 bist[15:0], where n num of cores in CCX.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.10.2.8 read_cclk_freq_limit()

```
oob_status_t read_cclk_freq_limit (
            uint8_t soc_die_num,
            uint32_t * cclk_freq )
```

Read CCLK frequency limit for the given socket.
This function will read CPU core clock frequency limit for the given socket. Supported platforms↩ : Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*cclk_freq*	CPU core clock frequency limit [MHz]

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.10.2.9 read_socket_c0_residency()

```
oob_status_t read_socket_c0_residency (
            uint8_t soc_die_num,
            uint32_t * c0_res )
```

Read socket C0 residency.

This function will provides average C0 residency across all cores in the socket. 100% specifies that all enabled cores in the socket are runningin C0. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*c0_res*	is to read Socket C0 residency[%].

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.10.2.10 read_ddr_bandwidth()

```
oob_status_t read_ddr_bandwidth (
            uint8_t soc_die_num,
            struct max_ddr_bw * max_ddr )
```

Get the Theoretical maximum DDR Bandwidth of the system in GB/s, Current utilized DDR Bandwidth (Read + Write) in GB/s and Current utilized DDR Bandwidth as a percentage of theoretical maximum. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*max_ddr*	max_ddr_bw struct containing max bandwidth, utilized bandwidth and utilized bandwidth percentage.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.11 using CPUID Register Access

Functions

- oob_status_t esmi_get_vendor_id (uint8_t soc_die_num, char *vendor_id)

  Get the number of logical cores per socket.
- oob_status_t esmi_get_processor_info (uint8_t soc_die_num, struct processor_info *proc_info)

  Get the number of logical cores per socket.
- oob_status_t esmi_get_logical_cores_per_socket (uint8_t soc_die_num, uint32_t *logical_cores_per_↩ socket)

  Get the number of logical cores per socket.
- oob_status_t esmi_get_threads_per_core (uint8_t soc_die_num, uint32_t *threads_per_core)

  Get number of threads per core.

5.11.1 Detailed Description

Below function provide interface to read the processor info using CPUID register. output from commmand will be written into the buffer.

5.11.2 Function Documentation

5.11.2.1 esmi_get_vendor_id()

```
oob_status_t esmi_get_vendor_id (
            uint8_t soc_die_num,
            char * vendor_id )
```

Get the number of logical cores per socket.
Get the processor vendor Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*vendor_id*	to get the processor vendor, 12 byte RO value

Return values

uint32_t	is returned upon successful call.

5.11.2.2 esmi_get_processor_info()

```
oob_status_t esmi_get_processor_info (
            uint8_t soc_die_num,
            struct processor_info * proc_info )
```

Get the number of logical cores per socket.
Get the effective family, model and step_id of the processor. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*proc_info*	to get family, model & stepping identifier

Return values

uint32_t	is returned upon successful call.

5.11.2.3 esmi_get_logical_cores_per_socket()

```
oob_status_t esmi_get_logical_cores_per_socket (
            uint8_t soc_die_num,
            uint32_t * logical_cores_per_socket )
```

Get the number of logical cores per socket.

Get the total number of logical cores in a socket. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*logical_cores_per_socket*	is returned

Return values

logical_cores_per_socket	is returned upon successful call.

5.11.2.4 esmi_get_threads_per_core()

```
oob_status_t esmi_get_threads_per_core (
            uint8_t soc_die_num,
            uint32_t * threads_per_core )
```

Get number of threads per core.

Get the number of threads per core. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*threads_per_core*	is returned

Return values

threads_per_core	is returned upon successful call.

5.12 SB_RMI Read Processor Register Access

Functions

- oob_status_t esmi_oob_read_msr (uint8_t soc_die_num, uint32_t thread, uint32_t msraddr, uint64_t *buffer)

  Read the MCA MSR register for a given thread.

5.12.1 Detailed Description

Below function provide interface to read the SB-RMI MCA MSR register. output from MCA MSR commmand will be written into the buffer.

5.12.2 Function Documentation

5.12.2.1 esmi_oob_read_msr()

```
oob_status_t esmi_oob_read_msr (
            uint8_t soc_die_num,
            uint32_t thread,
            uint32_t msraddr,
            uint64_t * buffer )
```

Read the MCA MSR register for a given thread.
Given a `thread` and SB-RMI register command, this function reads msr value.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*thread*	is a particular thread in the system.
in	*msraddr*	MCA MSR register to read
out	*buffer*	is to hold the return output of msr value.

Return values

OOB_SUCCESS	is returned upon successful call.
None-zero	is returned upon failure.

5.13 SB-RMI CPUID Register Access

Functions

- oob_status_t esmi_oob_cpuid (uint8_t soc_die_num, uint32_t thread, uint32_t *eax, uint32_t *ebx, uint32_t *ecx, uint32_t *edx)

  Read CPUID functionality for a particular thread in a system.
- oob_status_t esmi_oob_cpuid_eax (uint8_t soc_die_num, uint32_t thread, uint32_t fn_eax, uint32_t fn_ecx, uint32_t *eax)

  Read eax register on CPUID functionality.
- oob_status_t esmi_oob_cpuid_ebx (uint8_t soc_die_num, uint32_t thread, uint32_t fn_eax, uint32_t fn_ecx, uint32_t *ebx)

  Read ebx register on CPUID functionality.
- oob_status_t esmi_oob_cpuid_ecx (uint8_t soc_die_num, uint32_t thread, uint32_t fn_eax, uint32_t fn_ecx, uint32_t *ecx)

  Read ecx register on CPUID functionality.
- oob_status_t esmi_oob_cpuid_edx (uint8_t soc_die_num, uint32_t thread, uint32_t fn_eax, uint32_t fn_ecx, uint32_t *edx)

  Read edx register on CPUID functionality.
- oob_status_t read_max_threads_per_l3 (uint8_t soc_die_num, uint32_t *threads_l3)

  Read max threads per L3 cache.

5.13.1 Detailed Description

Below function provide interface to get the CPUID access via the SBRMI.
Output from CPUID commmand will be written into registers eax, ebx, ecx and edx.

5.13.2 Function Documentation

5.13.2.1 esmi_oob_cpuid()

```
oob_status_t esmi_oob_cpuid (
             uint8_t soc_die_num,
             uint32_t thread,
             uint32_t * eax,
             uint32_t * ebx,
             uint32_t * ecx,
             uint32_t * edx )
```

Read CPUID functionality for a particular thread in a system.
Given a `thread` , `eax` as function input and `ecx` as extended function input. this function will get the cpuid details for a particular thread in a pointer to `eax`, `ebx`, `ecx`, `edx`

Parameters

`in`	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
`in`	*thread*	is a particular thread in the system.
`in,out`	*eax*	a pointer uint32_t to get eax value
`out`	*ebx*	a pointer uint32_t to get ebx value
`in,out`	*ecx*	a pointer uint32_t to get ecx value
`out`	*edx*	a pointer uint32_t to get edx value

Return values

OOB_SUCCESS	is returned upon successful call.
None-zero	is returned upon failure.

5.13.2.2 esmi_oob_cpuid_eax()

```
oob_status_t esmi_oob_cpuid_eax (
             uint8_t soc_die_num,
             uint32_t thread,
             uint32_t fn_eax,
             uint32_t fn_ecx,
             uint32_t * eax )
```

Read eax register on CPUID functionality.
Given a `thread`, `fn_eax` as function and `fn_ecx` as extended function input, this function will get the cpuid details for a particular thread at `eax`.

Parameters

`in`	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
`in`	*thread*	is a particular thread in the system.
`in`	*fn_eax*	cpuid function
`in`	*fn_ecx*	cpuid extended function

Parameters

out	*eax*	is to read eax from cpuid functionality.

Return values

OOB_SUCCESS	is returned upon successful call.
None-zero	is returned upon failure.

5.13.2.3 esmi_oob_cpuid_ebx()

```
oob_status_t esmi_oob_cpuid_ebx (
            uint8_t soc_die_num,
            uint32_t thread,
            uint32_t fn_eax,
            uint32_t fn_ecx,
            uint32_t * ebx )
```

Read ebx register on CPUID functionality.
Given a `thread`, `fn_eax` as function and `fn_ecx` as extended function input, this function will get the cpuid details for a particular thread at `ebx`.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*thread*	is a particular thread in the system.
in	*fn_eax*	cpuid function
in	*fn_ecx*	cpuid extended function
out	*ebx*	is to read ebx from cpuid functionality.

Return values

OOB_SUCCESS	is returned upon successful call.
None-zero	is returned upon failure.

5.13.2.4 esmi_oob_cpuid_ecx()

```
oob_status_t esmi_oob_cpuid_ecx (
            uint8_t soc_die_num,
            uint32_t thread,
            uint32_t fn_eax,
            uint32_t fn_ecx,
            uint32_t * ecx )
```

Read ecx register on CPUID functionality.
Given a `thread`, `fn_eax` as function and `fn_ecx` as extended function input, this function will get the cpuid details for a particular thread at `ecx`.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*thread*	is a particular thread in the system.

Parameters

in	*fn_eax*	cpuid function
in	*fn_ecx*	cpuid extended function
out	*ecx*	is to read ecx from cpuid functionality.

Return values

OOB_SUCCESS	is returned upon successful call.
None-zero	is returned upon failure.

5.13.2.5 esmi_oob_cpuid_edx()

```
oob_status_t esmi_oob_cpuid_edx (
            uint8_t soc_die_num,
            uint32_t thread,
            uint32_t fn_eax,
            uint32_t fn_ecx,
            uint32_t * edx )
```

Read edx register on CPUID functionality.
Given a `thread`, `fn_eax` as function and `fn_ecx` as extended function input, this function will get the cpuid details for a particular thread at `edx`.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*thread*	is a particular thread in the system.
in	*fn_eax*	cpuid function
in	*fn_ecx*	cpuid extended function
out	*edx*	is to read edx from cpuid functionality.

Return values

OOB_SUCCESS	is returned upon successful call.
None-zero	is returned upon failure.

5.13.2.6 read_max_threads_per_l3()

```
oob_status_t read_max_threads_per_l3 (
            uint8_t soc_die_num,
            uint32_t * threads_l3 )
```

Read max threads per L3 cache.
Reads max threads per L3 cache.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*threads_l3*	threads per L3 cache.

Return values

OOB_SUCCESS	is returned upon successful call.
None-zero	is returned upon failure.

5.14 SB-RMI Register Read Byte Protocol

Functions

- oob_status_t read_sbrmi_revision (uint8_t soc_die_num, uint8_t *buffer)

  Read revision from SB_RMI register command. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_control (uint8_t soc_die_num, uint8_t *buffer)

  Read Control byte from SB_RMI register command. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_status (uint8_t soc_die_num, uint8_t *buffer)

  Read one byte of Status value from SB_RMI register command. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_readsize (uint8_t soc_die_num, uint8_t *buffer)

  This register specifies the number of bytes to return when using the block read protocol to read SBRMI_x[4F:10]. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_threadenablestatus (uint8_t soc_die_num, uint8_t *buffer)

  Read one byte of Thread Status from SB_RMI register command. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_multithreadenablestatus (uint8_t soc_die_num, uint8_t *buffer)

  Read one byte of Thread Status from SB_RMI register command. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_swinterrupt (uint8_t soc_die_num, uint8_t *buffer)

  This register is used by the SMBus master to generate an interrupt to the processor to indicate that a message is available.. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_threadnumber (uint8_t soc_die_num, uint8_t *buffer)

  This register indicates the maximum number of threads present. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_mp0_msg (uint8_t soc_die_num, uint8_t *buffer)

  This register will read the message running on the MP0. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_alert_status (uint8_t soc_die_num, uint8_t num_of_alert_status_reg, uint8_t **buffer)

  This function will read bit vector for all the threads. Value of 1 indicates MCE occured for the thread and is set by hardware. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_alert_mask (uint8_t soc_die_num, uint8_t num_of_alert_mask_reg, uint8_t **buffer)

  This function will read bit vector for all the threads. Value of 1 indicates alert signaling disabled for corresponding SBRMI::AlertStatus[MceStat] for the thread. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_inbound_msg (uint8_t soc_die_num, uint8_t *buffer)

  This register will read the inbound message. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_outbound_msg (uint8_t soc_die_num, uint8_t *buffer)

  This register will read the outbound message. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbrmi_threadnumberlow (uint8_t soc_die_num, uint8_t *buffer)

  This register indicates the low part of maximum number of threads. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_threadnumberhi (uint8_t soc_die_num, uint8_t *buffer)

  This register indicates the upper part of maximum number of threads. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_thread_cs (uint8_t soc_die_num, uint8_t *buffer)

  This register is used to read the thread cs. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_ras_status (uint8_t soc_die_num, uint8_t *buffer)

  This register will read the ras status. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t clear_sbrmi_ras_status (uint8_t soc_die_num, uint8_t buffer)

  This API will clear ras status register. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t esmi_get_threads_per_socket (uint8_t soc_die_num, uint32_t *threads_per_socket)

  Get the number of threads per socket.
- oob_status_t clear_sw_async_alert_status (uint8_t soc_die_num)

  Clears the software asynchronous alert status for a specific SoC die.
- oob_status_t clear_MP0_alert_status (uint8_t soc_die_num)

  Clears the MP0 alert status for the specified SoC die.
- oob_status_t set_sw_async_alert_mask (uint8_t soc_die_num, bool alert_mask_bit)

  Set the software asynchronous alert mask for a specific SoC die.
- oob_status_t read_sbrmi_ready_status (uint8_t soc_die_num, bool *buffer)

  Reads the SBRMI ready status for a specified SoC die.
- oob_status_t clear_shut_down_err (uint8_t soc_die_num)

  Clears the shutdown error for the specified SoC die.
- oob_status_t gpio_assertion_on_mailbox (uint8_t soc_die_num, uint8_t dis_alert)

  Controls enable/disable Alert_L assertion for mailbox for a specific SOC die.
- oob_status_t gpio_assertion_for_async_alerts (uint8_t soc_die_num, uint8_t dis_alert)

  Controls enable/disable Alert_L assertion for asynchronous alert signals.

Variables

- const uint8_t thread_en_reg_v10 [MAX_THREAD_REG_V10]

  thread enable register revision 0x10
- const uint8_t thread_en_reg_v20 [MAX_THREAD_REG_V20]

  thread enable register revision 0x20
- const uint16_t thread_en_reg_v21_dense [MAX_THREAD_REG_V21_DENSE]

  thread enable register revision 0x21 for dense plaform
- const uint8_t alert_status [MAX_ALERT_REG]

  alert status register
- const uint16_t alert_status_v21_dense [MAX_ALERT_REG_V21_DENSE]

  alert status register for V21 dense platform
- const uint8_t alert_mask [MAX_ALERT_REG]

  alert mask
- const uint16_t alert_mask_v21_dense [MAX_ALERT_REG_V21_DENSE]

  alert mask

5.14.1 Detailed Description

The SB-RMI registers can be read or written from the SMBus interface using the SMBus defined PEC-optional Read Byte and Write Byte protocols with the SB-RMI register number in the command byte.

5.14.2 Function Documentation

5.14.2.1 read_sbrmi_alert_status()

```
oob_status_t read_sbrmi_alert_status (
            uint8_t soc_die_num,
            uint8_t num_of_alert_status_reg,
            uint8_t ** buffer )
```

This function will read bit vector for all the threads. Value of 1 indicates MCE occured for the thread and is set by hardware. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*num_of_alert_status_reg*	number of alert status registers.
in,out	*buffer*	a pointer to read all "num_of_alert_status_reg" of alert status registers. Buffer length should be equal to "num_of_alert_status_reg" value.

Return values

OOB_SUCCESS	is returned upon successful call.
None-zero	is returned upon failure.

5.14.2.2 read_sbrmi_alert_mask()

```
oob_status_t read_sbrmi_alert_mask (
            uint8_t soc_die_num,
            uint8_t num_of_alert_mask_reg,
            uint8_t ** buffer )
```

This function will read bit vector for all the threads. Value of 1 indicates alert signaling disabled for corresponding SBRMI::AlertStatus[MceStat] for the thread. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*num_of_alert_mask_reg*	number of alert mask registers.
in,out	*buffer*	a pointer to read all "num_of_alert_mask_reg" of alert mask registers. Buffer length should be equal to "num_of_alert_mask_reg" value.

Return values

OOB_SUCCESS	is returned upon successful call.
None-zero	is returned upon failure.

5.14.2.3 clear_sbrmi_ras_status()

```
oob_status_t clear_sbrmi_ras_status (
            uint8_t soc_die_num,
            uint8_t buffer )
```

This API will clear ras status register. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*buffer*	bit mask to clear ras status bits

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.14.2.4 esmi_get_threads_per_socket()

```
oob_status_t esmi_get_threads_per_socket (
            uint8_t soc_die_num,
            uint32_t * threads_per_socket )
```

Get the number of threads per socket.
Get the total number of threads in a socket.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*threads_per_socket*	is returned

Return values

threads_per_socket	is returned upon successful call.

5.14.2.5 clear_sw_async_alert_status()

```
oob_status_t clear_sw_async_alert_status (
            uint8_t soc_die_num )
```

Clears the software asynchronous alert status for a specific SoC die.
This function clears the asynchronous alert status that was set by software for the specified SoC die number.To clear the sw asynchronous alert status we need to write 1 to BIT-3 of status register 0x02.

Parameters

in	*soc_die_num*	The SoC die number for which to clear the alert status.

Returns

oob_status_t Status code indicating success or failure of the operation.

5.14.2.6 clear_MP0_alert_status()

```
oob_status_t clear_MP0_alert_status (
            uint8_t soc_die_num )
```

Clears the MP0 alert status for the specified SoC die.

This function clears the MP0 (Management Processor 0) alert status for the given SoC die number. It is typically used to acknowledge and reset alert conditions reported by the MP0 subsystem.

Parameters

soc_die_num	The index of the SoC die for which the MP0 alert status should be cleared.

Returns

oob_status_t Status code indicating success or failure of the operation.

5.14.2.7 set_sw_async_alert_mask()

```
oob_status_t set_sw_async_alert_mask (
            uint8_t soc_die_num,
            bool alert_mask_bit )
```

Set the software asynchronous alert mask for a specific SoC die.

This function sets the software asynchronous alert mask bit for the specified SoC die. The alert_mask_bit parameter determines whether to set (true) or clear (false) the mask. Set alert_mask_bit to 0 to enable Alert-L signaling and set it 1 to disabled.

Parameters

in	*soc_die_num*	The SoC die number for which to set the alert mask.
in	*alert_mask_bit*	Boolean value to set (true) to disable Alert-L signaling else set to false to enable Alert-L signaling.

Returns

oob_status_t Status code indicating success or failure of the operation.

5.14.2.8 read_sbrmi_ready_status()

```
oob_status_t read_sbrmi_ready_status (
            uint8_t soc_die_num,
            bool * buffer )
```

Reads the SBRMI ready status for a specified SoC die.

This function checks the ready status of the SBRMI (Sideband Remote Management Interface) for the given SoC die number and stores the result in the provided buffer. Supported platforms: Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	The SoC die number to query.
out	*buffer*	Pointer to a boolean variable where the ready status will be stored. True indicates ready, false indicates not ready.

Returns

oob_status_t Status code indicating success or type of failure.

5.14.2.9 clear_shut_down_err()

```
oob_status_t clear_shut_down_err (
            uint8_t soc_die_num )
```

Clears the shutdown error for the specified SoC die.
This function attempts to clear any shutdown error associated with the given SoC die number.Set 1 to BIT-6 of status register 0x4C to clear the shutdown error. Supported platforms: Fam-1Ah_Mod-50h-57h

Parameters

soc_die_num	The number identifying the SoC die to clear the shutdown error for.

Returns

oob_status_t Status of the operation.

5.14.2.10 gpio_assertion_on_mailbox()

```
oob_status_t gpio_assertion_on_mailbox (
            uint8_t soc_die_num,
            uint8_t dis_alert )
```

Controls enable/disable Alert_L assertion for mailbox for a specific SOC die.
This function provides capability to enable/disable Alert_L assertion for mailbox for the specified SOC die number. ALERT_L signaling is enabled(0) when SBRMIx02[SwAlertSts] is set. ALERT_L signaling is disabled(1) when SBRMIx02[SwAlertSts] is set. Supported platforms: Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	The SOC die number on which to assert GPIO.
in	*dis_alert*	Flag to control alert_l signal behavior. • 1: Disable the alert_l signal assertion • 0: Enable the alert_l signal assertion

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.14.2.11 gpio_assertion_for_async_alerts()

```
oob_status_t gpio_assertion_for_async_alerts (
            uint8_t soc_die_num,
            uint8_t dis_alert )
```

Controls enable/disable Alert_L assertion for asynchronous alert signals.
This function provides capability to enable/disable Alert_L assertion for asynchronous alerts on a specified SOC die. ALERT_L signaling is enabled(0) when SBRMIx02[SwAsyncAlertSts] is set. ALERT_L signaling is disabled(1)

when SBRMIx02[SwAsyncAlertSts] is set. Supported platforms: Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	The SOC die number to target for GPIO assertion control. This identifies which die instance should be configured.
in	*dis_alert*	Flag to control the alert_l signal: • 1 to disable the alert_l signal • 0 to enable the alert_l signal

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.14.3 Variable Documentation

5.14.3.1 thread_en_reg_v10

`const uint8_t thread_en_reg_v10 [extern]`
thread enable register revision 0x10
Read one byte from a given SB_RMI register number provided Socket Die index and buffer to get the read data for a particular SB-RMI command register.
Given a Socket Die index `socket_ind` and a pointer to hold the output at uint8_t `buffer`, this function will get the value from a particular command of SB_RMI register.

Parameters

in	*soc_die_num*	Socket uindex.
in,out	*buffer*	a pointer to a uint8_t that indicates value to hold

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

This value specifies the APML specification revision that the product is compliant to. 0x10 = 1.0x Revision.
thread enable register revision 0x10

5.15 SBTSI Register Read Byte Protocol

Functions

- oob_status_t read_sbtsi_cpuinttemp (uint8_t soc_die_num, uint8_t *buffer)

  integer CPU temperature value The CPU temperature is calculated by adding the CPU temperature offset(↩ SBTSI::CpuTempOffInt, SBTSI::CpuTempOffDec) to the processor control temperature (Tctl). SBTSI::CpuTempInt and SBTSI::CpuTempDec combine to return the CPU temperature. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbtsi_status (uint8_t soc_die_num, uint8_t *buffer)

Status register is Read-only, volatile field If SBTSI::AlertConfig[AlertCompEn] == 0 , the temperature alert is latched high until the alert is read. If SBTSI::AlertConfig[AlertCompEn] == 1, the alert is cleared when the temperature does not meet the threshold conditions for temperature and number of samples. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_config (uint8_t soc_die_num, uint8_t *buffer)

  The bits in this register are Read-only and can be written by Writing to the corresponding bits in SBTSI::ConfigWr Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_updaterate (uint8_t soc_die_num, float *buffer)

  This register value specifies the rate at which CPU temperature is compared against the temperature thresholds to determine if an alert event has occurred. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t write_sbtsi_updaterate (uint8_t soc_die_num, float uprate)

  This register value specifies the rate at which CPU temperature is compared against the temperature thresholds to determine if an alert event has occurred. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_hitempint (uint8_t soc_die_num, uint8_t *buffer)

  This value specifies the integer portion of the high temperature threshold. The high temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is greater than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_lotempint (uint8_t soc_die_num, uint8_t *buffer)

  This value specifies the integer portion of the low temperature threshold. The low temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is less than or equal to the threshold Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_configwrite (uint8_t soc_die_num, uint8_t *buffer)

  This register provides write access to SBTSI::Config Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_cputempdecimal (uint8_t soc_die_num, float *buffer)

  The value returns the decimal portion of the CPU temperature Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_cputempoffint (uint8_t soc_die_num, uint8_t *temp_int)

  SBTSI::CpuTempOffInt and SBTSI::CpuTempOffDec combine to specify the CPU temperature offset Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_cputempoffdec (uint8_t soc_die_num, float *temp_dec)

  This value specifies the decimal/fractional portion of the CPU temperature offset added to Tctl to calculate the CPU temperature. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_hitempdecimal (uint8_t soc_die_num, float *temp_dec)

  This value specifies the decimal portion of the high temperature threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_lotempdecimal (uint8_t soc_die_num, float *temp_dec)

  value specifies the decimal portion of the low temperature threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_timeoutconfig (uint8_t soc_die_num, uint8_t *timeout)

  value specifies 0=SMBus defined timeout support disabled. 1=SMBus defined timeout support enabled. SMBus timeout enable. If SB-RMI is in use, SMBus timeouts should be enabled or disabled in a consistent manner on both interfaces. SMBus defined timeouts are not disabled for SB-RMI when this bit is set to 0. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_alertthreshold (uint8_t soc_die_num, uint8_t *samples)

  Specifies the number of consecutive CPU temperature samples for which a temperature alert condition needs to remain valid before the corresponding alert bit is set. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_alertconfig (uint8_t soc_die_num, uint8_t *mode)

Status register is Read-only, volatile field If SBTSI::AlertConfig[AlertCompEn] == 0 , the temperature alert is latched high until the alert is read. If SBTSI::AlertConfig[AlertCompEn] == 1, the alert is cleared when the temperature does not meet the threshold conditions for temperature and number of samples. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_manufid (uint8_t soc_die_num, uint8_t *man_id)

  Returns the AMD manufacture ID Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_revision (uint8_t soc_die_num, uint8_t *rivision)

  Specifies the SBI temperature sensor interface revision Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t sbtsi_get_cputemp (uint8_t soc_die_num, float *cpu_temp)

  CPU temperature value The CPU temperature is calculated by adding SBTSI::CpuTempInt and SBTSI::CpuTempDec combine to return the CPU temperature. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t sbtsi_get_temp_status (uint8_t soc_die_num, uint8_t *loalert, uint8_t *hialert)

  Status register is Read-only, volatile field If SBTSI::AlertConfig[AlertCompEn] == 0 , the temperature alert is latched high until the alert is read. If SBTSI::AlertConfig[AlertCompEn] == 1, the alert is cleared when the temperature does not meet the threshold conditions for temperature and number of samples. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t sbtsi_get_config (uint8_t soc_die_num, uint8_t *al_mask, uint8_t *run_stop, uint8_t *read_ord, uint8_t *ara)

  The bits in this register are Read-only and can be written by Writing to the corresponding bits in SBTSI::ConfigWr Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t sbtsi_set_configwr (uint8_t soc_die_num, uint8_t mode, uint8_t config_mask)

  The bits in this register are defined sbtsi_config_write and can be written by writing to the corresponding bits in SBTSI::ConfigWr Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t sbtsi_get_timeout (uint8_t soc_die_num, uint8_t *timeout_en)

  To verify if timeout support enabled or disabled Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t sbtsi_set_timeout_config (uint8_t soc_die_num, uint8_t mode)

  To enable/disable timeout support Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t sbtsi_set_hitemp_threshold (uint8_t soc_die_num, float hitemp_thr)

  This value set the high temperature threshold. The high temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is greater than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t sbtsi_set_lotemp_threshold (uint8_t soc_die_num, float lotemp_thr)

  This value set the low temperature threshold. The low temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is less than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t sbtsi_get_hitemp_threshold (uint8_t soc_die_num, float *hitemp_thr)

  This value specifies the high temperature threshold. The high temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is greater than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t sbtsi_get_lotemp_threshold (uint8_t soc_die_num, float *lotemp_thr)

  This value specifies the low temperature threshold. The low temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is less than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_cputempoffset (uint8_t soc_die_num, float *temp_offset)

  SBTSI::CpuTempOffInt and SBTSI::CpuTempOffDec combine to specify the CPU temperature offset Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t write_sbtsi_cputempoffset (uint8_t soc_die_num, float temp_offset)

  SBTSI::CpuTempOffInt and SBTSI::CpuTempOffDec combine to set the CPU temperature offset Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t sbtsi_set_alert_threshold (uint8_t soc_die_num, uint8_t samples)

  Specifies the number of consecutive CPU temperature samples for which a temperature alert condition needs to remain valid before the corresponding alert bit is set. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t sbtsi_set_alert_config (uint8_t soc_die_num, uint8_t mode)

  Alert comparator mode enable Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

5.15.1 Detailed Description

Below functions provide interface to read one byte from the SB-TSI register and output is from a given SB_TSI register command.

5.15.2 Function Documentation

5.15.2.1 read_sbtsi_cpuinttemp()

```
oob_status_t read_sbtsi_cpuinttemp (
            uint8_t soc_die_num,
            uint8_t * buffer )
```

integer CPU temperature value The CPU temperature is calculated by adding the CPU temperature offset(SBTSI↩ ::CpuTempOffInt, SBTSI::CpuTempOffDec) to the processor control temperature (Tctl). SBTSI::CpuTempInt and SBTSI::CpuTempDec combine to return the CPU temperature. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
This field returns the integer portion of the CPU temperature

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*buffer*	a pointer to hold the cpu temperature

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.2 read_sbtsi_status()

```
oob_status_t read_sbtsi_status (
            uint8_t soc_die_num,
            uint8_t * buffer )
```

Status register is Read-only, volatile field If SBTSI::AlertConfig[AlertCompEn] == 0 , the temperature alert is latched high until the alert is read. If SBTSI::AlertConfig[AlertCompEn] == 1, the alert is cleared when the temperature does not meet the threshold conditions for temperature and number of samples. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*buffer*	a pointer to hold the cpu temperature

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.3 read_sbtsi_config()

```
oob_status_t read_sbtsi_config (
            uint8_t soc_die_num,
            uint8_t * buffer )
```

The bits in this register are Read-only and can be written by Writing to the corresponding bits in SBTSI↩ ::ConfigWr Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*buffer*	a pointer to hold the cpu temperature

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.4 read_sbtsi_updaterate()

```
oob_status_t read_sbtsi_updaterate (
            uint8_t soc_die_num,
            float * buffer )
```

This register value specifies the rate at which CPU temperature is compared against the temperature thresholds to determine if an alert event has occurred. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*buffer*	a pointer to hold the cpu temperature

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.5 write_sbtsi_updaterate()

```
oob_status_t write_sbtsi_updaterate (
            uint8_t soc_die_num,
            float uprate )
```

This register value specifies the rate at which CPU temperature is compared against the temperature thresholds to determine if an alert event has occurred. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*uprate*	value to write in raw format

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.6 read_sbtsi_hitempint()

```
oob_status_t read_sbtsi_hitempint (
            uint8_t soc_die_num,
            uint8_t * buffer )
```

This value specifies the integer portion of the high temperature threshold. The high temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is greater than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*buffer*	a pointer to hold the integer part of high cpu temp

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.7 read_sbtsi_lotempint()

```
oob_status_t read_sbtsi_lotempint (
            uint8_t soc_die_num,
            uint8_t * buffer )
```

This value specifies the integer portion of the low temperature threshold. The low temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is less than or equal to the threshold Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*buffer*	a pointer to hold the integer part of low cpu temp

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.8 read_sbtsi_configwrite()

```
oob_status_t read_sbtsi_configwrite (
            uint8_t soc_die_num,
            uint8_t * buffer )
```

This register provides write access to SBTSI::Config Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*buffer*	a pointer to hold the configuraion

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.9 read_sbtsi_cputempdecimal()

```
oob_status_t read_sbtsi_cputempdecimal (
            uint8_t soc_die_num,
            float * buffer )
```

The value returns the decimal portion of the CPU temperature Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*buffer*	a pointer to hold the cpu temperature decimal

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.10 read_sbtsi_cputempoffint()

```
oob_status_t read_sbtsi_cputempoffint (
            uint8_t soc_die_num,
            uint8_t * temp_int )
```

SBTSI::CpuTempOffInt and SBTSI::CpuTempOffDec combine to specify the CPU temperature offset Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*temp_int*	a pointer to hold the cpu offset interger

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.11 read_sbtsi_cputempoffdec()

```
oob_status_t read_sbtsi_cputempoffdec (
            uint8_t soc_die_num,
            float * temp_dec )
```

This value specifies the decimal/fractional portion of the CPU temperature offset added to Tctl to calculate the CPU temperature. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*temp_dec*	a pointer to hold the cpu offset decimal

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.12 read_sbtsi_hitempdecimal()

```
oob_status_t read_sbtsi_hitempdecimal (
            uint8_t soc_die_num,
            float * temp_dec )
```

This value specifies the decimal portion of the high temperature threshold. Supported platforms↩ : Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*temp_dec*	a pointer to hold the decimal part of cpu high temp

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.13 read_sbtsi_lotempdecimal()

```
oob_status_t read_sbtsi_lotempdecimal (
            uint8_t soc_die_num,
            float * temp_dec )
```

value specifies the decimal portion of the low temperature threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*temp_dec*	a pointer to hold the decimal part of cpu low temperature

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.14 read_sbtsi_timeoutconfig()

```
oob_status_t read_sbtsi_timeoutconfig (
            uint8_t soc_die_num,
            uint8_t * timeout )
```

value specifies 0=SMBus defined timeout support disabled. 1=SMBus defined timeout support enabled. SMBus timeout enable. If SB-RMI is in use, SMBus timeouts should be enabled or disabled in a consistent manner on both interfaces. SMBus defined timeouts are not disabled for SB-RMI when this bit is set to 0. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*timeout*	a pointer to hold the cpu timeout configuration

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.15 read_sbtsi_alertthreshold()

```
oob_status_t read_sbtsi_alertthreshold (
            uint8_t soc_die_num,
            uint8_t * samples )
```

Specifies the number of consecutive CPU temperature samples for which a temperature alert condition needs to remain valid before the corresponding alert bit is set. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*samples*	a pointer to hold the cpu temperature alert threshold

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.16 read_sbtsi_alertconfig()

```
oob_status_t read_sbtsi_alertconfig (
            uint8_t soc_die_num,
            uint8_t * mode )
```

Status register is Read-only, volatile field If SBTSI::AlertConfig[AlertCompEn] == 0 , the temperature alert is latched high until the alert is read. If SBTSI::AlertConfig[AlertCompEn] == 1, the alert is cleared when the temperature does not meet the threshold conditions for temperature and number of samples. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*mode*	a pointer to hold the cpu temperature alert configuration

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.17 read_sbtsi_manufid()

```
oob_status_t read_sbtsi_manufid (
            uint8_t soc_die_num,
            uint8_t * man_id )
```

Returns the AMD manufacture ID Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*man_id*	a pointer to hold the manufacture id

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.18 read_sbtsi_revision()

```
oob_status_t read_sbtsi_revision (
            uint8_t soc_die_num,
            uint8_t * rivision )
```

Specifies the SBI temperature sensor interface revision Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*rivision*	a pointer to hold the cpu temperature revision

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.19 sbtsi_get_cputemp()

```
oob_status_t sbtsi_get_cputemp (
            uint8_t soc_die_num,
            float * cpu_temp )
```

CPU temperature value The CPU temperature is calculated by adding SBTSI::CpuTempInt and SBTSI↩ ::CpuTempDec combine to return the CPU temperature. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*cpu_temp*	a pointer to get temperature of the CPU

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.20 sbtsi_get_temp_status()

```
oob_status_t sbtsi_get_temp_status (
            uint8_t soc_die_num,
            uint8_t * loalert,
            uint8_t * hialert )
```

Status register is Read-only, volatile field If SBTSI::AlertConfig[AlertCompEn] == 0 , the temperature alert is latched high until the alert is read. If SBTSI::AlertConfig[AlertCompEn] == 1, the alert is cleared when the temperature does not meet the threshold conditions for temperature and number of samples. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*loalert*	1=> CPU temp is less than or equal to low temperature threshold for consecutive samples
in,out	*hialert*	1=> CPU temp is greater than or equal to high temperature threshold for consecutive samples

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.21 sbtsi_get_config()

```
oob_status_t sbtsi_get_config (
            uint8_t soc_die_num,
            uint8_t * al_mask,
            uint8_t * run_stop,
            uint8_t * read_ord,
            uint8_t * ara )
```

The bits in this register are Read-only and can be written by Writing to the corresponding bits in SBTSI↩ ::ConfigWr Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*al_mask*	0=> ALERT_L pin enabled. 1=> ALERT_L pin disabled and does not assert.
in,out	*run_stop*	0=> Updates to CpuTempInt and CpuTempDec and alert comparisons are enabled. 1=> Updates are disabled and alert comparisons are disabled.
in,out	*read_ord*	0=> Reading CpuTempInt causes the satate of CpuTempDec to be latched. 1=> Reading CpuTempInt causes the satate of CpuTempDec to be latched.
in,out	*ara*	1=> ARA response disabled.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.22 sbtsi_set_configwr()

```
oob_status_t sbtsi_set_configwr (
            uint8_t soc_die_num,
            uint8_t mode,
            uint8_t config_mask )
```

The bits in this register are defined sbtsi_config_write and can be written by writing to the corresponding bits in SBTSI::ConfigWr Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
NOTE: Currently testing is not done for this API.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*mode*	value to update 0 or 1
in	*config_mask*	which bit need to update

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.23 sbtsi_get_timeout()

```
oob_status_t sbtsi_get_timeout (
            uint8_t soc_die_num,
            uint8_t * timeout_en )
```

To verify if timeout support enabled or disabled Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*timeout_en*	0=>SMBus defined timeout support disabled.

1=SMBus defined timeout support enabled. SMBus timeout enable. If SB-RMI is in use, SMBus timeouts should be enabled or disabled in a consistent manner on both interfaces. SMBus defined timeouts are not disabled for SB-RMI when this bit is set to 0.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.24 sbtsi_set_timeout_config()

```
oob_status_t sbtsi_set_timeout_config (
            uint8_t soc_die_num,
            uint8_t mode )
```

To enable/disable timeout support Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*mode*	0=>SMBus defined timeout support disabled.

1=>SMBus defined timeout support enabled. SMBus timeout enable. If SB-RMI is in use, SMBus timeouts should be enabled or disabled in a consistent manner on both interfaces. SMBus defined timeouts are not disabled for SB-RMI when this bit is set to 0.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.25 sbtsi_set_hitemp_threshold()

```
oob_status_t sbtsi_set_hitemp_threshold (
            uint8_t soc_die_num,
            float hitemp_thr )
```

This value set the high temperature threshold. The high temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is greater than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*hitemp_thr*	Specifies the high temperature threshold

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.26 sbtsi_set_lotemp_threshold()

```
oob_status_t sbtsi_set_lotemp_threshold (
            uint8_t soc_die_num,
            float lotemp_thr )
```

This value set the low temperature threshold. The low temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is less than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*lotemp_thr*	Specifies the low temperature threshold

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.27 sbtsi_get_hitemp_threshold()

```
oob_status_t sbtsi_get_hitemp_threshold (
            uint8_t soc_die_num,
            float * hitemp_thr )
```

This value specifies the high temperature threshold. The high temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is greater than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*hitemp_thr*	Specifies the high temperature threshold

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.28 sbtsi_get_lotemp_threshold()

```
oob_status_t sbtsi_get_lotemp_threshold (
            uint8_t soc_die_num,
            float * lotemp_thr )
```

This value specifies the low temperature threshold. The low temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is less than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*lotemp_thr*	Get the low temperature threshold

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.29 read_sbtsi_cputempoffset()

```
oob_status_t read_sbtsi_cputempoffset (
            uint8_t soc_die_num,
            float * temp_offset )
```

SBTSI::CpuTempOffInt and SBTSI::CpuTempOffDec combine to specify the CPU temperature offset Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*temp_offset*	to get the offset value for temperature

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.30 write_sbtsi_cputempoffset()

```
oob_status_t write_sbtsi_cputempoffset (
            uint8_t soc_die_num,
            float temp_offset )
```

SBTSI::CpuTempOffInt and SBTSI::CpuTempOffDec combine to set the CPU temperature offset Supported

platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*temp_offset*	to set the offset value for temperature

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.31 sbtsi_set_alert_threshold()

```
oob_status_t sbtsi_set_alert_threshold (
            uint8_t soc_die_num,
            uint8_t samples )
```

Specifies the number of consecutive CPU temperature samples for which a temperature alert condition needs to remain valid before the corresponding alert bit is set. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*samples*	Number of samples 0h: 1 sample 6h-1h: (value + 1) sample 7h: 8 sample

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

5.15.2.32 sbtsi_set_alert_config()

```
oob_status_t sbtsi_set_alert_config (
            uint8_t soc_die_num,
            uint8_t mode )
```

Alert comparator mode enable Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*mode*	0=> SBTSI::Status[TempHighAlert] & SBTSI::Status[TempLowAlert] are read-clear. 1=> SBTSI::Status[TempHighAlert] & SBTSI::Status[TempLowAlert] are read-only. ARA response disabled.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

Chapter 6

Data Structure Documentation

6.1 apml_encodings Struct Reference

APML link ID encodings for legacy platforms and MI300A. Structure contains the following members. Link ID encoding value and its name. MI300A APML Link ID Encoding: 00000011b: P2 00000100b: P3 00001000b: G0 00001001b: G1 00001010b: G2 00001011b: G3 00001100b: G4 00001101b: G5 00001110b: G6 00001111b: G7 For other platforms the APML Link ID Encoding: 00000001b: P0 00000010b: P1 00000100b: P2 00001000b: P3 00010000b: G0 00100000b: G1 01000000b: G2 10000000b: G3.

`#include <apml_common.h>`

Data Fields

- uint16_t val

  Link ID encoding value.
- char name [3]

  Link ID encoding name.

6.1.1 Detailed Description

APML link ID encodings for legacy platforms and MI300A. Structure contains the following members. Link ID encoding value and its name. MI300A APML Link ID Encoding: 00000011b: P2 00000100b: P3 00001000b: G0 00001001b: G1 00001010b: G2 00001011b: G3 00001100b: G4 00001101b: G5 00001110b: G6 00001111b: G7 For other platforms the APML Link ID Encoding: 00000001b: P0 00000010b: P1 00000100b: P2 00001000b: P3 00010000b: G0 00100000b: G1 01000000b: G2 10000000b: G3.

The documentation for this struct was generated from the following file:

- apml_common.h

6.2 bmc_ras_runtime_din Struct Reference

Structure representing BMC RAS runtime error information.

`#include <esmi_mailbox.h>`

Data Fields

- uint8_t err_category

  0 for MCA, 1 for DRAM ECC, 2 PCIE
- uint32_t number_of_instance

  Number of mca/dram/pcie runtime errors.
- uint32_t number_of_offsets

6.2.1 Detailed Description

Structure representing BMC RAS runtime error information.
This structure holds information about runtime errors detected by the BMC, categorized by error type (MCA, DRAM ECC, or PCIe). It provides the number of error instances and the number of offsets, which can be calculated by querying with the appropriate message ID and category.

6.2.2 Field Documentation

6.2.2.1 err_category

`bmc_ras_runtime_din::err_category`
0 for MCA, 1 for DRAM ECC, 2 PCIE
Error category:

- 0: MCA
- 1: DRAM ECC
- 2: PCIe

6.2.2.2 number_of_instance

`bmc_ras_runtime_din::number_of_instance`
Number of mca/dram/pcie runtime errors.
Number of MCA/DRAM/PCIe runtime error instances.

6.2.2.3 number_of_offsets

`bmc_ras_runtime_din::number_of_offsets`
Number of offsets which can be calculcated by calling 6A message ID with category as MCA/dram/pcie and mode as 0 and it returns number of instances and number of bytes per instance. Number of offsets will be number of bytes per instance / 4.
Number of offsets, calculated by calling message ID 0x6A with the appropriate category and mode. The number of offsets is determined by dividing the number of bytes per instance by 4.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.3 core_floor_limit_din Struct Reference

Structure for APML floor core limit input.
`#include <esmi_mailbox.h>`

Data Fields

- uint16_t core_id

  Core ID for which the floor limit is set.
- uint16_t floor_limit

  Floor limit value for the specified core.

6.3.1 Detailed Description

Structure for APML floor core limit input.
This structure is used to specify the core ID and the desired floor limit for APML mailbox operations related to floor core limit configuration.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.4 dimm_pow_din Struct Reference

Input for DIMM power, DIMM address and power reporting flag. pow_reporting_flag: Valid values are 0, 1, 2.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t dimm_addr

  Dimm address.
- uint8_t pow_reporting_flag: 2

  Power reporting flag.

6.4.1 Detailed Description

Input for DIMM power, DIMM address and power reporting flag. pow_reporting_flag: Valid values are 0, 1, 2.

- 0: Power consumption of the given DIMM address.
- 1: DIMM consumping highest power and corresponding address
- 2: Total power consumption of all populated and enabled DIMM's dimm_addr: Valid only if pow_reporting_flag is 0, else reserved.

The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.5 dimm_pow_dout Struct Reference

DIMM power(mW), update rate(ms) and DIMM address.
`#include <esmi_mailbox.h>`

Data Fields

- uint32_t power

  DiMM power consumption.
- uint16_t update_rate: 9

  update rate in ms
- uint8_t dimm_addr

  DIMM address.

6.5.1 Detailed Description

DIMM power(mW), update rate(ms) and DIMM address.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.6 dimm_power Struct Reference

DIMM power(mW), update rate(ms) and dimm address.
`#include <esmi_mailbox.h>`

Data Fields

- uint16_t power: 15

  Dimm power consumption.
- uint16_t update_rate: 9

  update rate in ms
- uint8_t dimm_addr

  Dimm address.

6.6.1 Detailed Description

DIMM power(mW), update rate(ms) and dimm address.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.7 dimm_sb_reg_d_in Struct Reference

dimm_sb_reg_d_in: structure contains following members to be passed as input, to get 4 byte DIMM sb register data
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t dimm_addr

  DIMM address.
- uint8_t lid: 4

  local identifier of device
- uint16_t reg_offset: 11

  Register offset in given register space.
- uint8_t reg_space: 1

  Register space, Volatile:0, NVM:1.
- uint8_t rsvd

  reserved

6.7.1 Detailed Description

dimm_sb_reg_d_in: structure contains following members to be passed as input, to get 4 byte DIMM sb register data
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.8 dimm_sb_reg_write Struct Reference

Dimm side band register write: strucutre contains dimm address, LID of device, Register offset, Register space, and data to write.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t dimm_addr
  DIMM address.
- uint8_t lid: 4
  local identifier of device
- uint16_t reg_offset: 11
  Register offset in given register space.
- uint8_t reg_space: 1
  Register space, Volatile:0, NVM:1.
- uint8_t w_data
  Write data.

6.8.1 Detailed Description

Dimm side band register write: strucutre contains dimm address, LID of device, Register offset, Register space, and data to write.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.9 dimm_spd_d_in Struct Reference

dimm_spd_d_in: strucutre contains following memebers to be passed as input, to get 4 byte DIMM spd register data
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t dimm_addr
  DIMM address.
- uint8_t lid: 4
  local identifier of device
- uint16_t reg_offset: 11
  Register offset in given register space.
- uint8_t reg_space: 1
  Register space, Volatile:0, NVM:1.
- uint8_t rsvd
  reserved

6.9.1 Detailed Description

dimm_spd_d_in: strucutre contains following memebers to be passed as input, to get 4 byte DIMM spd register data
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.10 dimm_spd_write Struct Reference

SPD side band write: strucutre contains dimm address, LID of device, Register offset, Register space, and data to write.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t dimm_addr

  DIMM address.
- uint8_t lid: 4

  local identifier of device
- uint16_t reg_offset: 11

  Register offset in given register space.
- uint8_t reg_space: 1

  Register space, Volatile:0, NVM:1.
- uint8_t w_data

  Write data.

6.10.1 Detailed Description

SPD side band write: strucutre contains dimm address, LID of device, Register offset, Register space, and data to write.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.11 dimm_thermal Struct Reference

DIMM thermal sensor (degree C), update rate and dimm address.
`#include <esmi_mailbox.h>`

Data Fields

- uint16_t sensor: 11

  Dimm thermal sensor.
- uint16_t update_rate: 9

  update rate in ms
- uint8_t dimm_addr

  Dimm address.

6.11.1 Detailed Description

DIMM thermal sensor (degree C), update rate and dimm address.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.12 dimm_thermal_din Struct Reference

DIMM address and thermal sensor reporting flag. thermal_flag: Valid values are 0 and 1.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t thermal_flag: 1

  Thermal sensor reporting flag.
- uint8_t dimm_addr

  DIMM address.

6.12.1 Detailed Description

DIMM address and thermal sensor reporting flag. thermal_flag: Valid values are 0 and 1.

- 0: Temperature for the provided DIMM
- 1: Temperature of hottest DIMM dimm_addr: Valid only if the thermal_flag is 0, else reserved.

The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.13 dpm_level Struct Reference

Max and min LCK DPM level on a given NBIO ID. Valid Max and min DPM level values are 0 - 1.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t max_dpm_level

  Max LCLK DPM level [0 - 1].
- uint8_t min_dpm_level

  Min LCLK DPM level [0 - 1].

6.13.1 Detailed Description

Max and min LCK DPM level on a given NBIO ID. Valid Max and min DPM level values are 0 - 1.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.14 dram_thr_with_status Struct Reference

Structure for dram throttle percentage with throttle status.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t thr_pct

  throttle percentage
- uint8_t tsod_th_thr_stat: 1

  TSOD thermal throttle status.
- uint8_t tsod_en_stat: 1

  TSOD enable/disable status.
- uint8_t odts_th_thr_stat: 1

  ODTS thermal throttle status.
- uint8_t odts_en_stat: 1

  ODTS enable/disable status.

6.14.1 Detailed Description

Structure for dram throttle percentage with throttle status.
This structure is used to specify the following:

- DRAM Throttle percentage
- TSOD thermal throttle status (1 means active and 0 means inactive)

- TSOD enable/disable status (1 means enable and 0 means disable)
- ODTS thermal throttle status (1 means active and 0 means inactive)
- ODTS enable/disable status (1 means enable and 0 means disable)

The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.15 freq_limits Struct Reference

struct containing max frequency and min frequencey limit
`#include <rmi_mailbox_mi300.h>`

Data Fields

- uint16_t max

  Max clock frequency.
- uint16_t min

  Min clock frequency.

6.15.1 Detailed Description

struct containing max frequency and min frequencey limit
The documentation for this struct was generated from the following file:

- rmi_mailbox_mi300.h

6.16 get_hsmp_cmds_din Struct Reference

Data in for get enabled HSMP commands. structure contains: rmask (read mask indicator : 1 = for read commands, 0 = for write commands), offset for the 24-bitmask segment.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t rmask: 1

  1 = for read commands, 0 = for write commands.
- uint8_t offset: 4

  Offset id for the 24-bit bitmask segment, starting from 0, 1, 2, 3, etc.

6.16.1 Detailed Description

Data in for get enabled HSMP commands. structure contains: rmask (read mask indicator : 1 = for read commands, 0 = for write commands), offset for the 24-bitmask segment.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.17 get_hsmp_cmds_dout Struct Reference

Data out for get enabled HSMP commands. structure contains: offset for the 24-bitmask segment, bitmask (24-bit bitmask in hexa decimal)
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t offset: 4

  Offset id for the 24-bit bitmask segment, starting from 0, 1, 2, 3, etc.
- uint32_t bitmask: 24

  24-bit bit mask segment in hex

6.17.1 Detailed Description

Data out for get enabled HSMP commands. structure contains: offset for the 24-bitmask segment, bitmask (24-bit bitmask in hexa decimal)
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.18 hbm_device_info Struct Reference

struct containing device vendor, part number and total memory size in GBs.
`#include <rmi_mailbox_mi300.h>`

Data Fields

- uint8_t dev_vendor

  device vendor
- uint8_t part_num

  part number
- uint16_t total_mem

  total memory size(GBs)

6.18.1 Detailed Description

struct containing device vendor, part number and total memory size in GBs.
The documentation for this struct was generated from the following file:

- rmi_mailbox_mi300.h

6.19 host_status Struct Reference

struct containing power management controlling status, driving running status.
`#include <rmi_mailbox_mi300.h>`

Data Fields

- uint8_t controller_status: 1

  power mangagement controller status
- uint8_t driver_status: 1

  driver running status

6.19.1 Detailed Description

struct containing power management controlling status, driving running status.
The documentation for this struct was generated from the following file:

- rmi_mailbox_mi300.h

6.20 hottest_dimm_temp_refresh_rate Struct Reference

Hottest DIMM address, temperature range and refresh rate.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t dimm_addr

  DIMM address.
- uint8_t range: 3

  Temperature range (MR4[2:0]) (3-bit field)
- uint8_t ref_rate: 1

  Refresh rate (MR4[3]) (1-bit field, e.g., 1X/2X)

6.20.1 Detailed Description

Hottest DIMM address, temperature range and refresh rate.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.21 lclk_dpm_level_range Struct Reference

Max and Min Link frequency clock (LCLK) DPM level on a socket. 8 bit NBIO ID, dpm_level struct containing 8 bit max DPM level, 8 bit min DPM level.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t nbio_id

  NBIOD id (8 bit data [0 - 3])
- struct dpm_level dpm

  struct with max dpm, min dpm levels

6.21.1 Detailed Description

Max and Min Link frequency clock (LCLK) DPM level on a socket. 8 bit NBIO ID, dpm_level struct containing 8 bit max DPM level, 8 bit min DPM level.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.22 link_id_bw_type Struct Reference

APML LINK ID and Bandwidth type Information.It contains APML LINK ID Encoding. SP7 platforms valid Link ID encodings are 1(P0), 2(P1), 4(P2), 8(P3), 16(G0), 32(G1), 64(G2), 128(G3), 256(P4) and 512(P5) and rest of the Non-MI300 Platforms Valid Link ID encodings are 1(P0), 2(P1), 4(P2), 8(P3), 16(G0), 32(G1), 64(G2), 128(G3). Valid APML MI300 APML LINK ID Encoding. Valid Link ID encodings are 3(P2), 4(P3), 8(G0), 9(G1), 10(G2), 11(G3), 12(G4), 13(G5), 14(G6), 15(G7). IO Bandwidth types 1(Aggregate_BW), 2 (Read BW), 4 (Write BW).
`#include <esmi_mailbox.h>`

Data Fields

- apml_io_bw_encoding bw_type

  Bandwidth Type Information [1, 2, 4].
- uint16_t link_id

6.22.1 Detailed Description

APML LINK ID and Bandwidth type Information.It contains APML LINK ID Encoding. SP7 platforms valid Link ID encodings are 1(P0), 2(P1), 4(P2), 8(P3), 16(G0), 32(G1), 64(G2), 128(G3), 256(P4) and 512(P5) and rest of the Non-MI300 Platforms Valid Link ID encodings are 1(P0), 2(P1), 4(P2), 8(P3), 16(G0), 32(G1), 64(G2), 128(G3). Valid APML MI300 APML LINK ID Encoding. Valid Link ID encodings are 3(P2), 4(P3), 8(G0), 9(G1), 10(G2), 11(G3), 12(G4), 13(G5), 14(G6), 15(G7). IO Bandwidth types 1(Aggregate_BW), 2 (Read BW), 4 (Write BW).

6.22.2 Field Documentation

6.22.2.1 link_id

`uint16_t link_id_bw_type::link_id`

Link ID [1,2,4,8,16,32,64,128] for SP7 platform Link ID [1,2,4,8,16,32,64,128] for rest of the Non-MI300 platforms Link ID [3,4,8,9,10,11,12,13,14,15] for MI300 platforms

The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.23 max_ddr_bw Struct Reference

Structure for Max DDR bandwidth and utilization. It contains max bandwidth(12 bit data) in GBps, current utilization bandwidth(12 bit data) in GBps, current utilized bandwidth(8 bit data) in percentage.

`#include <esmi_mailbox.h>`

Data Fields

- uint16_t max_bw: 12

  Max Bandwidth (12 bit data)
- uint16_t utilized_bw: 12

  Utilized Bandwidth (12 bit data)
- uint8_t utilized_pct

  Utliized Bandwidth percentage.

6.23.1 Detailed Description

Structure for Max DDR bandwidth and utilization. It contains max bandwidth(12 bit data) in GBps, current utilization bandwidth(12 bit data) in GBps, current utilized bandwidth(8 bit data) in percentage.

The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.24 max_mem_bw Struct Reference

Structure for Max DDR/HBM bandwidth and utilization. It contains max bandwidth(16 bit data) in GBps, current utilization bandwidth(Read+Write)(16 bit data) in GBps.

`#include <rmi_mailbox_mi300.h>`

Data Fields

- uint16_t max_bw

  Max Bandwidth (16 bit data)
- uint16_t utilized_bw

  Utilized Bandwidth (16 bit data)

6.24.1 Detailed Description

Structure for Max DDR/HBM bandwidth and utilization. It contains max bandwidth(16 bit data) in GBps, current utilization bandwidth(Read+Write)(16 bit data) in GBps.
The documentation for this struct was generated from the following file:

- rmi_mailbox_mi300.h

6.25 mca_bank Struct Reference

MCA bank information.It contains 16 bit Index for MCA Bank and 16 bit offset.
`#include <esmi_mailbox.h>`

Data Fields

- uint16_t offset

  Offset with in MCA Bank.
- uint16_t index

  Index of MCA Bank.

6.25.1 Detailed Description

MCA bank information.It contains 16 bit Index for MCA Bank and 16 bit offset.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.26 mclk_fclk_pstates Struct Reference

struct containing memory clock and fabric clock pstate mappings.
`#include <rmi_mailbox_mi300.h>`

Data Fields

- uint16_t mem_clk

  memory clock frequency in MHz
- uint16_t f_clk

  fabric clock frequnecy in MHz

6.26.1 Detailed Description

struct containing memory clock and fabric clock pstate mappings.
The documentation for this struct was generated from the following file:

- rmi_mailbox_mi300.h

6.27 nbio_err_log Struct Reference

NBIO quadrant(8 bit data) and NBIO register offset(24 bit) data.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t quadrant

  < NBIO quadrant data
- uint32_t offset: 24

  < NBIO register offset (24 bit data)

6.27.1 Detailed Description

NBIO quadrant(8 bit data) and NBIO register offset(24 bit) data.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.28 oob_config_d_in Struct Reference

Configure oob state infrastructure in SoC. structure consists of mca_oob_misc0_ec_enable, dram_cecc_oob_ec↩ _mode, dram_cecc_leak_rate, pcie_err_reporting_en and mca_err_reporting_en.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t mca_oob_misc0_ec_enable: 1

  Enable: 0 = disable, 1 = enable.
- uint8_t dram_cecc_oob_ec_mode: 2
- uint8_t dram_cecc_leak_rate: 5

  Valid values are 00 - 1Fh.
- uint8_t pcie_err_reporting_en: 1

  0 disable and 1 for enable
- uint8_t core_mca_err_reporting_en: 1

  0 disable and 1 for enable

6.28.1 Detailed Description

Configure oob state infrastructure in SoC. structure consists of mca_oob_misc0_ec_enable, dram_cecc_oob_ec↩ _mode, dram_cecc_leak_rate, pcie_err_reporting_en and mca_err_reporting_en.

6.28.2 Field Documentation

6.28.2.1 mca_oob_misc0_ec_enable

`uint8_t oob_config_d_in::mca_oob_misc0_ec_enable`
Enable: 0 = disable, 1 = enable.
MCA OOB MISC0 Error Counter

6.28.2.2 dram_cecc_oob_ec_mode

`uint8_t oob_config_d_in::dram_cecc_oob_ec_mode`
DRAM CECC OOB error counter mode 00 = disable, 01 enable in noleak mode, 10 = enable in leak mode and 11 is reserved

6.28.2.3 dram_cecc_leak_rate

`uint8_t oob_config_d_in::dram_cecc_leak_rate`
Valid values are 00 - 1Fh.
DRAM CECC OOB leak rate.

6.28.2.4 pcie_err_reporting_en

`uint8_t oob_config_d_in::pcie_err_reporting_en`
0 disable and 1 for enable
PCIe OOB error reporting enable

6.28.2.5 core_mca_err_reporting_en

`uint8_t oob_config_d_in::core_mca_err_reporting_en`
0 disable and 1 for enable
MCA OOB error reporting enable
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.29 oob_err_inj_types Struct Reference

Error injection types containing generic error types, vendor specfic error types, error size and vendor flag.
`#include <esmi_mailbox.h>`

Data Fields

- uint32_t err_types_generic

  Generic error types.
- uint32_t err_types_vendor [3]
- uint16_t err_size

  includes generic and vendor specfic error bytes,
- uint16_t vendor_flag

  [15:0]: vendor error enabled/disabled

6.29.1 Detailed Description

Error injection types containing generic error types, vendor specfic error types, error size and vendor flag.

6.29.2 Field Documentation

6.29.2.1 err_types_vendor

`uint32_t oob_err_inj_types::err_types_vendor[3]`
vendor specific error types Index 0 : Lower 32 bit error types Index 1 : Next 32 bit error types Index 2 : Upper 32 bit error types

6.29.2.2 err_size

`uint16_t oob_err_inj_types::err_size`
includes generic and vendor specfic error bytes,
[15:0] : should return error size which
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.30 pci_address Struct Reference

PCI address information .PCI address includes 4 bit segment, 12 bit aligned offset, 8 bit bus, 5 bit device info and 3 bit function.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t func: 3

  function (3 bit data)
- uint8_t device: 5

  device info (5 bit data)
- uint8_t bus

  bus (8 bit data)
- uint16_t offset: 12

  offset address (12 bit data)
- uint8_t segment: 4

  segment (4 bit data)

6.30.1 Detailed Description

PCI address information .PCI address includes 4 bit segment, 12 bit aligned offset, 8 bit bus, 5 bit device info and 3 bit function.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.31 pcie_link_training Struct Reference

PCIe Halt Link Training information. It includes 10 bit link ID, 4 bit Pcie Port number range from 0 - 7.
`#include <esmi_mailbox.h>`

Data Fields

- uint16_t link_id: 10

  Link ID.
- uint8_t pcie_port: 4

  PCIe port.

6.31.1 Detailed Description

PCIe Halt Link Training information. It includes 10 bit link ID, 4 bit Pcie Port number range from 0 - 7.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.32 pow_eff_mode Struct Reference

Structure for Power Efficiency Mode selection.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t mode: 3

  Power efficiency mode selection (3 bits)
- uint8_t utilization_point

  Utilization point for balanced core modes.
- uint32_t ppt_limit: 21

  PPT limit for balanced core modes (21 bits)

6.32.1 Detailed Description

Structure for Power Efficiency Mode selection.
This structure is used to specify the power efficiency mode, utilization point, and PPT (Package Power Tracking) limit for balanced core modes.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.33 processor_info Struct Reference

Read Proccessor Info.
`#include <esmi_cpuid_msr.h>`

Data Fields

- uint32_t family
  Processor Family in hexa.
- uint32_t model
  Processor Model in hexa.
- uint32_t step_id
  Stepping Identifier in hexa.

6.33.1 Detailed Description

Read Proccessor Info.
The documentation for this struct was generated from the following file:

- esmi_cpuid_msr.h

6.34 pstate_freq Struct Reference

DF P-state frequency.It includes mem clock(16 bit data) frequency (DRAM memory clock), data fabric clock (12 bit data), UMC clock divider (UMC) (1 bit data).
`#include <esmi_mailbox.h>`

Data Fields

- uint16_t mem_clk
  DRAM Memory clock Frequency (MHz)(12 bit)
- uint16_t fclk: 12
  Data fabric clock (MHz)(12 bit data)
- uint8_t uclk: 1
  UMC clock divider (1 bit data)

6.34.1 Detailed Description

DF P-state frequency.It includes mem clock(16 bit data) frequency (DRAM memory clock), data fabric clock (12 bit data), UMC clock divider (UMC) (1 bit data).
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.35 ras_df_err_chk Struct Reference

RAS df err validity check output status. Structure contains the following members. df_block_instances number of block instance with error log to report (0 - 256) err_log_len length of error log in bytes per instance (0 - 256).
`#include <esmi_mailbox.h>`

Data Fields

- uint32_t add_err_data

  field is updated when additional error data is available
- uint16_t df_block_instances: 9

  Number of DF block instances.
- uint16_t err_log_len: 9

  len of er log in bytes per inst.

6.35.1 Detailed Description

RAS df err validity check output status. Structure contains the following members. df_block_instances number of block instance with error log to report (0 - 256) err_log_len length of error log in bytes per instance (0 - 256).
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.36 ras_df_err_dump Union Reference

RAS df error dump input. Union contains the following members. input[0] 4 byte alligned offset in error log (0 - 255) input[1] DF block ID (0 - 255) input[2] Zero based index of DF block instance (0 - 255) input[3] Reserved data_in 32-bit data input.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t input [4]

  [2] block ID inst, [3] RESERVED
- uint32_t data_in

  32 bit data in for the DF err dump

6.36.1 Detailed Description

RAS df error dump input. Union contains the following members. input[0] 4 byte alligned offset in error log (0 - 255) input[1] DF block ID (0 - 255) input[2] Zero based index of DF block instance (0 - 255) input[3] Reserved data_in 32-bit data input.

6.36.2 Field Documentation

6.36.2.1 input

`uint8_t ras_df_err_dump::input[4]`
[2] block ID inst, [3] RESERVED
[0] offset, [1] DF block ID
The documentation for this union was generated from the following file:

- esmi_mailbox.h

6.37 ras_override_delay Struct Reference

BMC RAS override delay reset CPU on sync flood. The structure contains delay value override in mins [5 -120 mins], disable delay counter and stop delay counter. If disable delay counter is set ResetCpuOnSyncFlood response will NOT be delayed in the next SyncFlood regardless of the value specified in delay_val_override. If StopDelayCounter is set it stops the active delay countdown which extends the DelayResetCpuOnSyncFlood indefinitely and system will not reset.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t delay_val_override

  Delay value override [5 -120 mins].
- uint8_t disable_delay_counter: 1

  Disable delay counter.
- uint8_t stop_delay_counter: 1

  stop delay counter

6.37.1 Detailed Description

BMC RAS override delay reset CPU on sync flood. The structure contains delay value override in mins [5 -120 mins], disable delay counter and stop delay counter. If disable delay counter is set ResetCpuOnSyncFlood response will NOT be delayed in the next SyncFlood regardless of the value specified in delay_val_override. If StopDelayCounter is set it stops the active delay countdown which extends the DelayResetCpuOnSyncFlood indefinitely and system will not reset.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.38 ras_rt_err_req_type Struct Reference

Get the Error type and request type. Supported Runtime error type[1:0]:
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t err_type: 2

  Error type, [00, 01, 10].
- uint8_t req_type: 1

  Request type, [0, 1].

6.38.1 Detailed Description

Get the Error type and request type. Supported Runtime error type[1:0]:

- 00 = MCA
- 01 = DRAM CECC
- 10 = PCIe Supported Request type[31]:
- 0: polling mode
- 1: interrupt mode

The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.39 ras_rt_valid_err_inst Struct Reference

Number of valid error instance per category. It consists of number of bytes per each category and number of instances of each category.
`#include <esmi_mailbox.h>`

Data Fields

- uint16_t number_bytes

  Number of bytes of given category.
- uint16_t number_of_inst

  Number of instances of given catg.

6.39.1 Detailed Description

Number of valid error instance per category. It consists of number of bytes per each category and number of instances of each category.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.40 ras_runtime_response Struct Reference

Structure representing the response from a RAS runtime operation.
`#include <esmi_mailbox.h>`

Data Fields

- uint32_t ** buffer_data

  buffer data for all the instances
- uint32_t num_of_instances

  Number of instances.
- uint32_t buffer_size_per_instance

  buffer size per instance
- uint32_t op_buffer_size

6.40.1 Detailed Description

Structure representing the response from a RAS runtime operation.
This structure contains information about the buffer data for multiple instances, the number of instances, buffer sizes, and the status of the operation.

6.40.2 Field Documentation

6.40.2.1 buffer_data

`ras_runtime_response::buffer_data`
buffer data for all the instances
Pointer to an array of pointers, each pointing to buffer data for an instance.

6.40.2.2 num_of_instances

`ras_runtime_response::num_of_instances`
Number of instances.
Number of instances for which the buffer data is maintained.

6.40.2.3 buffer_size_per_instance

`ras_runtime_response::buffer_size_per_instance`
buffer size per instance
Size of the buffer allocated for each instance.

6.40.2.4 op_buffer_size

`ras_runtime_response::op_buffer_size`
operation buffer size .On successful operation op_buffer_size will be number of instances * buffer_size_per_↩ instance
Total size of the operation buffer. On successful operation, this is equal to num_of_instances multiplied by buffer↩ _size_per_instance.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.41 run_time_err_d_in Struct Reference

Get run time error information data_in. Runtime error data_in includes 4 byte aligned offset in instance, runtime error category and zero based index of valid instance number of bytes per each category and number of instances of each category.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t offset

  4 byte aligned offset in instance
- uint8_t category

  Runtime error category.
- uint8_t valid_inst_index

  Valid inst index returned by 61h.

6.41.1 Detailed Description

Get run time error information data_in. Runtime error data_in includes 4 byte aligned offset in instance, runtime error category and zero based index of valid instance number of bytes per each category and number of instances of each category.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.42 run_time_threshold Struct Reference

Configure threshold counters for MCA, DRAM CECC and PCIE. structure consists of error type, error count threshold and max interrupt rate.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t err_type: 2

  10 (PCIE_UE) and 11(RSVD)
- uint16_t err_count_th

  error count threshold
- uint8_t max_intrupt_rate: 4

  Max interrupt rate.

6.42.1 Detailed Description

Configure threshold counters for MCA, DRAM CECC and PCIE. structure consists of error type, error count threshold and max interrupt rate.

6.42.2 Field Documentation

6.42.2.1 err_type

`uint8_t run_time_threshold::err_type`
10 (PCIE_UE) and 11(RSVD)
error type [00(MCA), 01(DRAM CECC)
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.43 set_hsmp_cmds_din Struct Reference

Data in for set enabled HSMP commands. structure contains: rmask (read mask indicator : 1 = for read commands, 0 = for write commands), offset for the 24-bitmask segment. bitmask (24-bit bitmask in hexa decimal).
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t rmask: 1

  1 = for read commands, 0 = for write commands
- uint8_t offset: 4

  Offset id for the 24-bit bitmask segment, starting from 0, 1, 2, 3, etc.
- uint32_t bitmask: 24

  24-bit bit mask segment in hex

6.43.1 Detailed Description

Data in for set enabled HSMP commands. structure contains: rmask (read mask indicator : 1 = for read commands, 0 = for write commands), offset for the 24-bitmask segment. bitmask (24-bit bitmask in hexa decimal).
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.44 statistics Struct Reference

struct containing statistics parameter of interest and output control pstate mappings.
`#include <rmi_mailbox_mi300.h>`

Data Fields

- uint16_t stat_param

  statistics parameter of interest
- uint16_t output_control

  Output control.

6.44.1 Detailed Description

struct containing statistics parameter of interest and output control pstate mappings.
The documentation for this struct was generated from the following file:

- rmi_mailbox_mi300.h

6.45 svi3_vr_cont_data_in Struct Reference

Data in for the SVI3 VR controller temperature by rail . strucutre contains rail mode and svi3 rail index.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t rail_mode: 1
- uint8_t svi3_rail_index: 3

  SVI3 Rail Index.

6.45.1 Detailed Description

Data in for the SVI3 VR controller temperature by rail . strucutre contains rail mode and svi3 rail index.

6.45.2 Field Documentation

6.45.2.1 rail_mode

`uint8_t svi3_vr_cont_data_in::rail_mode`
Read temperature of individual SVI3 rails if the input is 1 else it will read the hottest rail temperature
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.46 svi3_vr_cont_data_out Struct Reference

Data in for the SVI3 VR controller temperature by rail . strucutre contains read svi3 temp data bit and svi3 rail index.
`#include <esmi_mailbox.h>`

Data Fields

- uint32_t svi3_temp: 28

  Read temperature data for SVI3 rail.
- uint8_t svi3_rail_index: 3

  SVI3 Rail Index.

6.46.1 Detailed Description

Data in for the SVI3 VR controller temperature by rail . strucutre contains read svi3 temp data bit and svi3 rail index.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.47 svi_port_domain Struct Reference

struct containing port and slave address.
`#include <rmi_mailbox_mi300.h>`

Data Fields

- uint8_t port: 2

  SVI port.
- uint8_t slave_addr: 3

  slave address

6.47.1 Detailed Description

struct containing port and slave address.
The documentation for this struct was generated from the following file:

- rmi_mailbox_mi300.h

6.48 temp_refresh_rate Struct Reference

DIMM temperature range and refresh rate, temperature update flag.
`#include <esmi_mailbox.h>`

Data Fields

- uint8_t range: 3

  temp refresh rate (3 bit data)
- uint8_t ref_rate: 1

  temp update flag (1 bit data)

6.48.1 Detailed Description

DIMM temperature range and refresh rate, temperature update flag.
The documentation for this struct was generated from the following file:

- esmi_mailbox.h

6.49 xgmi_speed_rate_n_width Struct Reference

struct containing xgmi speed rate in MHZ and link width in units of Gpbs. If link_width[0] = 1 then XGMI link X2 is supported. If link_width[1] = 1 then XGMI link X4 is supported. If Link_width[2] = 1 then XGMI link X4 is supported and similarly if link_width[3] = 1 then XGMI link X8 is supported.
`#include <rmi_mailbox_mi300.h>`

Data Fields

- uint16_t speed_rate

  Speed rate.
- uint8_t link_width: 4

  XGMI link width.

6.49.1 Detailed Description

struct containing xgmi speed rate in MHZ and link width in units of Gpbs. If link_width[0] = 1 then XGMI link X2 is supported. If link_width[1] = 1 then XGMI link X4 is supported. If Link_width[2] = 1 then XGMI link X4 is supported and similarly if link_width[3] = 1 then XGMI link X8 is supported.
The documentation for this struct was generated from the following file:

- rmi_mailbox_mi300.h

Chapter 7

File Documentation

7.1 apml.h File Reference

```
#include <inttypes.h>
#include <stdbool.h>
#include <linux/amd-apml.h>
#include "apml_err.h"
```

Macros

- #define SBRMI "sbrmi"

  SBRMI module //.
- #define SBTSI "sbtsi"

  SBTSI module //.
- #define MAX_DEV_COUNT 8

  APML ADDRESSES count.

Enumerations

- enum sbrmi_outbnd_msg {
  SBRMI_OUTBNDMSG0 = 0x30 , **SBRMI_OUTBNDMSG1** , **SBRMI_OUTBNDMSG2** , **SBRMI_OUTBNDMSG3** ,
  SBRMI_OUTBNDMSG4 , **SBRMI_OUTBNDMSG5** , **SBRMI_OUTBNDMSG6** , **SBRMI_OUTBNDMSG7** }

  SBRMI outband messages.
- enum sbrmi_inbnd_msg {
  SBRMI_INBNDMSG0 = 0x38 , **SBRMI_INBNDMSG1** , **SBRMI_INBNDMSG2** , **SBRMI_INBNDMSG3** ,
  SBRMI_INBNDMSG4 , **SBRMI_INBNDMSG5** , **SBRMI_INBNDMSG6** , **SBRMI_INBNDMSG7** }

  SBRMI inbound messages defined in the APML library.
- enum PROC_DETAILS {
  NOT_SUPPORTED , **LEGACY_PLATFORMS** , **FAM_19_MOD_10** , **FAM_19_MOD_90** ,
  FAM_1A_MOD_00 , **FAM_1A_MOD_10** , **FAM_19_MOD_A0** , **FAM_1A_MOD_50** }

  processor family and model details LEGACY_PLATFORMS: Family:0x17, Model: 30h∼3Fh (Rome), Family:0x19, MOdel: 00h∼0Fh (Milan) FAM_19_MOD_10: Family:0x19, MOdel: 10h∼1Fh (Genoa) FAM_19_MOD_90: Family:0x19, MOdel: 90h∼9Fh (MI300A) FAM_1A_MOD_00: Family:0x1A, MOdel: 00h∼0Fh (Turin) FAM_1A_MOD_10: Family:0x1A, MOdel: 10h∼1Fh (Turin-D) FAM_19_MOD_A0: Family:0x19, Model: A0∼AFh (Genoa Dense) FAM_1A_MOD_50: Family:0x1A, Model: 50∼5Fh (SP7 WH)

Functions

- oob_status_t esmi_oob_read_byte (uint8_t soc_die_num, uint16_t reg_offset, char *file_name, uint8_t *buffer)

  Reads data for the given register.

- oob_status_t esmi_oob_write_byte (uint8_t soc_die_num, uint16_t reg_offset, char *file_name, uint8_t value)
  Writes data to the specified register.
- oob_status_t esmi_oob_rmi_read_byte (uint8_t soc_die_num, uint16_t reg_offset, uint8_t *buffer)
  Reads data for the given RMI register.
- oob_status_t esmi_oob_tsi_read_byte (uint8_t soc_die_num, uint8_t reg_offset, uint8_t *buffer)
  Reads data for the given TSI register.
- oob_status_t esmi_oob_rmi_write_byte (uint8_t soc_die_num, uint16_t reg_offset, uint8_t value)
  Writes data to the specified RMI register.
- oob_status_t esmi_oob_tsi_write_byte (uint8_t soc_die_num, uint8_t reg_offset, uint8_t value)
  Writes data to the specified TSI register.
- oob_status_t esmi_oob_read_mailbox (uint8_t soc_die_num, uint32_t cmd, uint32_t input, uint32_t *buffer)
  Reads mailbox command data.
- oob_status_t esmi_oob_write_mailbox (uint8_t soc_die_num, uint32_t cmd, uint32_t data)
  Writes data to the given mailbox command.
- oob_status_t sbrmi_xfer_msg (uint8_t soc_die_num, struct apml_message *msg)
  Writes data to RMI device file.
- oob_status_t sbtsi_xfer_msg (uint8_t soc_die_num, struct apml_message *msg)
  Writes data to TSI device file.
- oob_status_t validate_sbtsi_module (uint8_t soc_die_num, bool *is_sbtsi)
  Validates sbtsi module is present for the given socket.
- oob_status_t validate_sbrmi_module (uint8_t soc_die_num, bool *is_sbrmi)
  Validates sbrmi module is present for the given socket.
- oob_status_t validate_apml_dependency (uint8_t soc_die_num, bool *is_sbrmi, bool *is_sbtsi)
  Validates sbrmi and sbtsi modules are present for the given socket.

Variables

- const uint16_t sbrmi_addr [MAX_DEV_COUNT]
  SBRMI addresses //.
- const uint16_t sbtsi_addr [MAX_DEV_COUNT]
  SBTSI addresses //.

7.1.1 Detailed Description

Main header file for the APML library. All required function, structure, enum, etc. definitions should be defined in this file.
This header file contains the following: APIs prototype of the APIs exported by the APML library. Description of the API, arguments and return values. The Error codes returned by the API.

7.1.2 Enumeration Type Documentation

7.1.2.1 sbrmi_inbnd_msg

`enum sbrmi_inbnd_msg`
SBRMI inbound messages defined in the APML library.
Usage convention is: • SBRMI::InBndMsg_inst0 is command • SBRMI::InBndMsg_inst[4:1] are 32 bit data • SBRMI::InBndMsg_inst[6:5] are reserved. • SBRMI::InBndMsg_inst7: [7] Must be 1'b1 to send message to firmware SBRMI inband messages

7.1.3 Function Documentation

7.1.3.1 esmi_oob_read_byte()

```
oob_status_t esmi_oob_read_byte (
            uint8_t soc_die_num,
            uint16_t reg_offset,
            char * file_name,
            uint8_t * buffer )
```

Reads data for the given register.

This function will read the data for the given register. NOTE: In future, this API will be deprecated, as individual APIs for read SBRMI and read SBTSI support is available.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*reg_offset*	Register offset for RMI/TSI I/F.
in	*file_name*	Character device file name for RMI/TSI I/F.
out	*buffer*	output value for the register.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.1.3.2 esmi_oob_write_byte()

```
oob_status_t esmi_oob_write_byte (
            uint8_t soc_die_num,
            uint16_t reg_offset,
            char * file_name,
            uint8_t value )
```

Writes data to the specified register.

This function will write the data to the specified register. NOTE: In future, this API will be deprecated, as individual APIs for write SBRMI and write SBTSI support is available.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*file_name*	Character device file name for RMI/TSI I/F.
in	*reg_offset*	Register offset for RMI/TSI I/F.
in	*value*	data to write to the register.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.1.3.3 esmi_oob_rmi_read_byte()

```
oob_status_t esmi_oob_rmi_read_byte (
            uint8_t soc_die_num,
```

```
            uint16_t reg_offset,
            uint8_t * buffer )
```

Reads data for the given RMI register.
This function will read the data for the given RMI register.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*reg_offset*	RMI register offset.
out	*buffer*	output value for the register.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.1.3.4 esmi_oob_tsi_read_byte()

```
oob_status_t esmi_oob_tsi_read_byte (
            uint8_t soc_die_num,
            uint8_t reg_offset,
            uint8_t * buffer )
```

Reads data for the given TSI register.
This function will read the data for the given TSI register.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*reg_offset*	TSI register offset.
out	*buffer*	output value for the register.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.1.3.5 esmi_oob_rmi_write_byte()

```
oob_status_t esmi_oob_rmi_write_byte (
            uint8_t soc_die_num,
            uint16_t reg_offset,
            uint8_t value )
```

Writes data to the specified RMI register.
This function will write the data to the specified RMI register.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*reg_offset*	RMI register offset.
in	*value*	data to write to the TSI register.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.1.3.6 esmi_oob_tsi_write_byte()

```
oob_status_t esmi_oob_tsi_write_byte (
            uint8_t soc_die_num,
            uint8_t reg_offset,
            uint8_t value )
```

Writes data to the specified TSI register.
This function will write the data to the specified TSI register.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*reg_offset*	TSI register offset.
in	*value*	data to write to the TSI register.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.1.3.7 esmi_oob_read_mailbox()

```
oob_status_t esmi_oob_read_mailbox (
            uint8_t soc_die_num,
            uint32_t cmd,
            uint32_t input,
            uint32_t * buffer )
```

Reads mailbox command data.
This function will read mailbox command data.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*cmd*	mailbox command.
in	*input*	data.
out	*buffer*	output data for the given mailbox command.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.1.3.8 esmi_oob_write_mailbox()

```
oob_status_t esmi_oob_write_mailbox (
            uint8_t soc_die_num,
            uint32_t cmd,
            uint32_t data )
```

Writes data to the given mailbox command.
This function will writes data to mailbox command.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*cmd*	mailbox command.
in	*data*	input data.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.1.3.9 sbrmi_xfer_msg()

```
oob_status_t sbrmi_xfer_msg (
            uint8_t soc_die_num,
            struct apml_message * msg )
```

Writes data to RMI device file.
This function will write data to RMI device file, through ioctl.

Parameters

in	*soc_die_num*	Socket Die index.
in	*msg*	struct apml_message which contains information about the protocol, input/output data etc.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.1.3.10 sbtsi_xfer_msg()

```
oob_status_t sbtsi_xfer_msg (
            uint8_t soc_die_num,
            struct apml_message * msg )
```

Writes data to TSI device file.
This function will write data to TSI device file, through ioctl.

Parameters

in	*soc_die_num*	Socket Die index.
in	*msg*	struct apml_message which contains information about the protocol, input/output data etc.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.1.3.11 validate_sbtsi_module()

```
oob_status_t validate_sbtsi_module (
            uint8_t soc_die_num,
            bool * is_sbtsi )
```

Validates sbtsi module is present for the given socket.
This function will validate sbtsi module is present for the specified socket.

Parameters

in	*soc_die_num*	Socket Die index.
out	*is_sbtsi*	returns true if the sbtsi is present else false

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.1.3.12 validate_sbrmi_module()

```
oob_status_t validate_sbrmi_module (
            uint8_t soc_die_num,
            bool * is_sbrmi )
```

Validates sbrmi module is present for the given socket.
This function will validate sbrmi module is present for the specified socket.

Parameters

in	*soc_die_num*	Socket Die index.
out	*is_sbrmi*	returns true if the sbrmi is present else false

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.1.3.13 validate_apml_dependency()

```
oob_status_t validate_apml_dependency (
            uint8_t soc_die_num,
            bool * is_sbrmi,
            bool * is_sbtsi )
```

Validates sbrmi and sbtsi modules are present for the given socket.

This function will validate sbrmi and sbtsi modules are present for the specified socket.

Parameters

in	*soc_die_num*	Socket Die index.
out	*is_sbrmi*	returns true if the sbrmi is present else false
out	*is_sbtsi*	returns true if the sbtsi is present else false

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.2 apml_err.h File Reference

Macros

- #define OOB_CPUID_MSR_ERR_BASE 0x800

  CPUID MSR FW error code.
- #define OOB_MAILBOX_ERR_BASE 0x900

  MAILBOX FW error code.

Enumerations

- enum oob_status_t {
  OOB_SUCCESS = 0 , OOB_NOT_FOUND , OOB_PERMISSION , OOB_NOT_SUPPORTED ,
  OOB_FILE_ERROR , OOB_INTERRUPTED , OOB_UNEXPECTED_SIZE , OOB_UNKNOWN_ERROR ,
  OOB_ARG_PTR_NULL , OOB_NO_MEMORY , OOB_NOT_INITIALIZED , OOB_TRY_AGAIN ,
  OOB_INVALID_INPUT , OOB_CMD_TIMEOUT , OOB_INVALID_MSGSIZE , OOB_FORMATTING_ERROR
  ,
  OOB_CPUID_MSR_ERR_START , OOB_CPUID_MSR_CMD_TIMEOUT , OOB_CPUID_MSR_CMD_WARM_RESET
  , OOB_CPUID_MSR_CMD_UNKNOWN_FMT ,
  OOB_CPUID_MSR_CMD_INVAL_RD_LEN , OOB_CPUID_MSR_CMD_EXCESS_DATA_LEN , OOB_CPUID_MSR_CMD_INV
  , OOB_CPUID_MSR_CMD_UNSUPP ,
  OOB_CPUID_MSR_CMD_ABORTED , **OOB_CPUID_MSR_ERR_END** , **OOB_MAILBOX_ERR_START** ,
  OOB_MAILBOX_CMD_ABORTED ,
  OOB_MAILBOX_CMD_UNKNOWN , OOB_MAILBOX_CMD_INVAL_CORE , OOB_MAILBOX_ADD_ERR_DATA
  , OOB_MAILBOX_INVALID_INPUT_ARGS ,
  OOB_MAILBOX_INVALID_OOBRAS_CONFIG , OOB_MAILBOX_NOT_READY , **OOB_MAILBOX_ERR_**↩
  END }

  Error codes retured by APML_ERR functions.

Functions

- oob_status_t errno_to_oob_status (int err)

  convert linux error to esmi error.
- char * esmi_get_err_msg (oob_status_t oob_err)

  Get the error string message for esmi oob errors.

7.2.1 Detailed Description

Header file for the APML library error/return codes.
This header file has error/return codes for the API.

7.2.2 Enumeration Type Documentation

7.2.2.1 oob_status_t

enum oob_status_t
Error codes retured by APML_ERR functions.

Enumerator

OOB_SUCCESS	Operation was successful.
OOB_NOT_FOUND	An item was searched for but not found.
OOB_PERMISSION	many functions require root access to run. Permission denied/EACCESS file error.
OOB_NOT_SUPPORTED	The requested information or action is not available for the given input, on the given system
OOB_FILE_ERROR	Problem accessing a file. This may because the operation is not supported by the Linux kernel version running on the executing machine
OOB_INTERRUPTED	execution of function An interrupt occurred during
OOB_UNEXPECTED_SIZE	was read An unexpected amount of data
OOB_UNKNOWN_ERROR	An unknown error occurred.
OOB_ARG_PTR_NULL	Parsed argument ptr null.
OOB_NO_MEMORY	Not enough memory to allocate.
OOB_NOT_INITIALIZED	APML object not initialized.
OOB_TRY_AGAIN	No match Try again.
OOB_INVALID_INPUT	Input value is invalid.
OOB_CMD_TIMEOUT	Command timed out.
OOB_INVALID_MSGSIZE	Mesg size too long.
OOB_FORMATTING_ERROR	Formatting or encoding error.
OOB_CPUID_MSR_CMD_TIMEOUT	RMI cmd timeout.
OOB_CPUID_MSR_CMD_WARM_RESET	Warm reset during RMI cmd.
OOB_CPUID_MSR_CMD_UNKNOWN_FMT	Cmd fmt field not recongnised.
OOB_CPUID_MSR_CMD_INVAL_RD_LEN	RMI cmd invalid read len.
OOB_CPUID_MSR_CMD_EXCESS_DATA_LEN	excess data
OOB_CPUID_MSR_CMD_INVAL_THREAD	Invalid thread selected.
OOB_CPUID_MSR_CMD_UNSUPP	Cmd not supported.
OOB_CPUID_MSR_CMD_ABORTED	Cmd aborted.
OOB_MAILBOX_CMD_ABORTED	Mailbox cmd aborted.
OOB_MAILBOX_CMD_UNKNOWN	Unknown mailbox cmd.
OOB_MAILBOX_CMD_INVAL_CORE	Invalid core.
OOB_MAILBOX_ADD_ERR_DATA	Additional error data with error.
OOB_MAILBOX_INVALID_INPUT_ARGS	Invalid Input Arguments.
OOB_MAILBOX_INVALID_OOBRAS_CONFIG	Invalid OOB RAS config.
OOB_MAILBOX_NOT_READY	Data not ready.

7.3 apml_recovery.h File Reference

Enumerations

- enum apml_client { **DEV_SBRMI** = 0x0 , **DEV_SBTSI** }

APML Client Devices.

Functions

- oob_status_t apml_recover_dev (uint8_t soc_die_num, uint8_t client)

  Recover the APML client device for the given socket.

7.3.1 Detailed Description

Header file for the APML recovery flow

7.3.2 Function Documentation

7.3.2.1 apml_recover_dev()

```
oob_status_t apml_recover_dev (
            uint8_t soc_die_num,
            uint8_t client )
```

Recover the APML client device for the given socket.
This function will recover the APML client device and returns successful on recovery or error if recovery is unsuccessful
NOTE: This fix is a software workaround for the erratum, Erratum:1444, "Advanced Platform Management Link (↩ APML) May Cease to Function After Incomplete Read Transaction" Further details can be found in mentioned tech doc https://www.amd.com/system/files/TechDocs/57095-PUB_1.00.pdf

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*client*	DEV_SBRMI[0]/DEV_SBTSI[1] enum: apml_client

Return values

OOB_SUCCESS	is returned upon successful recovery
Non-zero	is returned upon failure.

7.4 esmi_cpuid_msr.h File Reference

```
#include "apml_err.h"
```

Data Structures

- struct processor_info

  Read Proccessor Info.

Enumerations

- enum cpuid_reg { **EAX** = 0 , **EBX** , **ECX** , **EDX** }

Functions

- oob_status_t esmi_get_vendor_id (uint8_t soc_die_num, char *vendor_id)

Get the number of logical cores per socket.

- oob_status_t esmi_get_processor_info (uint8_t soc_die_num, struct processor_info *proc_info)

  Get the number of logical cores per socket.
- oob_status_t esmi_get_logical_cores_per_socket (uint8_t soc_die_num, uint32_t *logical_cores_per_socket)

  Get the number of logical cores per socket.
- oob_status_t esmi_get_threads_per_core (uint8_t soc_die_num, uint32_t *threads_per_core)

  Get number of threads per core.
- oob_status_t esmi_oob_read_msr (uint8_t soc_die_num, uint32_t thread, uint32_t msraddr, uint64_t *buffer)

  Read the MCA MSR register for a given thread.
- oob_status_t esmi_oob_cpuid (uint8_t soc_die_num, uint32_t thread, uint32_t *eax, uint32_t *ebx, uint32_t *ecx, uint32_t *edx)

  Read CPUID functionality for a particular thread in a system.
- oob_status_t esmi_oob_cpuid_eax (uint8_t soc_die_num, uint32_t thread, uint32_t fn_eax, uint32_t fn_ecx, uint32_t *eax)

  Read eax register on CPUID functionality.
- oob_status_t esmi_oob_cpuid_ebx (uint8_t soc_die_num, uint32_t thread, uint32_t fn_eax, uint32_t fn_ecx, uint32_t *ebx)

  Read ebx register on CPUID functionality.
- oob_status_t esmi_oob_cpuid_ecx (uint8_t soc_die_num, uint32_t thread, uint32_t fn_eax, uint32_t fn_ecx, uint32_t *ecx)

  Read ecx register on CPUID functionality.
- oob_status_t esmi_oob_cpuid_edx (uint8_t soc_die_num, uint32_t thread, uint32_t fn_eax, uint32_t fn_ecx, uint32_t *edx)

  Read edx register on CPUID functionality.
- oob_status_t read_max_threads_per_l3 (uint8_t soc_die_num, uint32_t *threads_l3)

  Read max threads per L3 cache.

Variables

- struct processor_info plat_info [1]

  Platform Info instance.

7.4.1 Detailed Description

Header file for the APML library cpuid and msr read functions. All required function, structure, enum and protocol specific data etc. definitions should be defined in this header.
This header file contains the following: APIs prototype of the APIs exported by the APML library. Description of the API, arguments and return values. The Error codes returned by the API.

7.4.2 Enumeration Type Documentation

7.4.2.1 cpuid_reg

`enum cpuid_reg`
CPUID register indexes 0 for EAX, 1 for EBX, 2 ECX and 3 for EDX

7.5 esmi_mailbox.h File Reference

```
#include "apml_common.h"
#include "apml_err.h"
#include <stdbool.h>
```

Data Structures

- struct dimm_power

  DIMM power(mW), update rate(ms) and dimm address.
- struct dimm_pow_din

  Input for DIMM power, DIMM address and power reporting flag. pow_reporting_flag: Valid values are 0, 1, 2.
- struct dimm_pow_dout

  DIMM power(mW), update rate(ms) and DIMM address.
- struct dimm_thermal_din

  DIMM address and thermal sensor reporting flag. thermal_flag: Valid values are 0 and 1.
- struct dimm_thermal

  DIMM thermal sensor (degree C), update rate and dimm address.
- struct temp_refresh_rate

  DIMM temperature range and refresh rate, temperature update flag.
- struct pci_address

  PCI address information .PCI address includes 4 bit segment, 12 bit aligned offset, 8 bit bus, 5 bit device info and 3 bit function.
- struct pcie_link_training

  PCIe Halt Link Training information. It includes 10 bit link ID, 4 bit Pcie Port number range from 0 - 7.
- struct dpm_level

  Max and min LCK DPM level on a given NBIO ID. Valid Max and min DPM level values are 0 - 1.
- struct lclk_dpm_level_range

  Max and Min Link frequency clock (LCLK) DPM level on a socket. 8 bit NBIO ID, dpm_level struct containing 8 bit max DPM level, 8 bit min DPM level.
- struct nbio_err_log

  NBIO quadrant(8 bit data) and NBIO register offset(24 bit) data.
- struct max_ddr_bw

  Structure for Max DDR bandwidth and utilization. It contains max bandwidth(12 bit data) in GBps, current utilization bandwidth(12 bit data) in GBps, current utilized bandwidth(8 bit data) in percentage.
- struct mca_bank

  MCA bank information.It contains 16 bit Index for MCA Bank and 16 bit offset.
- struct link_id_bw_type

  APML LINK ID and Bandwidth type Information.It contains APML LINK ID Encoding. SP7 platforms valid Link ID encodings are 1(P0), 2(P1), 4(P2), 8(P3), 16(G0), 32(G1), 64(G2), 128(G3), 256(P4) and 512(P5) and rest of the Non-MI300 Platforms Valid Link ID encodings are 1(P0), 2(P1), 4(P2), 8(P3), 16(G0), 32(G1), 64(G2), 128(G3). Valid APML MI300 APML LINK ID Encoding. Valid Link ID encodings are 3(P2), 4(P3), 8(G0), 9(G1), 10(G2), 11(G3), 12(G4), 13(G5), 14(G6), 15(G7). IO Bandwidth types 1(Aggregate_BW), 2 (Read BW), 4 (Write BW).
- struct pstate_freq

  DF P-state frequency.It includes mem clock(16 bit data) frequency (DRAM memory clock), data fabric clock (12 bit data), UMC clock divider (UMC) (1 bit data).
- struct ras_df_err_chk

  RAS df err validity check output status. Structure contains the following members. df_block_instances number of block instance with error log to report (0 - 256) err_log_len length of error log in bytes per instance (0 - 256).
- union ras_df_err_dump

  RAS df error dump input. Union contains the following members. input[0] 4 byte alligned offset in error log (0 - 255) input[1] DF block ID (0 - 255) input[2] Zero based index of DF block instance (0 - 255) input[3] Reserved data_in 32-bit data input.
- struct ras_override_delay

  BMC RAS override delay reset CPU on sync flood. The structure contains delay value override in mins [5 -120 mins], disable delay counter and stop delay counter. If disable delay counter is set ResetCpuOnSyncFlood response will NOT be delayed in the next SyncFlood regardless of the value specified in delay_val_override. If StopDelayCounter is set it stops the active delay countdown which extends the DelayResetCpuOnSyncFlood indefinitely and system will not reset.
- struct ras_rt_err_req_type

Get the Error type and request type. Supported Runtime error type[1:0]:

- struct ras_rt_valid_err_inst

  Number of valid error instance per category. It consists of number of bytes per each category and number of instances of each category.

- struct run_time_err_d_in

  Get run time error information data_in. Runtime error data_in includes 4 byte aligned offset in instance, runtime error category and zero based index of valid instance number of bytes per each category and number of instances of each category.

- struct run_time_threshold

  Configure threshold counters for MCA, DRAM CECC and PCIE. structure consists of error type, error count threshold and max interrupt rate.

- struct oob_config_d_in

  Configure oob state infrastructure in SoC. structure consists of mca_oob_misc0_ec_enable, dram_cecc_oob_ec_↩ mode, dram_cecc_leak_rate, pcie_err_reporting_en and mca_err_reporting_en.

- struct dimm_spd_d_in

  dimm_spd_d_in: strucutre contains following memebers to be passed as input, to get 4 byte DIMM spd register data

- struct dimm_sb_reg_d_in

  dimm_sb_reg_d_in: structure contains following members to be passed as input, to get 4 byte DIMM sb register data

- struct dimm_spd_write

  SPD side band write: strucutre contains dimm address, LID of device, Register offset, Register space, and data to write.

- struct dimm_sb_reg_write

  Dimm side band register write: strucutre contains dimm address, LID of device, Register offset, Register space, and data to write.

- struct svi3_vr_cont_data_in

  Data in for the SVI3 VR controller temperature by rail . strucutre contains rail mode and svi3 rail index.

- struct svi3_vr_cont_data_out

  Data in for the SVI3 VR controller temperature by rail . strucutre contains read svi3 temp data bit and svi3 rail index.

- struct ras_runtime_response

  Structure representing the response from a RAS runtime operation.

- struct bmc_ras_runtime_din

  Structure representing BMC RAS runtime error information.

- struct core_floor_limit_din

  Structure for APML floor core limit input.

- struct pow_eff_mode

  Structure for Power Efficiency Mode selection.

- struct set_hsmp_cmds_din

  Data in for set enabled HSMP commands. structure contains: rmask (read mask indicator : 1 = for read commands, 0 = for write commands), offset for the 24-bitmask segment. bitmask (24-bit bitmask in hexa decimal).

- struct get_hsmp_cmds_din

  Data in for get enabled HSMP commands. structure contains: rmask (read mask indicator : 1 = for read commands, 0 = for write commands), offset for the 24-bitmask segment.

- struct get_hsmp_cmds_dout

  Data out for get enabled HSMP commands. structure contains: offset for the 24-bitmask segment, bitmask (24-bit bitmask in hexa decimal)

- struct dram_thr_with_status

  Structure for dram throttle percentage with throttle status.

- struct oob_err_inj_types

  Error injection types containing generic error types, vendor specfic error types, error size and vendor flag.

- struct hottest_dimm_temp_refresh_rate

  Hottest DIMM address, temperature range and refresh rate.

Macros

- #define DRAM_CECC_OOB_EC_MODE 1

  DRAM CECC OOB EC Mode //.
- #define ERR_COUNT_TH 2

  Error count threshold position //.
- #define DRAM_CECC_LEAK_RATE 3

  DRAM CECC Leak rate //.
- #define PCIE_ERR_REPORT_EN 8

  PCIE oob counter enable //.
- #define MCA_TH_INTR 11

  MCA threshold interrupt //.
- #define CECC_TH_INTR 12

  CECC threshold interrupt //.
- #define PCIE_TH_INTR 13

  PCIE threshold interrupt //.
- #define MCA_MAX_INTR_RATE 15

  MCA max interrupt rate //.
- #define MAX_INTR_RATE_POS 18

  Max interrupt rate position //.
- #define DRAM_CECC_MAX_INTR_RATE 19

  DRAM CECC Max interrupt rate //.
- #define PCIE_MAX_INTR_RATE 23

  PCIE Max interrupt rate //.
- #define MCA_ERR_REPORT_EN 31

  MCA error report enable //.
- #define LEGACY_ENCODING_SIZE 8

  Legacy encoding size //.
- #define SP7_ENCODING_SIZE 10

  Sp7 APML encoding size //.
- #define MCA_DEBUG_LOG_ID 32

  RAS MCA Dump //.
- #define OOB_ERR_TYPES_BLK_ID 253

  OOB Error Types Block ID //.
- #define MCA_ERR_CATEGORY 0

  MCA Error category //.
- #define DRAM_ECC_ERR_CATEGORY 1

  DRAM ECC Error category //.
- #define PCIE_ERR_CATEGORY 2

  DRAM ECC Error category //.

Enumerations

- enum esb_mailbox_commmands {
  READ_PACKAGE_POWER_CONSUMPTION = 0x1 , **WRITE_PACKAGE_POWER_LIMIT** , **READ_PACKAGE_POWER_LIMIT** , **READ_MAX_PACKAGE_POWER_LIMIT** ,
  READ_TDP , **READ_MAX_cTDP** , **READ_MIN_cTDP** , **READ_BIOS_BOOST_Fmax** ,
  READ_APML_BOOST_LIMIT , **WRITE_APML_BOOST_LIMIT** , **WRITE_APML_BOOST_LIMIT_ALLCORES** , **READ_DRAM_THROTTLE** ,
  WRITE_DRAM_THROTTLE , **READ_PROCHOT_STATUS** , **READ_PROCHOT_RESIDENCY** , **READ_NBIO_ERROR_LOGGING_REGISTER** = 0x11 ,
  READ_IOD_BIST = 0x13 , **READ_CCD_BIST_RESULT** , **READ_CCX_BIST_RESULT** , **READ_PACKAGE_CCLK_FREQ_LIMIT** ,

READ_PACKAGE_C0_RESIDENCY , **READ_DDR_BANDWIDTH** = 0x18 , **READ_SMU_FW_VER** = 0x1C , **READ_PPIN_FUSE** = 0x1F ,
GET_POST_CODE = 0x20 , **GET_RTC** , **WRITE_BMC_REPORT_DIMM_POWER** = 0X40 , **WRITE_BMC_REPORT_DIMM_THERMAL_SENSOR** ,
READ_BMC_RAS_PCIE_CONFIG_ACCESS , **READ_BMC_RAS_MCA_VALIDITY_CHECK** , **READ_BMC_RAS_MCA_MSR_DUMP** , **READ_BMC_RAS_FCH_RESET_REASON** ,
READ_DIMM_TEMP_RANGE_AND_REFRESH_RATE , **READ_DIMM_POWER_CONSUMPTION** , **READ_DIMM_THERMAL_SENSOR** , **READ_PWR_CURRENT_ACTIVE_FREQ_LIMIT_SOCKET** ,
READ_PWR_CURRENT_ACTIVE_FREQ_LIMIT_CORE , **READ_PWR_SVI_TELEMETRY_ALL_RAILS** , **READ_SOCKET_FREQ_RANGE** , **READ_CURRENT_IO_BANDWIDTH** ,
READ_CURRENT_XGMI_BANDWIDTH , **WRITE_GMI3_LINK_WIDTH_RANGE** , **WRITE_XGMI_LINK_WIDTH_RANGE** , **WRITE_APB_DISABLE** ,
WRITE_APB_ENABLE , **READ_CURRENT_DFPSTATE_FREQUENCY** , **WRITE_LCLK_DPM_LEVEL_RANGE** , **READ_BMC_RAPL_UNITS** ,
READ_BMC_RAPL_CORE_LO_COUNTER , **READ_BMC_RAPL_CORE_HI_COUNTER** , **READ_BMC_RAPL_PKG_COUNTER** , **READ_BMC_CPU_BASE_FREQUENCY** ,
READ_BMC_CONTROL_PCIE_GEN5_RATE , **READ_RAS_LAST_TRANS_ADDR_CHK** , **READ_RAS_LAST_TRANS_ADDR_DUMP** , **WRITE_PWR_EFFICIENCY_MODE** ,
WRITE_DF_PSTATE_RANGE , **READ_LCLK_DPM_LEVEL_RANGE** , **READ_UCODE_REVISION** , **GET_BMC_RAS_RUNTIME_ERR_VALIDITY_CHECK** ,
GET_BMC_RAS_RUNTIME_ERR_INFO , **SET_BMC_RAS_ERR_THRESHOLD** , **SET_BM_RAS_OOB_CONFIG** , **GET_BMC_RAS_OOB_CONFIG** ,
SET_BMC_PCIE_CONFIG = 0x68 , **BMC_RAS_DELAY_RESET_ON_SYNCFLOOD_OVERRIDE** = 0x6A , **READ_BMC_RAS_RESET_ON_SYNC_FLOOD** , GET_DIMM_SPD = 0x70 ,
SET_DIMM_SPD , WRITE_PCIE_LINK_CONTROL , **SET_XGMI_PSTATE_RANGE** , **CPU_RAIL_ISO_FREQ_POLICY** ,
PC6_ENABLE , **DFC_ENABLE** , **CC6_ENABLE** , **GET_DRAM_THROTTLE_CHANNELS** = 0x78 ,
SET_PCIE_LINK_TRAINING , **GET_CCD_POW_CONSUMPTION** = 0x7A , **GET_T_DELTA** , **GET_SVI3_VR_CONTROLLER_TEMP_BY_RAIL** ,
ENABLED_HSMP_COMMANDS , **GET_SET_APML_FLOOR_LIMIT** , **GET_SET_SOC_DIMM_POWER_LIMIT** }

Mailbox message types defined in the APML library.

- enum apml_io_bw_encoding { **AGG_BW** = BIT(0) , **RD_BW** = BIT(1) , **WR_BW** = BIT(2) }

  APML IO Bandwidth Encoding defined in the APML library.

- enum apml_link_id_encoding {
  P0 = BIT(0) , **P1** = BIT(1) , **P2** = BIT(2) , **P3** = BIT(3) ,
  G0 = BIT(4) , **G1** = BIT(5) , **G2** = BIT(6) , **G3** = BIT(7) }

  APML IO LINK ID Encoding defined in the APML library.

Functions

- oob_status_t read_socket_power (uint8_t soc_die_num, uint32_t *buffer)

  Get the power consumption of the socket.

- oob_status_t read_socket_power_limit (uint8_t soc_die_num, uint32_t *buffer)

  Get the current power cap/limit value for a given socket.

- oob_status_t read_max_socket_power_limit (uint8_t soc_die_num, uint32_t *buffer)

  Get the maximum value that can be assigned as a power cap/limit for a given socket.

- oob_status_t write_socket_power_limit (uint8_t soc_die_num, uint32_t limit)

  Set the power cap/limit value for a given socket.

- oob_status_t read_esb_boost_limit (uint8_t soc_die_num, uint32_t value, uint32_t *buffer)

  Get the Out-of-band boostlimit value for a given core.

- oob_status_t read_bios_boost_fmax (uint8_t soc_die_num, uint32_t value, uint32_t *buffer)

  Get the In-band maximum boostlimit value for a given core.

- oob_status_t write_esb_boost_limit (uint8_t soc_die_num, uint32_t cpu_ind, uint32_t limit)

  Set the Out-of-band boostlimit value for a given core.

- oob_status_t write_esb_boost_limit_allcores (uint8_t soc_die_num, uint32_t limit)

  Set the boostlimit value for the whole socket (whole system).
- oob_status_t read_tdp (uint8_t soc_die_num, uint32_t *buffer)

  Get the Thermal Design Power limit TDP of the socket with provided Socket Die index.
- oob_status_t read_max_tdp (uint8_t soc_die_num, uint32_t *buffer)

  Get the Maximum Thermal Design Power limit TDP of the socket with provided Socket Die index.
- oob_status_t read_min_tdp (uint8_t soc_die_num, uint32_t *buffer)

  Get the Minimum Thermal Design Power limit TDP of the socket.
- oob_status_t read_prochot_status (uint8_t soc_die_num, uint32_t *buffer)

  Get the Prochot Status of the socket with provided Socket Die index.
- oob_status_t read_prochot_residency (uint8_t soc_die_num, float *buffer)

  Get the Prochot Residency (since the boot time or last read of Prochot Residency) of the socket.
- oob_status_t read_dram_throttle (uint8_t soc_die_num, uint32_t *buffer)

  Read Dram Throttle will always read the highest percentage value.
- oob_status_t write_dram_throttle (uint8_t soc_die_num, uint32_t limit)

  Set Dram Throttle value in terms of percentage.
- oob_status_t read_vddio_mem_power (uint8_t soc_die_num, uint32_t *buffer)

  Read VDDIOMem Power returns the estimated VDDIOMem power consumed within the socket.
- oob_status_t read_nbio_error_logging_register (uint8_t soc_die_num, struct nbio_err_log nbio, uint32_t *buffer)

  Read NBIO Error Logging Register.
- oob_status_t read_iod_bist (uint8_t soc_die_num, uint32_t *buffer)

  Read IOD Bist status.
- oob_status_t read_ccd_bist_result (uint8_t soc_die_num, uint32_t input, uint32_t *buffer)

  Read CCD Bist status. Results are read for each CCD present in the system.
- oob_status_t read_ccx_bist_result (uint8_t soc_die_num, uint32_t value, uint32_t *ccx_bist)

  Read CPU Core Complex Bist result. results are read for each Logical CCX instance number and returns a value which is the concatenation of L3 pass status and all cores in the complex(n:0).
- oob_status_t read_cclk_freq_limit (uint8_t soc_die_num, uint32_t *cclk_freq)

  Read CCLK frequency limit for the given socket.
- oob_status_t read_socket_c0_residency (uint8_t soc_die_num, uint32_t *c0_res)

  Read socket C0 residency.
- oob_status_t read_ddr_bandwidth (uint8_t soc_die_num, struct max_ddr_bw *max_ddr)

  Get the Theoretical maximum DDR Bandwidth of the system in GB/s, Current utilized DDR Bandwidth (Read + Write) in GB/s and Current utilized DDR Bandwidth as a percentage of theoretical maximum. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.
- oob_status_t write_bmc_report_dimm_power (uint8_t soc_die_num, struct dimm_power dp_info)

  Set DIMM Power consumption in mwatts.
- oob_status_t write_bmc_report_dimm_thermal_sensor (uint8_t soc_die_num, struct dimm_thermal dt_info)

  Set DIMM thermal Sensor in degree Celcius.
- oob_status_t read_bmc_ras_pcie_config_access (uint8_t soc_die_num, struct pci_address pci_addr, uint32_t *out_buf)

  Read BMC RAS PCIE config access.
- oob_status_t read_bmc_ras_mca_validity_check (uint8_t soc_die_num, uint16_t *bytes_per_mca, uint16_t *mca_banks)

  Read number of MCA banks with valid status after a fatal error.
- oob_status_t read_bmc_ras_mca_msr_dump (uint8_t soc_die_num, struct mca_bank mca_dump, uint32_t *out_buf)

  Read data from mca bank reported by bmc ras mca validity check.
- oob_status_t read_bmc_ras_fch_reset_reason (uint8_t soc_die_num, uint32_t input, uint32_t *out_buf)

  Read FCH reason code from the previous reset.

- oob_status_t read_dimm_temp_range_and_refresh_rate (uint8_t soc_die_num, uint32_t dimm_addr, struct temp_refresh_rate *rate)

  Read DIMM temperature range and refresh rate.
- oob_status_t read_dimm_power_consumption (uint8_t soc_die_num, uint32_t dimm_addr, struct dimm_power *dimm_pow)

  Read DIMM power consumption.
- oob_status_t read_dimm_thermal_sensor (uint8_t soc_die_num, uint32_t dimm_addr, struct dimm_thermal *dimm_temp)

  Read DIMM thermal sensor.
- oob_status_t read_pwr_current_active_freq_limit_socket (uint8_t soc_die_num, uint16_t *freq, char **source_type)

  Read current active frequency limit per socket.
- oob_status_t read_pwr_current_active_freq_limit_core (uint8_t soc_die_num, uint32_t core_id, uint16_t *base_freq)

  Read current active frequency limit set per core.
- oob_status_t read_pwr_svi_telemetry_all_rails (uint8_t soc_die_num, uint32_t *power)

  Read SVR based telemtry for all rails.
- oob_status_t read_socket_freq_range (uint8_t soc_die_num, uint16_t *fmax, uint16_t *fmin)

  Read socket frequency range.
- oob_status_t read_current_io_bandwidth (uint8_t soc_die_num, struct link_id_bw_type link, uint32_t *io_bw)

  Read current bandwidth on IO Link.
- oob_status_t read_current_xgmi_bandwidth (uint8_t soc_die_num, struct link_id_bw_type link, uint32_t *xgmi_bw)

  Read current bandwidth on xGMI Link.
- oob_status_t write_gmi3_link_width_range (uint8_t soc_die_num, uint8_t min_link_width, uint8_t max_link_width)

  Set the max and min width of GMI3 link.
- oob_status_t write_xgmi_link_width_range (uint8_t soc_die_num, uint8_t min_link_width, uint8_t max_link_width)

  Set the max and min width of xGMI link.
- oob_status_t write_apb_disable (uint8_t soc_die_num, uint8_t df_pstate, bool *prochot_asserted)

  Set the APBDisabled.
- oob_status_t write_apb_enable (uint8_t soc_die_num, bool *prochot_asserted)

  Enable the DF p-state performance boost algorithm.
- oob_status_t read_current_dfpstate_frequency (uint8_t soc_die_num, struct pstate_freq *df_pstate)

  Read current DF p-state frequency .
- oob_status_t write_lclk_dpm_level_range (uint8_t soc_die_num, struct lclk_dpm_level_range lclk)

  Set the max and min LCK DPM Level on a given NBIO per socket.
- oob_status_t read_bmc_rapl_units (uint8_t soc_die_num, uint8_t *tu_value, uint8_t *esu_value)

  Read RAPL (Running Average Power Limit) Units.
- oob_status_t read_bmc_cpu_base_frequency (uint8_t soc_die_num, uint16_t *base_freq)

  Read RAPL base frequency per CPU socket.
- oob_status_t read_bmc_control_pcie_gen5_rate (uint8_t soc_die_num, uint8_t rate, uint8_t *mode)

  Control PCIe Rate on Gen5-Capable devices..
- oob_status_t read_rapl_core_energy_counters (uint8_t soc_die_num, uint32_t core_id, double *energy_counters)

  Read RAPL core energy counters.
- oob_status_t read_rapl_pckg_energy_counters (uint8_t soc_die_num, double *energy_counters)

  Read RAPL package energy counters.
- oob_status_t write_pwr_efficiency_mode (uint8_t soc_die_num, uint8_t mode)

  Write power efficiency profile policy.
- oob_status_t write_df_pstate_range (uint8_t soc_die_num, uint8_t max_pstate, uint8_t min_pstate)

Write df pstate range.

- oob_status_t read_lclk_dpm_level_range (uint8_t soc_die_num, uint8_t nbio_id, struct dpm_level *dpm)

  Read LCLK Max and Min DPM level range.
- oob_status_t read_ucode_revision (uint8_t soc_die_num, uint32_t *ucode_rev)

  Read ucode revision.
- oob_status_t read_ras_df_err_validity_check (uint8_t soc_die_num, uint8_t df_block_id, struct ras_df_err_chk *err_chk)

  Read number of instances of DF blocks of type DF_BLOCK_ID with errors.
- oob_status_t read_ras_df_err_dump (uint8_t soc_die_num, union ras_df_err_dump ras_err, uint32_t *data)

  Read RAS DF error dump.
- oob_status_t reset_on_sync_flood (uint8_t soc_die_num, uint32_t *ack_resp)

  Read BMC RAS reset on sync flood.
- oob_status_t override_delay_reset_on_sync_flood (uint8_t soc_die_num, struct ras_override_delay data_in, bool *ack_resp)

  Overrides delay reset cpu on sync flood value.
- oob_status_t get_post_code (uint8_t soc_die_num, uint32_t offset, uint32_t *post_code)

  Read post code.
- oob_status_t get_bmc_ras_run_time_err_validity_ck (uint8_t soc_die_num, struct ras_rt_err_req_type err_category, struct ras_rt_valid_err_inst *inst)

  Reads number of valid error instances per category.
- oob_status_t get_bmc_ras_run_time_error_info (uint8_t soc_die_num, struct run_time_err_d_in d_in, uint32_t *err_info)

  Reads BMC RAS runtime error information.
- oob_status_t set_bmc_ras_err_threshold (uint8_t soc_die_num, struct run_time_threshold th)

  Sets BMC RAS error threshold.
- oob_status_t set_bmc_ras_oob_config (uint8_t soc_die_num, struct oob_config_d_in d_in)

  Configures OOB state infrastructure in SoC.
- oob_status_t get_bmc_ras_oob_config (uint8_t soc_die_num, uint32_t *oob_config)

  Reads the current status of OOB state infrastructure in SoC.
- oob_status_t read_ppin_fuse (uint8_t soc_die_num, uint64_t *data)

  Get the 64 bit PPIN fuse.
- oob_status_t read_rtc (uint8_t soc_die_num, uint64_t *rtc)

  Read RTC.
- oob_status_t read_dimm_spd_register (uint8_t soc_die_num, struct dimm_spd_d_in spd_d_in, uint32_t *spd_data)

  Get SPD SB data.
- oob_status_t get_dimm_sb_register (uint8_t soc_die_num, struct dimm_sb_reg_d_in d_in, uint32_t *reg_data)

  Get dimm SB register data.
- oob_status_t write_dimm_spd_register (uint8_t soc_die_num, struct dimm_spd_write d_in)

  Set SPD SB data.
- oob_status_t set_dimm_sb_register_data (uint8_t soc_die_num, struct dimm_sb_reg_write d_in)

  Set dimm SB data.
- oob_status_t get_dimm_serial_num (uint8_t soc_die_num, uint8_t dimm_addr, uint32_t *serial_num)

  Get DIMM serial number.
- oob_status_t read_smu_fw_ver (uint8_t soc_die_num, uint32_t *smu_fw_ver)

  Get SMU FW version.
- oob_status_t write_bmc_pcie_config (uint8_t soc_die_num, struct pci_address pci_addr, uint32_t pcie_data, uint32_t *r_code)

  Write to BMC PCIE config.
- oob_status_t set_xgmi_pstate_range (uint8_t soc_die_num, uint8_t min_xgmi_pstate, uint8_t max_xgmi_pstate)

Set the max and min xGMI pstate.

- oob_status_t set_cpu_rail_iso_freq_policy (uint8_t soc_die_num, uint8_t policy)

  Set the cpu rail iso frequency policy.
- oob_status_t get_cpu_rail_iso_freq_policy (uint8_t soc_die_num, uint8_t *policy)

  Get the cpu rail iso frequency policy.
- oob_status_t set_dfc_enable (uint8_t soc_die_num, uint8_t state)

  Set the DF C-state enabling control.
- oob_status_t get_dfc_enable (uint8_t soc_die_num, uint8_t *state)

  Get the DF C-state enabling control.
- oob_status_t get_ch_dram_thr_with_status (uint8_t soc_die_num, uint8_t dimm_addr, struct dram_thr_with_status *thr_info)

  Get the channel DRAM throttle.
- oob_status_t get_avg_dram_thr_with_status (uint8_t soc_die_num, struct dram_thr_with_status *thr_info)

  Get the Avg DRAM throttle and throttle status across all the channels.
- oob_status_t get_avg_dram_throttle (uint8_t soc_die_num, uint32_t *dram_throttle)

  Get the Avg DRAM throttle across all the channels.
- oob_status_t get_ch_dram_throttle (uint8_t soc_die_num, uint8_t dimm_addr, uint32_t *dram_throttle)

  Get the channel DRAM throttle.
- oob_status_t get_pc6_enable (uint8_t soc_die_num, uint8_t *pc6_state)

  Get the PC6 enabling control.
- oob_status_t set_pc6_enable (uint8_t soc_die_num, uint8_t pc6_state, uint8_t *updated_pc6_state)

  Set the PC6 enabling control.
- oob_status_t get_cc6_enable (uint8_t soc_die_num, uint8_t *cc6_state)

  Get the CC6 enabling control.
- oob_status_t set_cc6_enable (uint8_t soc_die_num, uint8_t cc6_state, uint8_t *updated_cc6_state)

  Set the CC6 enabling control.
- oob_status_t get_xgmi_link_width_range (uint8_t soc_die_num, uint8_t *min_link_width, uint8_t *max_link_width)

  Get the max and min width of xGMI link.
- oob_status_t get_apb_state (uint8_t soc_die_num, uint8_t *apb_state, uint8_t *df_pstate)

  Get the apb state.
- oob_status_t get_df_pstate_range (uint8_t soc_die_num, uint8_t *max_pstate, uint8_t *min_pstate)

  Get df pstate range.
- oob_status_t set_pcie_link_training (uint8_t soc_die_num, struct pcie_link_training d_in, uint32_t *auth_status)

  Set PCIe Link Training.
- oob_status_t get_xgmi_pstate_range (uint8_t soc_die_num, uint8_t *min_xgmi_pstate, uint8_t *max_xgmi_pstate)

  Get the max and min xGMI pstate.
- oob_status_t get_ccd_power_consumption (uint8_t soc_die_num, uint32_t logical_core_id, uint32_t *ccd_pow)

  Get CCD power consumption.
- oob_status_t get_tdelta (uint8_t soc_die_num, uint32_t *thermal_behav)

  Get Tdelta.
- oob_status_t get_svi3_vr_controller_temp_by_rail (uint8_t soc_die_num, struct svi3_vr_cont_data_in d_in, struct svi3_vr_cont_data_out *d_out)

  Get SVI3 VR controller temperature by RAIL.
- oob_status_t get_supporting_error_types (uint8_t soc_die_num, struct oob_err_inj_types *d_out)

  Get supporting error types.
- oob_status_t get_bmc_ras_mca_run_time_error_info (uint8_t soc_die_num, struct bmc_ras_runtime_din din, struct ras_runtime_response *resp)

  Retrieves BMC RAS MCA runtime error information for a specified SoC die.

- oob_status_t get_bmc_ras_dram_ecc_run_time_error_info (uint8_t soc_die_num, struct bmc_ras_runtime_din din, struct ras_runtime_response *resp)

  Retrieves BMC RAS DRAM ECC runtime error information for a specified SoC die.
- oob_status_t get_bmc_ras_pcie_run_time_error_info (uint8_t soc_die_num, struct bmc_ras_runtime_din din, struct ras_runtime_response *resp)

  Retrieves BMC RAS PCIe runtime error information for a specified SoC die.
- oob_status_t get_floor_core_limit (uint8_t soc_die_num, uint16_t core_id, uint16_t *floor_core_limit)

  Retrieves the APML floor core limit for a specified core.
- oob_status_t set_core_floor_limit (uint8_t soc_die_num, struct core_floor_limit_din d_in)

  Sets the APML floor core limit for a specified core.
- oob_status_t set_floor_limit_for_all_cores (uint8_t soc_die_num, uint16_t floor_limit)

  Sets the APML floor limit for all cores in a specified SoC die.
- oob_status_t set_power_efficiency_mode_selection (uint8_t soc_die_num, struct pow_eff_mode d_in, struct pow_eff_mode *d_out)

  Sets the Power Efficiency Mode selection for a specified SoC die.
- oob_status_t get_power_efficiency_mode_selection (uint8_t soc_die_num, struct pow_eff_mode *d_out)

  Gets the Power Efficiency Mode selection for a specified SoC die.
- oob_status_t get_effective_floor_freq_per_core (uint8_t soc_die_num, uint16_t core_id, uint16_t *eff_floor↩ _freq)

  Retrieves the effective floor frequency for a specified core.
- oob_status_t soc_dimm_power_limit (uint8_t soc_die_num, uint8_t is_get, uint32_t *limit)

  Set or Get SOC+DIMM combined power limit (mWatts).
- oob_status_t get_enabled_hsmp_commands (uint8_t soc_die_num, struct get_hsmp_cmds_din d_in, struct get_hsmp_cmds_dout *d_out)

  Get enabled HSMP commands.
- oob_status_t set_enabled_hsmp_commands (uint8_t soc_die_num, struct set_hsmp_cmds_din d_in)

  Set enabled HSMP commands.
- oob_status_t get_dimm_pow_data (uint8_t soc_die_num, struct dimm_pow_din d_in, struct dimm_pow_dout *d_out)

  Read DIMM power consumption.
- oob_status_t get_dimm_thermal_sensor_data (uint8_t soc_die_num, struct dimm_thermal_din din, struct dimm_thermal *dimm_temp)

  Read DIMM thermal sensor.
- oob_status_t get_hottest_dimm_temp_range_ref_rate (uint8_t soc_die_num, struct hottest_dimm_temp_refresh_rate *rate)

  Read hottest DIMM address, temperature range and refresh rate.
- oob_status_t write_pcie_link_control (uint8_t soc_die_num, uint32_t pcie_link_control)

  Write PCIe Link Control.

Variables

- float esu_multiplier

  energy status multiplier value is 1/2^ ESU where ESU is [12:8] bit of the mailbox command 0x55h.

7.5.1 Detailed Description

Header file for the Mailbox messages supported by APML library. All required function, structure, enum, etc. definitions should be defined in this file.
This header file contains the following: APIs prototype of the Mailbox messages exported by the APML library. Description of the API, arguments and return values. The Error codes returned by the API.

7.5.2 Enumeration Type Documentation

7.5.2.1 esb_mailbox_commmands

```
enum esb_mailbox_commmands
```

Mailbox message types defined in the APML library.

Enumerator

GET_DIMM_SPD	its extended to PMIC, TS0 and TS1 Initially it was supported for SPD only and later
SET_DIMM_SPD	its extended to PMIC, TS0 and TS1 Initially it was supported for SPD only and later
WRITE_PCIE_LINK_CONTROL	Enable/Disable PCIe port Link Control.

7.5.3 Function Documentation

7.5.3.1 write_bmc_report_dimm_power()

```
oob_status_t write_bmc_report_dimm_power (
            uint8_t soc_die_num,
            struct dimm_power dp_info )
```

Set DIMM Power consumption in mwatts.

This function will set DIMM Power consumption periodically by BMC at specified update rate (10 ms or less) when bmc owns the SPD side-band bus. Supported platforms: Fam-19h_Mod-10h-1Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*dp_info*	dimm_power Struct with power(mw), updaterate(ms) & dimm address

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.2 write_bmc_report_dimm_thermal_sensor()

```
oob_status_t write_bmc_report_dimm_thermal_sensor (
            uint8_t soc_die_num,
            struct dimm_thermal dt_info )
```

Set DIMM thermal Sensor in degree Celcius.

This function will set DIMM thermal sensor (in degree celcius) periodically by BMC at specified update rate (10 ms or less) when bmc owns the SPD side-band bus. Supported platforms: Fam-19h_Mod-10h-1Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*dt_info*	struct with temp(ºC), updaterate(ms) & dimm address

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.3 read_bmc_ras_pcie_config_access()

```
oob_status_t read_bmc_ras_pcie_config_access (
            uint8_t soc_die_num,
            struct pci_address pci_addr,
            uint32_t * out_buf )
```

Read BMC RAS PCIE config access.
This function will read the 32 bit BMC RAS extended PCI config space.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*pci_addr*	pci_address structure with fucntion(3 bit), device(4 bit) bus(8 bit), offset(12 bit), segment(4 bit). SEGMENT:0 BUS 0:DEVICE 18, SEGMENT:0 BUS 0:DEVICE 19, SEGMENT:0 BUS 0:DEVICE 1A and SEGMENT:0 BUS 0:DEVICE 1B are inaccessible. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-50h-57h
out	*out_buf*	32 bit data from offset in PCI config space.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.4 read_bmc_ras_mca_validity_check()

```
oob_status_t read_bmc_ras_mca_validity_check (
            uint8_t soc_die_num,
            uint16_t * bytes_per_mca,
            uint16_t * mca_banks )
```

Read number of MCA banks with valid status after a fatal error.
This function returns the number of MCA banks with valid status after a fatal error. Supported platforms↩ : Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*bytes_per_mca*	returns bytes per mca.
out	*mca_banks*	number of mca banks.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.5 read_bmc_ras_mca_msr_dump()

```
oob_status_t read_bmc_ras_mca_msr_dump (
            uint8_t soc_die_num,
            struct mca_bank mca_dump,
            uint32_t * out_buf )
```

Read data from mca bank reported by bmc ras mca validity check.

This function returns the data from mca bank reported by bmc ras mca validity check. Supported platforms↩ : Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*mca_dump*	mca_bank Struct containing offset, index of MCA bank.
out	*out_buf*	32 bit data from offset in mca bank.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.6 read_bmc_ras_fch_reset_reason()

```
oob_status_t read_bmc_ras_fch_reset_reason (
            uint8_t soc_die_num,
            uint32_t input,
            uint32_t * out_buf )
```

Read FCH reason code from the previous reset.

This function reads the FCH reason code from the previous reset. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-19h_Mod-90h-9Fh and and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*input*	integer for id of FCH register.
out	*out_buf*	Data from FCH register.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.7 read_dimm_temp_range_and_refresh_rate()

```
oob_status_t read_dimm_temp_range_and_refresh_rate (
            uint8_t soc_die_num,
            uint32_t dimm_addr,
            struct temp_refresh_rate * rate )
```

Read DIMM temperature range and refresh rate.

This function returns the per DIMM temperature range and refresh rate from the MR4 register, per JEDEC spec. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*dimm_addr*	Encoded address of the dimm.
out	*rate*	temp_refresh_rate structure with refresh rate(1 bit) and range(3 bit). refresh rate: 0 = 1X, 1 = 2X. Temperature range: 001b = 1X, 101b = 2X.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.8 read_dimm_power_consumption()

```
oob_status_t read_dimm_power_consumption (
            uint8_t soc_die_num,
            uint32_t dimm_addr,
            struct dimm_power * dimm_pow )
```

Read DIMM power consumption.

This function returns the DIMM power consumption when bmc does not own the SPD side band bus. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*dimm_addr*	Encoded address of the dimm.
out	*dimm_pow*	struct dimm_power contains updaterate(ms): Time since last update (0-511ms). 0 means last update was < 1ms, and 511 means update was >= 511ms power consumption(mw): power consumed (0 - 32767 mW)

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.9 read_dimm_thermal_sensor()

```
oob_status_t read_dimm_thermal_sensor (
            uint8_t soc_die_num,
            uint32_t dimm_addr,
            struct dimm_thermal * dimm_temp )
```

Read DIMM thermal sensor.

This function returns the DIMM thermal sensor (2 sensors per DIMM) when bmc does not own the SPD side band bus. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.

Parameters

in	*dimm_addr*	Encoded address of the dimm.
out	*dimm_temp*	struct dimm_thermal struct contains updaterate(ms): Time since last update (0-511ms). 0 means last update was < 1ms, and 511 means update was >= 511ms temperature (Degrees C): Temperature (-256 - 255.75 degree C)

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.10 read_pwr_current_active_freq_limit_socket()

```
oob_status_t read_pwr_current_active_freq_limit_socket (
            uint8_t soc_die_num,
            uint16_t * freq,
            char ** source_type )
```

Read current active frequency limit per socket.

This function returns the current active frequency limit per socket. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*freq*	Frequency (MHz).
out	*source_type*	Source of limit.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.11 read_pwr_current_active_freq_limit_core()

```
oob_status_t read_pwr_current_active_freq_limit_core (
            uint8_t soc_die_num,
            uint32_t core_id,
            uint16_t * base_freq )
```

Read current active frequency limit set per core.

This function returns the current active frequency limit per core. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh and and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*core_id*	index.
out	*base_freq*	Frequency (MHz).

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.12 read_pwr_svi_telemetry_all_rails()

```
oob_status_t read_pwr_svi_telemetry_all_rails (
            uint8_t soc_die_num,
            uint32_t * power )
```

Read SVR based telemtry for all rails.

This function returns the SVR based telemetry (power and update rate) for all rails. Supported platforms↩ : Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh and and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*power*	SVI-based Telemetry for all rails(mW)

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.13 read_socket_freq_range()

```
oob_status_t read_socket_freq_range (
            uint8_t soc_die_num,
            uint16_t * fmax,
            uint16_t * fmin )
```

Read socket frequency range.

This function returns the fmax and fmin frequency per socket. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*fmax*	maximum frequency (MHz).
out	*fmin*	minimum frequency (MHz).

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.14 read_current_io_bandwidth()

```
oob_status_t read_current_io_bandwidth (
```

```
            uint8_t soc_die_num,
            struct link_id_bw_type link,
            uint32_t * io_bw )
```

Read current bandwidth on IO Link.

This function returns the current IO bandwidth. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh and and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*link*	link_id_bw_type struct containing bandwidth type and Link ID encoding bandwidth type: 001b Aggregate BW Other Reserved MI300A APML Link ID Encoding: 00000011b: P2 00000100b: P3 00001000b: G0 00001001b: G1 00001010b: G2 00001011b: G3 00001100b: G4 00001101b: G5 00001110b: G6 00001111b: G7 For other platforms the APML Link ID Encoding: 00000001b: P0 00000010b: P1 00000100b: P2 00001000b: P3 00010000b: G0 00100000b: G1 01000000b: G2 10000000b: G3
out	*io_bw*	io bandwidth (Mbps).

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.15 read_current_xgmi_bandwidth()

```
oob_status_t read_current_xgmi_bandwidth (
            uint8_t soc_die_num,
            struct link_id_bw_type link,
            uint32_t * xgmi_bw )
```

Read current bandwidth on xGMI Link.

This function returns the current xGMI bandwidth. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*link*	link_id_bw_type struct containing link id and bandwidth type info. Valid BW type are 001b Aggregate BW 010b Read BW 100b Write BW Other Reserved MI300A APML Link ID Encoding: 00000011b: P2 00000100b: P3 00001000b: G0 00001001b: G1 00001010b: G2 00001011b: G3 00001100b: G4 00001101b: G5 00001110b: G6 00001111b: G7 For other platforms the APML Link ID Encoding: 00000001b: P0 00000010b: P1 00000100b: P2 00001000b: P3 00010000b: G0 00100000b: G1 01000000b: G2 10000000b: G3
out	*xgmi_bw*	io bandwidth (Mbps).

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.16 write_gmi3_link_width_range()

```
oob_status_t write_gmi3_link_width_range (
            uint8_t soc_die_num,
            uint8_t min_link_width,
            uint8_t max_link_width )
```

Set the max and min width of GMI3 link.

This function will set the max and min width of GMI3 Link. Supported platforms: Fam-19h_Mod-10h-1Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*min_link_width*	minimum link width. 0 = Quarter width 1 = Half width 2 = full width
in	*max_link_width*	maximum link width. 0 = Quarter width 1 = Half width 2 = full width NOTE: max value must be greater than or equal to min value.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.17 write_xgmi_link_width_range()

```
oob_status_t write_xgmi_link_width_range (
            uint8_t soc_die_num,
            uint8_t min_link_width,
            uint8_t max_link_width )
```

Set the max and min width of xGMI link.

This function will set the max and min width of xGMI Link. If this API is called from both the master and the slave sockets, then the largest width values from either calls are used. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*min_link_width*	minimum link width. 0 = X4 1 = X8 2 = X16
in	*max_link_width*	maximum link width. 0 = X4 1 = X8 2 = X16 NOTE: Max value must be greater than or equal to min value.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.18 write_apb_disable()

```
oob_status_t write_apb_disable (
            uint8_t soc_die_num,
            uint8_t df_pstate,
            bool * prochot_asserted )
```

Set the APBDisabled.
This function will set the APBDisabled by specifying the Data Fabric(DF) P-state. Messages APBEnable and APB-Disable specify DF(Data Fabric) P-state behavior. DF P-states specify the frequency of clock domains from the CPU core boundary through to and including system memory, where 0 is the highest DF P-state and 2 is the lowest. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*df_pstate*	data fabric p-state.
out	*prochot_asserted*	prochot asserted status. True indicates asserted False indicates not-asserted.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.19 write_apb_enable()

```
oob_status_t write_apb_enable (
            uint8_t soc_die_num,
            bool * prochot_asserted )
```

Enable the DF p-state performance boost algorithm.
This function will enable the DF p-state performance boost algorithm. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*prochot_asserted*	prochot asserted status. True indicates asserted and false indicates not-asserted.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.20 read_current_dfpstate_frequency()

```
oob_status_t read_current_dfpstate_frequency (
            uint8_t soc_die_num,
            struct pstate_freq * df_pstate )
```

Read current DF p-state frequency .
This function returns the current DF p-state frequency. Returns the Fclck, DRAM memory clock(memclk),umc clock divider for the current socket DF P-state. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh and and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*df_pstate*	struct pstate_freq contains DRAM memory clock(mem clk) data fabric clock (Fclk) UMC clock divider Uclk = 0 means divide by 1 else divide by 2.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.21 write_lclk_dpm_level_range()

```
oob_status_t write_lclk_dpm_level_range (
            uint8_t soc_die_num,
            struct lclk_dpm_level_range lclk )
```

Set the max and min LCK DPM Level on a given NBIO per socket.

This function will set the LCK DPM Level on a given NBIO per socket.The DPM Level is an encoding to represent the PCIE Link Frequency (LCLK) under a root complex (NBIO). Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*lclk*	lclk_dpm_level_range struct containing NBIOID (8 bit) Min dpm level (8 bit) and Max dpm level(8 bit). Valid NBIOID, min dpm level and max dpm level values are between 0 $\sim$ 3.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.22 read_bmc_rapl_units()

```
oob_status_t read_bmc_rapl_units (
            uint8_t soc_die_num,
            uint8_t * tu_value,
            uint8_t * esu_value )
```

Read RAPL (Running Average Power Limit) Units.

This function returns the RAPL (Running Average Power Limit) Units. Energy information (in Joules) is based on the multiplier: 1/(2^ESU). Time information (in Seconds) is based on the multiplier: 1/(2^TU). Supported platforms : Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*tu_value*	TU value.
out	*esu_value*	esu value.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.23 read_bmc_cpu_base_frequency()

```
oob_status_t read_bmc_cpu_base_frequency (
            uint8_t soc_die_num,
            uint16_t * base_freq )
```

Read RAPL base frequency per CPU socket.

This function returns the base frequency per CPU socket. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*base_freq*	base frequency.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.24 read_bmc_control_pcie_gen5_rate()

```
oob_status_t read_bmc_control_pcie_gen5_rate (
            uint8_t soc_die_num,
            uint8_t rate,
            uint8_t * mode )
```

Control PCIe Rate on Gen5-Capable devices..

This function returns the PCIe rate. Supported platforms: Fam-19h_Mod-10h-1Fh and Fam-1Ah_Mod-00h-0Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*rate*	PCIe gen rate. 0 indicates Auto-Detect BW and set link rate accordingly. 1 is for Limit at Gen4 Rate. 2 is for Limit at Gen5 rate.
out	*mode*	previous mode. 0 for Auto-Detect, 1 for Limit at Gen4 rate, 2 for limit at Gen5 rate.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.25 read_rapl_core_energy_counters()

```
oob_status_t read_rapl_core_energy_counters (
            uint8_t soc_die_num,
            uint32_t core_id,
            double * energy_counters )
```

Read RAPL core energy counters.

This function returns the RAPL core energy counters. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	Soskcet index.
in	*core_id*	core id.
out	*energy_counters*	core energy.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.26 read_rapl_pckg_energy_counters()

```
oob_status_t read_rapl_pckg_energy_counters (
            uint8_t soc_die_num,
            double * energy_counters )
```

Read RAPL package energy counters.

This function returns the RAPL package energy counters. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*energy_counters*	core energy.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.27 write_pwr_efficiency_mode()

```
oob_status_t write_pwr_efficiency_mode (
            uint8_t soc_die_num,
            uint8_t mode )
```

Write power efficiency profile policy.

This function writes power efficiency mode Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*mode*	power efficiency mode. 0 indicates High Performance Mode 1 indicates Power Efficiency Mode. 2 indicates I/O Performance Mode. 3 Balanced Memory Performance Mode. 4 Balanced Core Performance Mode. 5 Balanced Core and Memory Performance Mode.

Modes supported as per platform, 0 to 2: Fam-19h_Mod-10h-1Fh Fam-19h_Mod-90h-9Fh 0 to 5: Fam-1Ah_Mod-00h-0Fh

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.28 write_df_pstate_range()

```
oob_status_t write_df_pstate_range (
            uint8_t soc_die_num,
            uint8_t max_pstate,
            uint8_t min_pstate )
```

Write df pstate range.

This function writes df pstate range Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh, and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*max_pstate*	value.Max value must be less than or equal to min value. Valid values are 0 - 2.
in	*min_pstate*	value. Valid values are from 0 - 2.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.29 read_lclk_dpm_level_range()

```
oob_status_t read_lclk_dpm_level_range (
            uint8_t soc_die_num,
            uint8_t nbio_id,
            struct dpm_level * dpm )
```

Read LCLK Max and Min DPM level range.

This function returns the LCLK Max and Min DPM level range. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*nbio_id*	nbio for a socket.
out	*dpm*	struct dpm level containing max and min dpm levels. Valid max and min dpm levels are from 0 - 1.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.30 read_ucode_revision()

```
oob_status_t read_ucode_revision (
            uint8_t soc_die_num,
            uint32_t * ucode_rev )
```

Read ucode revision.

This function reads the micro code revision number. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-19h_Mod-90h-9Fh, and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*ucode_rev*	micro code revision.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.31 read_ras_df_err_validity_check()

```
oob_status_t read_ras_df_err_validity_check (
            uint8_t soc_die_num,
            uint8_t df_block_id,
            struct ras_df_err_chk * err_chk )
```

Read number of instances of DF blocks of type DF_BLOCK_ID with errors.

This function reads the number instance of DF blocks of type DF_BLOCK_ID that have an error log to report. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	Socket index.
in	*df_block_id*	DF block ID.
out	*err_chk*	ras_df_err_chk Struct containing number of DF block instances andclength of error log in bytes per instanace.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.32 read_ras_df_err_dump()

```
oob_status_t read_ras_df_err_dump (
            uint8_t soc_die_num,
            union ras_df_err_dump ras_err,
            uint32_t * data )
```

Read RAS DF error dump.

This function reads 32 bits of data from the offset provided for DF block instance retported by read_ras_df_err↩_validity_check. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh

and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	Socket index.
in	*ras_err*	ras_df_err_dump Union containing 4 byte offset, DF block ID and block ID instance.
out	*data*	output data from offset of DF block instance.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.33 reset_on_sync_flood()

```
oob_status_t reset_on_sync_flood (
            uint8_t soc_die_num,
            uint32_t * ack_resp )
```

Read BMC RAS reset on sync flood.

This function requests reset after sync flood. Reset only works during sync flood condition Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index. Only Socket0 is valid for this request. In multi-socket systems, the command must be executed on Socket0; requests to other sockets will be ignored. In 2x1P systems, command should be executed at individual node.
out	*ack_resp*	acknowledgement response.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.34 override_delay_reset_on_sync_flood()

```
oob_status_t override_delay_reset_on_sync_flood (
            uint8_t soc_die_num,
            struct ras_override_delay data_in,
            bool * ack_resp )
```

Overrides delay reset cpu on sync flood value.

This function will override delay reset cpu on sync flood value for the current boot instance. Delay value reverts to BIOS config selection after reboot.Number of override requests is limited to 5 per boot instance. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index. Only Socket0 is valid for this request. In multi-socket systems, the command must be executed on Socket0; requests to other sockets will be ignored. In 2x1P systems, command should be executed at individual node.

Parameters

in	*data_in*	struct ras_override_delay_d_in containing delay value override [5 - 120 mins], disable delay counter bit and stop delay counter bit.
out	*ack_resp*	acknowledgment response.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.35 get_post_code()

```
oob_status_t get_post_code (
            uint8_t soc_die_num,
            uint32_t offset,
            uint32_t * post_code )
```

Read post code.

This function will read most recent post code at the specified offset.SMU caches 8 most recent post codes.↩ When the input is 0 the SMU will refresh the cache before running the latest post code. Input as 0 refers to most recent post code and higher inputs refers to the older post code. Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*offset*	post code offset
out	*post_code*	recent post code of error category and number of instances of error category.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.36 get_bmc_ras_run_time_err_validity_ck()

```
oob_status_t get_bmc_ras_run_time_err_validity_ck (
            uint8_t soc_die_num,
            struct ras_rt_err_req_type err_category,
            struct ras_rt_valid_err_inst * inst )
```

Reads number of valid error instances per category.

This function will read number of valid error instances per category. Valid categories are MCA[00], DRAM CECC[01], PCIE [10], RSVD[11]. Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*err_category*	Runtime error category MCA[00], DRAM CECC[01], PCIe[10] and RSVD [11].
out	*inst*	struct ras_rt_valid_err_inst containing number of bytes of error category and number of instances of error category.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.37 get_bmc_ras_run_time_error_info()

```
oob_status_t get_bmc_ras_run_time_error_info (
            uint8_t soc_die_num,
            struct run_time_err_d_in d_in,
            uint32_t * err_info )
```

Reads BMC RAS runtime error information.

This function will read BMC RAS runtime error information based on category. If category is MCA then it will read 32 bit of data from MCA MSR Bank. If category is DRAM CECC then will read error count, counter info and corresponding from the valid DRAM ECC correctable error counter instance. If category is PCIE then it returns 32 bit PCIE error data from given offset of given instance. Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*d_in*	struct run_time_err_d_in contatining 4 byte aligned offset, runtime error category and 0 based index of valid instance returned by BMC_RAS_RUNTIME_ERR_VALIDITY_CHECK.
out	*err_info*	error information for a given category with valid instance and offset.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.38 set_bmc_ras_err_threshold()

```
oob_status_t set_bmc_ras_err_threshold (
            uint8_t soc_die_num,
            struct run_time_threshold th )
```

Sets BMC RAS error threshold.

This function will configure thresholding counters for MCA, DRAM CECC or PCIE. Supported platforms↩ : Fam-1Ah_Mod-00h-0Fh, Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*th*	struct run_time_threshold containing error type, error count threshold and max interrupt rate. error type : • b00: MCA • b01: DRAM CECC • b10: PCIE • b11: UMC MCA (applicable only for Fam-1Ah_Mod-50h-57h

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.39 set_bmc_ras_oob_config()

```
oob_status_t set_bmc_ras_oob_config (
            uint8_t soc_die_num,
            struct oob_config_d_in d_in )
```

Configures OOB state infrastructure in SoC.

This function will configure OOB state infrastructure in the SoC. Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*d_in*	struct oob_config_d_in containing [0] mca_oob_misc0_ec_enable, [2:1] dram_cecc_oob_ec_mode, [7:3] dram_cecc_leak_rate, [8] pcie_err_reporting_en, [31] mca_err_reporting_en.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.40 get_bmc_ras_oob_config()

```
oob_status_t get_bmc_ras_oob_config (
            uint8_t soc_die_num,
            uint32_t * oob_config )
```

Reads the current status of OOB state infrastructure in SoC.

This function will read the current status of OOB state configuration in the SoC. Supported platforms↩ : Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*oob_config*	containing [0] mca_oob_misc0_ec_enable, [2:1] dram_cecc_oob_ec_mode, [7:3] dram_cecc_leak_rate, [8] pcie_err_reporting_en, [11] MCA Thresholding Interrupt Enable value [12] DRAM CECC Thresholding Interrupt Enable value [13] PCIe error Thresholding Interrupt Enable value [18:15] MCA MaxIntRate value [22:19] DRAM CECC MaxIntRate1 value [26:23] PCIe error MaxIntRate1 value [31] mca_err_reporting_en

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.41 read_ppin_fuse()

```
oob_status_t read_ppin_fuse (
            uint8_t soc_die_num,
            uint64_t * data )
```

Get the 64 bit PPIN fuse.

This function will read the 64 bit PPIN fuse available via OPN_PPIN fuse. Supported platforms : Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*data*	PPIN fuse data

7.5.3.42 read_rtc()

```
oob_status_t read_rtc (
            uint8_t soc_die_num,
            uint64_t * rtc )
```

Read RTC.

Read RTC timer value. RTC time represents the year, month, day, hour, minute and seconds value in a 64b encoding. Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*rtc*	[63:0] = RTC value, [31:0]=> DataIn==0 & [63:32]=> DataIn==4 If DataIn==0 RTC=DD_hh_mm_ss If DataIn==4 RTC=00_YYYY_MM All digits in BCD format.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.43 read_dimm_spd_register()

```
oob_status_t read_dimm_spd_register (
            uint8_t soc_die_num,
            struct dimm_spd_d_in spd_d_in,
            uint32_t * spd_data )
```

Get SPD SB data.

Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*spd_d_in*	struct dimm_spd_d_in containing DIMM address, LID, register offset, register space.
out	*spd_data*	containing spd data as per input

7.5.3.44 get_dimm_sb_register()

```
oob_status_t get_dimm_sb_register (
            uint8_t soc_die_num,
            struct dimm_sb_reg_d_in d_in,
            uint32_t * reg_data )
```

Get dimm SB register data.

This function will read dimm sb register data. For Fam-1Ah_Mod-50h-57h, valid LID codes are 1001(PMIC0), 1010 (SPD Hub), 0010 (TS0), 0110 (TS1)

Supported platforms: Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*d_in*	struct dimm_sb_reg_d_in containing DIMM address, LID, register offset, register space.
out	*reg_data*	containing spd data as per input

7.5.3.45 write_dimm_spd_register()

```
oob_status_t write_dimm_spd_register (
            uint8_t soc_die_num,
            struct dimm_spd_write d_in )
```

Set SPD SB data.

Supported platforms: Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*d_in*	struct dimm_spd_write containing DIMM address, LID, register offset, register space and write data.

7.5.3.46 set_dimm_sb_register_data()

```
oob_status_t set_dimm_sb_register_data (
            uint8_t soc_die_num,
            struct dimm_sb_reg_write d_in )
```

Set dimm SB data.

This function will write dimm sb register data. For Fam-1Ah_Mod-50h-57h, valid LID codes are 1001(PMIC0), 1010 (SPD Hub), 0010 (TS0), 0110 (TS1) Supported platforms: Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*d_in*	struct dimm_sb_reg_write containing DIMM address, LID, register offset, register space and write data.

7.5.3.47 get_dimm_serial_num()

```
oob_status_t get_dimm_serial_num (
            uint8_t soc_die_num,
            uint8_t dimm_addr,
            uint32_t * serial_num )
```

Get DIMM serial number.
Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	Socket index.
in	*dimm_addr*	DIMM address
out	*serial_num*	containing DIMM serial number

7.5.3.48 read_smu_fw_ver()

```
oob_status_t read_smu_fw_ver (
            uint8_t soc_die_num,
            uint32_t * smu_fw_ver )
```

Get SMU FW version.
Supported platforms: Fam-1Ah_Mod-00h-0Fh

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*smu_fw_ver*	SMU firmware version

7.5.3.49 write_bmc_pcie_config()

```
oob_status_t write_bmc_pcie_config (
            uint8_t soc_die_num,
            struct pci_address pci_addr,
            uint32_t pcie_data,
            uint32_t * r_code )
```

Write to BMC PCIE config.
This function will write 32 bit data_in to extended PCI config space of target specified in previous READ↩ _BMC_RAS_PCIE_CONFIG_ACCESS (cmd ID 0x42h). Supported platforms: Fam-19h_Mod-10h-1Fh, Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*pci_addr*	pci_address structure with fucntion(3 bit), device(4 bit) bus(8 bit), offset(12 bit), segment(4 bit). SEGMENT:0 BUS 0:DEVICE 18 and SEGMENT:0 BUS 0:DEVICE 19 are inaccessable.
in	*pcie_data*	data_in to extended PCI config space of target.
out	*r_code*	return code.Response is 0 if operation is successful.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.50 set_xgmi_pstate_range()

```
oob_status_t set_xgmi_pstate_range (
            uint8_t soc_die_num,
            uint8_t min_xgmi_pstate,
            uint8_t max_xgmi_pstate )
```

Set the max and min xGMI pstate.

This function will set the max and min xGMI pstate. pstate values are 0 - DFP0 (high performace) and 1 - DFP1 (low performance). Max value must be less than or equal to min value. XGMI p-state can be set through either HSMP or APML and the value that was last set will be enforced. Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*min_xgmi_pstate*	minimum xgmi pstate.Valid values are 0,1
in	*max_xgmi_pstate*	maximum xgmi pstate.Valid values are 0,1

NOTE: Max value must be less than or equal to min value.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.51 set_cpu_rail_iso_freq_policy()

```
oob_status_t set_cpu_rail_iso_freq_policy (
            uint8_t soc_die_num,
            uint8_t policy )
```

Set the cpu rail iso frequency policy.

This function will set the cpu rail iso freq policy. This policy allows or disables independent core clocks per rail (VDDCR_CPU0 or VDDCR_CPU1). If a socket wide limit (e.g. PPT) is setting the core clock frequency, then this setting has no effect. Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*policy*	frequency policy. 0 means both rails have same freqyency limit. 1 means each rail has independent frequency limit.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.52 get_cpu_rail_iso_freq_policy()

```
oob_status_t get_cpu_rail_iso_freq_policy (
            uint8_t soc_die_num,
            uint8_t * policy )
```

Get the cpu rail iso frequency policy.

This function will get the cpu rail iso freq policy. The response 0 means both rails have same frequency limit and 1 means each rail has independent frequency limit. Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*policy*	current frequency policy. 0 means both rails have same freqyency limit. 1 means each rail has independent frequency limit.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.53 set_dfc_enable()

```
oob_status_t set_dfc_enable (
            uint8_t soc_die_num,
            uint8_t state )
```

Set the DF C-state enabling control.
This function will set the DF C-state enabling control. DF C-state is a low power state for IOD. Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*state*	DF C-state. 0 to disable DFC 1 to enable DFC

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.54 get_dfc_enable()

```
oob_status_t get_dfc_enable (
            uint8_t soc_die_num,
            uint8_t * state )
```

Get the DF C-state enabling control.
This function will get the DF C-state enabling control. DF C-state is a low power state for IOD. Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*state*	current DF C-state

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.55 get_ch_dram_thr_with_status()

```
oob_status_t get_ch_dram_thr_with_status (
            uint8_t soc_die_num,
            uint8_t dimm_addr,
            struct dram_thr_with_status * thr_info )
```

Get the channel DRAM throttle.

This function will get the DRAM throttle for the given channel Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*dimm_addr*	dimm address for Mode 1.
out	*thr_info*	struct dram_thr_with_status containing thr_pct, tsod_th_thr_stat, tsod_en_stat, odts_th_thr_stat and odts_en_stat

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.56 get_avg_dram_thr_with_status()

```
oob_status_t get_avg_dram_thr_with_status (
            uint8_t soc_die_num,
            struct dram_thr_with_status * thr_info )
```

Get the Avg DRAM throttle and throttle status across all the channels.

This function will get the average DRAM throttle, ODTS enable/disable status, ODTS thermal throttle status, TSOD enable/disable status, TSOD thermal throttle status across all the channels Supported platforms↩ : Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*thr_info*	struct dram_thr_with_status containing average thr_pct, tsod_th_thr_stat, tsod_en_stat, odts_th_thr_stat and odts_en_stat

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.57 get_avg_dram_throttle()

```
oob_status_t get_avg_dram_throttle (
            uint8_t soc_die_num,
            uint32_t * dram_throttle )
```

Get the Avg DRAM throttle across all the channels.
This function will get the average DRAM throttle across all the channels Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*dram_throttle*	average dram throttle

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.58 get_ch_dram_throttle()

```
oob_status_t get_ch_dram_throttle (
            uint8_t soc_die_num,
            uint8_t dimm_addr,
            uint32_t * dram_throttle )
```

Get the channel DRAM throttle.
This function will get the DRAM throttle for the given channel Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*dimm_addr*	dimm address for mode 1 containing umc instance ID, dimm 0/1 and temp sensor 0/1.
out	*dram_throttle*	average dram throttle

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.59 get_pc6_enable()

```
oob_status_t get_pc6_enable (
            uint8_t soc_die_num,
            uint8_t * pc6_state )
```

Get the PC6 enabling control.
This function will get the PC6 enabling control. PC6 is low power state for the CCD's also known as package C6 state. Note: This feature cannot be enabled if not enabled by BIOS during POST. Supported platforms↩ : Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*pc6_state*	

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.60 set_pc6_enable()

```
oob_status_t set_pc6_enable (
            uint8_t soc_die_num,
            uint8_t pc6_state,
            uint8_t * updated_pc6_state )
```

Set the PC6 enabling control.

This function will set the PC6 enabling control. PC6 is low power state for the CCD's also known as package C6 state Note: This feature cannot be enabled if not enabled by BIOS during POST. Supported platforms: Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*pc6_state*	
out	*updated_pc6_state*	updated PC6 state

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.61 get_cc6_enable()

```
oob_status_t get_cc6_enable (
            uint8_t soc_die_num,
            uint8_t * cc6_state )
```

Get the CC6 enabling control.

This function will get the CC6 enabling control. This controls CC6 behavior for all the cores. A change to CC6 enablement using the CC6_ENABLE corresponding APML or HSMP messages does not change the state of the cores. Note: This feature cannot be enabled if not enabled by BIOS during POST. Supported platforms↩ : Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*cc6_state*	

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.62 set_cc6_enable()

```
oob_status_t set_cc6_enable (
            uint8_t soc_die_num,
            uint8_t cc6_state,
            uint8_t * updated_cc6_state )
```

Set the CC6 enabling control.

This function will set the CC6 enabling control. This controls CC6 behavior for all the cores. A change to CC6 enablement using the CC6_ENABLE corresponding APML or HSMP messages does not change the state of the cores. Note: This feature cannot be enabled if not enabled by BIOS during POST. Supported platforms↩ : Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*cc6_state*	
out	*updated_cc6_state*	updated PC6 state

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.63 get_xgmi_link_width_range()

```
oob_status_t get_xgmi_link_width_range (
            uint8_t soc_die_num,
            uint8_t * min_link_width,
            uint8_t * max_link_width )
```

Get the max and min width of xGMI link.

This function will set the max and min width of xGMI Link. Note1: Max value must be >= Min value. Invalid configurations result in a Failed response. Note2: If the API is called from both Primary and Secondary sockets, the largest Width values from either call are used. Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*min_link_width*	minimum link width. 0 = X4 1 = X8 2 = X16
out	*max_link_width*	maximum link width. 0 = X4 1 = X8 2 = X16

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.64 get_apb_state()

```
oob_status_t get_apb_state (
            uint8_t soc_die_num,
            uint8_t * apb_state,
            uint8_t * df_pstate )
```

Get the apb state.

This function will get the apb state and DF P-state locked value. It specifies DF (Infinity Fabric TM) Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*apb_state*	APB state. If apb_state is disabled (i.e. value is 1) then DF pstate value is reserved or NA.
out	*df_pstate*	DF P-State locked value.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.65 get_df_pstate_range()

```
oob_status_t get_df_pstate_range (
            uint8_t soc_die_num,
            uint8_t * max_pstate,
            uint8_t * min_pstate )
```

Get df pstate range.

This function get df pstate range. Max pstate must be <= Min min pstate. Supported platforms↩ : Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*max_pstate*	maximum DF p-state
in	*min_pstate*	minimum DF p-state

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.66 set_pcie_link_training()

```
oob_status_t set_pcie_link_training (
            uint8_t soc_die_num,
            struct pcie_link_training d_in,
            uint32_t * auth_status )
```

Set PCIe Link Training.
This function will halt PCIe link training for for OOB devie authentication. BMC communicates information of PCIe links/ports to be cleared for training to MP1 firmware over APML. Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*d_in*	struct pcie_link_training containing link_id and pcie_port.
out	*auth_status*	return authentication status.Response is 0 if Authentication is successful else not authenticated/ untrusted.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.67 get_xgmi_pstate_range()

```
oob_status_t get_xgmi_pstate_range (
            uint8_t soc_die_num,
            uint8_t * min_xgmi_pstate,
            uint8_t * max_xgmi_pstate )
```

Get the max and min xGMI pstate.
This function will get the max and min xGMI pstate. pstate values are 0 - DFP0 (high performace) and 1 - DFP1 (low performance). Max value must be <= Min value. XGMI P-state can be set through APML or HSMP; the value that was last set is enforced. Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*min_xgmi_pstate*	minimum xgmi pstate.
out	*max_xgmi_pstate*	maximum xgmi pstate.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.68 get_ccd_power_consumption()

```
oob_status_t get_ccd_power_consumption (
            uint8_t soc_die_num,
            uint32_t logical_core_id,
            uint32_t * ccd_pow )
```

Get CCD power consumption.
This function will get average power consumption by the CCD. Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.

Parameters

in	*logical_core↩ _id*	core ID
out	*ccd_pow*	average ccd power in mWatts

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.69 get_tdelta()

```
oob_status_t get_tdelta (
            uint8_t soc_die_num,
            uint32_t * thermal_behav )
```

Get Tdelta.
This function will get temperature delta to indicate behavior.
Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*thermal_behav*	Thermal solution behavior. Response as 0 means thermal solution behavior is normal Non-zero response indicates thermal soultion is out of range.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.70 get_svi3_vr_controller_temp_by_rail()

```
oob_status_t get_svi3_vr_controller_temp_by_rail (
            uint8_t soc_die_num,
            struct svi3_vr_cont_data_in d_in,
            struct svi3_vr_cont_data_out * d_out )
```

Get SVI3 VR controller temperature by RAIL.
This function will get SVI3 VR controller temperature by RAIL. If rail mode is 0 then it will read the temperature of the hottest rail. IF rail mode is 1 it will read the temperature of given SVI3 rail index.
Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*d_in*	struct svi3_vr_cont_data_in containing rail_mode and svi3_rail_index.
out	*d_out*	struct svi3_vr_cont_data_out containing svi3_temp and svi3_rail_index

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.71 get_supporting_error_types()

```
oob_status_t get_supporting_error_types (
            uint8_t soc_die_num,
            struct oob_err_inj_types * d_out )
```

Get supporting error types.
This function will get supporting error types from the OOB error injection and also the vendor specific error types.
Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*d_out*	struct oob_err_inj_types containing generic error types, vendor specific error types, vendor error size and vendor flag. err_types_vendor is only applicable in case vendor_flag is set.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.72 get_bmc_ras_mca_run_time_error_info()

```
oob_status_t get_bmc_ras_mca_run_time_error_info (
            uint8_t soc_die_num,
            struct bmc_ras_runtime_din din,
            struct ras_runtime_response * resp )
```

Retrieves BMC RAS MCA runtime error information for a specified SoC die.
This function queries the BMC for Machine Check Architecture (MCA) runtime error information related to Reliability, Availability, and Serviceability (RAS) for the given SoC die number. The input data is provided via the `din` structure, and the response is populated in the `resp` structure. 0x61 has to be called with category as MCA(0) and mode as 0 and the response will be number of MCA error instances and number of bytes per instance. Number of MCA error instances will be din.number_of_instance and number of bytes per instance / 4 will be assigned to din.number_of↩_offsets. Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	The SoC die number to query.
in	*din*	Structure containing input data for the runtime query.
out	*resp*	Pointer to structure to be filled with the runtime response.

Returns

oob_status_t Status of the operation.

- OOB_STATUS_SUCCESS on success
- Appropriate error code otherwise

7.5.3.73 get_bmc_ras_dram_ecc_run_time_error_info()

```
oob_status_t get_bmc_ras_dram_ecc_run_time_error_info (
            uint8_t soc_die_num,
            struct bmc_ras_runtime_din din,
            struct ras_runtime_response * resp )
```

Retrieves BMC RAS DRAM ECC runtime error information for a specified SoC die.
This function queries the BMC for DRAM ECC runtime error information related to Reliability, Availability, and Serviceability (RAS) for the given SoC die number. The input data is provided via the `din` structure, and the response is populated in the `resp` structure.
0x61 has to be called with category as DRAM ECC (01) and mode as 0. The response will be:

- Number of DRAM ECC error instances (assigned to din.number_of_instance)
- Number of bytes per instance / 4 (assigned to din.number_of_offsets) Supported platforms↩ : Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	The SoC die number to query.
in	*din*	Structure containing input data for the runtime query.
out	*resp*	Pointer to structure to be filled with the runtime response.

Return values

OOB_SUCCESS	Operation successful.
Non-zero	Appropriate error code otherwise.

7.5.3.74 get_bmc_ras_pcie_run_time_error_info()

```
oob_status_t get_bmc_ras_pcie_run_time_error_info (
            uint8_t soc_die_num,
            struct bmc_ras_runtime_din din,
            struct ras_runtime_response * resp )
```

Retrieves BMC RAS PCIe runtime error information for a specified SoC die.
This function queries the BMC for PCIe runtime error information related to Reliability, Availability, and Serviceability (RAS) for the given SoC die number.The input data is provided via the `din` structure, and the response is populated in the `resp` structure.
0x61 has to be called with category as PCIe (02) and mode as 0. The response will be:

- Number of PCIe error instances (assigned to din.number_of_instance)
- Number of bytes per instance divided by 4 (assigned to din.number_of_offsets) Supported platforms↩ : Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	The SoC die number to query.
in	*din*	Structure containing input data for the runtime query.
out	*resp*	Pointer to structure to be filled with the runtime response.

Return values

OOB_SUCCESS	Operation successful.
Non-zero	Appropriate error code otherwise.

7.5.3.75 get_floor_core_limit()

```
oob_status_t get_floor_core_limit (
            uint8_t soc_die_num,
            uint16_t core_id,
            uint16_t * floor_core_limit )
```

Retrieves the APML floor core limit for a specified core.

This function queries the APML (Advanced Platform Management Link) mailbox to obtain the floor core limit value for a given core on a specified SoC die. Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	The SoC die number to query.
in	*core_id*	The core ID for which the floor core limit is requested.
out	*floor_core_limit*	Pointer to a variable where the retrieved floor core limit will be stored.

Return values

OOB_SUCCESS	Operation successful.
Non-zero	Appropriate error code otherwise.

7.5.3.76 set_core_floor_limit()

```
oob_status_t set_core_floor_limit (
            uint8_t soc_die_num,
            struct core_floor_limit_din d_in )
```

Sets the APML floor core limit for a specified core.

This function sets the floor core limit value for a given core on a specified SoC die. Supported platforms↩ : Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	The SoC die number to configure.
in	*d_in*	Structure containing the core ID and desired floor limit.

Return values

OOB_SUCCESS	Operation successful.
Non-zero	Appropriate error code otherwise.

7.5.3.77 set_floor_limit_for_all_cores()

```
oob_status_t set_floor_limit_for_all_cores (
```

```
            uint8_t soc_die_num,
            uint16_t floor_limit )
```

Sets the APML floor limit for all cores in a specified SoC die.

This function sets the floor limit value for all cores on a specified SoC die. Supported platforms↩ : Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	The SoC die number to configure.
in	*floor_limit*	The floor limit value to be set for all cores.

Return values

OOB_SUCCESS	Operation successful.
Non-zero	Appropriate error code otherwise.

7.5.3.78 set_power_efficiency_mode_selection()

```
oob_status_t set_power_efficiency_mode_selection (
            uint8_t soc_die_num,
            struct pow_eff_mode d_in,
            struct pow_eff_mode * d_out )
```

Sets the Power Efficiency Mode selection for a specified SoC die.

This function configures the power efficiency mode, utilization point, and PPT limit for balanced core modes on the specified SoC die. Utilization point and PPT limit are applicable only for the modes greater than 3. Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	The SoC die number to configure.
in	*d_in*	Structure containing the desired power efficiency mode selection.
out	*d_out*	Pointer to a structure where the updated power efficiency mode will be stored.

Return values

OOB_SUCCESS	Operation successful.
Non-zero	Appropriate error code otherwise.

7.5.3.79 get_power_efficiency_mode_selection()

```
oob_status_t get_power_efficiency_mode_selection (
            uint8_t soc_die_num,
            struct pow_eff_mode * d_out )
```

Gets the Power Efficiency Mode selection for a specified SoC die.

This function retrieves the current power efficiency mode, utilization point, and PPT limit for balanced core modes on the specified SoC die.Utilization point and PPT limit are applicable only for the modes greater than 3. Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	The SoC die number to query.

Parameters

out	*d_out*	Pointer to a structure where the current power efficiency mode will be stored.

Return values

OOB_SUCCESS	Operation successful.
Non-zero	Appropriate error code otherwise.

7.5.3.80 get_effective_floor_freq_per_core()

```
oob_status_t get_effective_floor_freq_per_core (
            uint8_t soc_die_num,
            uint16_t core_id,
            uint16_t * eff_floor_freq )
```

Retrieves the effective floor frequency for a specified core.
This function queries the APML mailbox to obtain the effective floor frequency value for a given core on a specified SoC die. Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	The SoC die number to query.
in	*core_id*	The core ID for which the effective floor frequency is requested.
out	*eff_floor_freq*	Pointer to a variable where the retrieved effective floor frequency will be stored.

Return values

OOB_SUCCESS	Operation successful.
Non-zero	Appropriate error code otherwise.

7.5.3.81 soc_dimm_power_limit()

```
oob_status_t soc_dimm_power_limit (
            uint8_t soc_die_num,
            uint8_t is_get,
            uint32_t * limit )
```

Set or Get SOC+DIMM combined power limit (mWatts).
This function sets or gets the combined power limit for the socket and DIMM. If is_get is 0, sets the power limit to the value pointed by limit (in mWatts). If is_get is 1, retrieves the current power limit into the value pointed by limit (in mWatts). A value of 0 disables the SOC+DIMM power limit. Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*is_get*	0 = Set Power Limit, 1 = Get Power Limit
in,out	*limit*	Pointer to power limit value (mWatts). Input for set, output for get.

Return values

OOB_SUCCESS	is returned upon successful call.

Return values

Non-zero	is returned upon failure.

7.5.3.82 get_enabled_hsmp_commands()

```
oob_status_t get_enabled_hsmp_commands (
            uint8_t soc_die_num,
            struct get_hsmp_cmds_din d_in,
            struct get_hsmp_cmds_dout * d_out )
```

Get enabled HSMP commands.

This function retrieves the enabled HSMP commands for the specified offset. HSMP read mask indicator (rmask): 1 = for read commands, 0 = for write commands. Offset indicating the segment of the bitmask (0, 1, 2, 3, etc.).

Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*d_in*	Struct get_hsmp_cmds_din containing the following fields: rmask : read mask indicator (1 = for read commands, 0 = for write commands) offset : Offset for the 24-bit bitmask segment.
out	*d_out*	Struct containing: offset : Offset returned bitmask : 24-bit bitmask segment of enabled commands.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.83 set_enabled_hsmp_commands()

```
oob_status_t set_enabled_hsmp_commands (
            uint8_t soc_die_num,
            struct set_hsmp_cmds_din d_in )
```

Set enabled HSMP commands.

This function sets the HSMP commands using a 24-bit bitmask segment. The bitmask specifies which HSMP commands are enabled for the given offset. The APML bitmask can override the BIOS-provided bitmask for runtime reconfigurability.

Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*d_in*	Struct containing the following fields: rmask: read mask indicator (1 = for read commands, 0 = for write commands) offset: Offset for the 24-bit bitmask segment bitmask: 24-bit bitmask specifying enabled commands.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.84 get_dimm_pow_data()

```
oob_status_t get_dimm_pow_data (
            uint8_t soc_die_num,
            struct dimm_pow_din d_in,
            struct dimm_pow_dout * d_out )
```

Read DIMM power consumption.
This function returns the DIMM power consumption when bmc does not own the SPD side band bus. Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*d_in*	struct dimm_pow_din contains pow_reporting_flag and dimm_addr. pow_reporting_flag: Valid values are 0, 1, 2 • 0: Power consumption of the given DIMM address. • 1: DIMM consumping highest power and corresponding address • 2: Total power consumption of all populated and enabled DIMM's dimm_addr: Valid only if pow_reporting_flag is 0, else reserved.
out	*d_out*	struct dimm_pow_dout contains updaterate(ms): Time since last update (0-511ms). 0 means last update was < 1ms, and 511 means update was >= 511ms power consumption(mw): power consumption is reported at 1 mw/s (range from 0 - 655340 for reporting_flag is 2 else the range is from 0 - 32767)

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.85 get_dimm_thermal_sensor_data()

```
oob_status_t get_dimm_thermal_sensor_data (
            uint8_t soc_die_num,
            struct dimm_thermal_din din,
            struct dimm_thermal * dimm_temp )
```

Read DIMM thermal sensor.
This function returns the DIMM thermal sensor (2 sensors per DIMM) when bmc does not own the SPD side band bus. thermal_flag: Valid values are 0 and 1

- 0: API returns temperature for the provided DIMM
- 1: API returns temperature of hottest DIMM Supported platforms: Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.

Parameters

in	*din*	struct dimm_thermal_din containing dimm address and thermal sensor reporting flag. thermal_flag: Valid values are 0 and 1 • 0: API returns temperature for the provided DIMM • 1: API returns temperature of hottest DIMM dimm_addr: Valid only if the thermal_flag is 0, else reserved
out	*dimm_temp*	struct dimm_thermal struct contains updaterate(ms): Time since last update (0-511ms). 0 means last update was < 1ms, and 511 means update was >= 511ms temperature (Degrees C): Temperature (-256 - 255.75 degree C)

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.86 get_hottest_dimm_temp_range_ref_rate()

```
oob_status_t get_hottest_dimm_temp_range_ref_rate (
            uint8_t soc_die_num,
            struct hottest_dimm_temp_refresh_rate * rate )
```

Read hottest DIMM address, temperature range and refresh rate.

This function returns the hottest DIMM address, temperature range and refresh rate from the MR4 register, per JEDEC spec. Supported platforms: Fam-1Ah_Mod-50h-57h

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*rate*	temp_refresh_rate structure with hottest DIMM address and its corresponding refresh rate (1 bit) and temperature range (3 bits). refresh rate: 0 = 1X, 1 = 2X. Temperature range: 3-bit MR4[2:0] temperature range code (see JEDEC MR4 specification).

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.5.3.87 write_pcie_link_control()

```
oob_status_t write_pcie_link_control (
            uint8_t soc_die_num,
            uint32_t pcie_link_control )
```

Write PCIe Link Control.

This function will Enable or Disable the PCIe port link behavior, allowing dynamic switching between the CPU and port drives for link establishment. Supported platforms: Fam-1Ah_Mod-00h-0Fh and Fam-1Ah_Mod-50h-57h.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.

Parameters

in	*pcie_link_control*	0 for Normal Operation, 1 for Disable Link.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.6 esmi_rmi.h File Reference

```
#include "apml_err.h"
```

Macros

- #define MAX_ALERT_REG_V21_DENSE 48

  Max alert registers for V21 dense platform //.
- #define MAX_ALERT_REG 32

  Max alert register //.
- #define MAX_THREAD_REG_V20 32

  Max thread register for rev 20 //.
- #define MAX_THREAD_REG_V10 16

  Max thread register for rev 10 //.
- #define MAX_THREAD_REG_V21_DENSE 48

  Max thread register for rev 21 dense platform //.

Enumerations

- enum sbrmi_status_code {
  SBRMI_SUCCESS = 0x0 , **SBRMI_CMD_TIMEOUT** = 0x11 , **SBRMI_WARM_RESET** = 0x22 , **SBRMI_UNKNOWN_CMD_FORMAT** = 0x40 ,
  SBRMI_INVALID_READ_LENGTH = 0x41 , **SBRMI_EXCESSIVE_DATA_LENGTH** = 0x42 , **SBRMI_INVALID_THREAD** = 0x44 , **SBRMI_UNSUPPORTED_CMD** = 0x45 ,
  SBRMI_CMD_ABORTED = 0x81 }

  Error codes retured by APML mailbox functions.
- enum sbrmi_registers {
  SBRMI_REVISION = 0x0 , **SBRMI_CONTROL** , **SBRMI_STATUS** , **SBRMI_READSIZE** ,
  SBRMI_THREADENABLESTATUS0 , **SBRMI_ALERTSTATUS0** = 0x10 , **SBRMI_ALERTSTATUS15** = 0x1F , **SBRMI_ALERTMASK0** = 0x20 ,
  SBRMI_ALERTMASK15 = 0x2F , **SBRMI_SOFTWAREINTERRUPT** = 0x40 , **SBRMI_THREADNUMBER** , **SBRMI_THREAD128CS** = 0x4B ,
  SBRMI_RASSTATUS , **SBRMI_CONTROL2** , **SBRMI_THREADNUMBERLOW** = 0x4E , **SBRMI_THREADNUMBERHIGH** = 0x4F ,
  SBRMI_ALERTSTATUS16 = 0x50 , **SBRMI_ALERTSTATUS31** = 0x5F , **SBRMI_ALERTSTATUS32** = 0x220 , **SBRMI_ALERTSTATUS47** = 0x22F ,
  SBRMI_MP0OUTBNDMSG0 = 0x80 , **SBRMI_MP0OUTBNDMSG7** = 0x87 , **SBRMI_ALERTMASK16** = 0xC0 , **SBRMI_ALERTMASK31** = 0xCF ,
  SBRMI_ALERTMASK32 = 0x1C0 , **SBRMI_ALERTMASK47** = 0x1CF }

  SB-RMI(Side-Band Remote Management Interface) features register access.

Functions

- oob_status_t read_sbrmi_revision (uint8_t soc_die_num, uint8_t *buffer)

  Read revision from SB_RMI register command. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_control (uint8_t soc_die_num, uint8_t *buffer)

  Read Control byte from SB_RMI register command. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_status (uint8_t soc_die_num, uint8_t *buffer)

  Read one byte of Status value from SB_RMI register command. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_readsize (uint8_t soc_die_num, uint8_t *buffer)

  This register specifies the number of bytes to return when using the block read protocol to read SBRMI_x[4F:10]. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_threadenablestatus (uint8_t soc_die_num, uint8_t *buffer)

  Read one byte of Thread Status from SB_RMI register command. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_multithreadenablestatus (uint8_t soc_die_num, uint8_t *buffer)

  Read one byte of Thread Status from SB_RMI register command. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_swinterrupt (uint8_t soc_die_num, uint8_t *buffer)

  This register is used by the SMBus master to generate an interrupt to the processor to indicate that a message is available.. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_threadnumber (uint8_t soc_die_num, uint8_t *buffer)

  This register indicates the maximum number of threads present. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_mp0_msg (uint8_t soc_die_num, uint8_t *buffer)

  This register will read the message running on the MP0. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_alert_status (uint8_t soc_die_num, uint8_t num_of_alert_status_reg, uint8_t **buffer)

  This function will read bit vector for all the threads. Value of 1 indicates MCE occured for the thread and is set by hardware. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_alert_mask (uint8_t soc_die_num, uint8_t num_of_alert_mask_reg, uint8_t **buffer)

  This function will read bit vector for all the threads. Value of 1 indicates alert signaling disabled for corresponding SBRMI::AlertStatus[MceStat] for the thread. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_inbound_msg (uint8_t soc_die_num, uint8_t *buffer)

  This register will read the inbound message. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_outbound_msg (uint8_t soc_die_num, uint8_t *buffer)

  This register will read the outbound message. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_threadnumberlow (uint8_t soc_die_num, uint8_t *buffer)

  This register indicates the low part of maximum number of threads. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_threadnumberhi (uint8_t soc_die_num, uint8_t *buffer)

  This register indicates the upper part of maximum number of threads. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbrmi_thread_cs (uint8_t soc_die_num, uint8_t *buffer)

  This register is used to read the thread cs. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbrmi_ras_status (uint8_t soc_die_num, uint8_t *buffer)

  This register will read the ras status. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t clear_sbrmi_ras_status (uint8_t soc_die_num, uint8_t buffer)

  This API will clear ras status register. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t esmi_get_threads_per_socket (uint8_t soc_die_num, uint32_t *threads_per_socket)

  Get the number of threads per socket.
- oob_status_t clear_sw_async_alert_status (uint8_t soc_die_num)

  Clears the software asynchronous alert status for a specific SoC die.
- oob_status_t clear_MP0_alert_status (uint8_t soc_die_num)

  Clears the MP0 alert status for the specified SoC die.
- oob_status_t set_sw_async_alert_mask (uint8_t soc_die_num, bool alert_mask_bit)

  Set the software asynchronous alert mask for a specific SoC die.
- oob_status_t read_sbrmi_ready_status (uint8_t soc_die_num, bool *buffer)

  Reads the SBRMI ready status for a specified SoC die.
- oob_status_t clear_shut_down_err (uint8_t soc_die_num)

  Clears the shutdown error for the specified SoC die.
- oob_status_t gpio_assertion_on_mailbox (uint8_t soc_die_num, uint8_t dis_alert)

  Controls enable/disable Alert_L assertion for mailbox for a specific SOC die.
- oob_status_t gpio_assertion_for_async_alerts (uint8_t soc_die_num, uint8_t dis_alert)

  Controls enable/disable Alert_L assertion for asynchronous alert signals.

Variables

- const uint8_t thread_en_reg_v10 [MAX_THREAD_REG_V10]

  thread enable register revision 0x10
- const uint8_t thread_en_reg_v20 [MAX_THREAD_REG_V20]

  thread enable register revision 0x20
- const uint8_t alert_status [MAX_ALERT_REG]

  alert status register
- const uint8_t alert_mask [MAX_ALERT_REG]

  alert mask
- const uint16_t thread_en_reg_v21_dense [MAX_THREAD_REG_V21_DENSE]

  thread enable register revision 0x21 for dense plaform
- const uint16_t alert_status_v21_dense [MAX_ALERT_REG_V21_DENSE]

  alert status register for V21 dense platform
- const uint16_t alert_mask_v21_dense [MAX_ALERT_REG_V21_DENSE]

  alert mask

7.6.1 Detailed Description

Header file for the APML library for SB-RMI functionality access. All required function, structure, enum, etc. definitions should be defined in this file for SB-RMI Register accessing.
This header file contains the following: APIs prototype of the APIs exported by the APML library. Description of the API, arguments and return values. The Error codes returned by the API.

7.7 esmi_tsi.h File Reference

```
#include "apml_err.h"
```

Macros

- #define TEMP_INC 0.125

  Register encode the temperature to increase in 0.125 In decimal portion one increase in byte is equivalent to 0.125.

Enumerations

- enum sbtsi_registers {
  SBTSI_CPUTEMPINT = 0x1 , **SBTSI_STATUS** , **SBTSI_CONFIGURATION** , **SBTSI_UPDATERATE** ,
  SBTSI_HITEMPINT = 0x7 , **SBTSI_LOTEMPINT** , **SBTSI_CONFIGWR** , **SBTSI_CPUTEMPDEC** = 0x10 ,
  SBTSI_CPUTEMPOFFINT , **SBTSI_CPUTEMPOFFDEC** , **SBTSI_HITEMPDEC** , **SBTSI_LOTEMPDEC** ,
  SBTSI_TIMEOUTCONFIG = 0x22 , **SBTSI_ALERTTHRESHOLD** = 0x32 , **SBTSI_ALERTCONFIG** = 0xBF ,
  SBTSI_MANUFID = 0xFE ,
  SBTSI_REVISION = 0xFF }

  SB-TSI(Side-Band Temperature Sensor Interface) commands register access. The below registers mentioned as per Genessis PPR.

- enum sbtsi_config_write { **ARA_MASK** = 0x2 , **READORDER_MASK** = 0x20 , **RUNSTOP_MASK** = 0x40 , **ALERTMASK_MASK** = 0x80 }

  Bitfield values to be set for SBTSI confirwr register [7] Alert mask [6] RunStop [5] ReadOrder [1] AraDis.

Functions

- oob_status_t read_sbtsi_cpuinttemp (uint8_t soc_die_num, uint8_t *buffer)

  integer CPU temperature value The CPU temperature is calculated by adding the CPU temperature offset(↩ SBTSI::CpuTempOffInt, SBTSI::CpuTempOffDec) to the processor control temperature (Tctl). SBTSI::CpuTempInt and SBTSI::CpuTempDec combine to return the CPU temperature. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_status (uint8_t soc_die_num, uint8_t *buffer)

  Status register is Read-only, volatile field If SBTSI::AlertConfig[AlertCompEn] == 0 , the temperature alert is latched high until the alert is read. If SBTSI::AlertConfig[AlertCompEn] == 1, the alert is cleared when the temperature does not meet the threshold conditions for temperature and number of samples. Supported platforms↩ : Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_config (uint8_t soc_die_num, uint8_t *buffer)

  The bits in this register are Read-only and can be written by Writing to the corresponding bits in SBTSI↩ ::ConfigWr Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_updaterate (uint8_t soc_die_num, float *buffer)

  This register value specifies the rate at which CPU temperature is compared against the temperature thresholds to determine if an alert event has occurred. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t write_sbtsi_updaterate (uint8_t soc_die_num, float uprate)

  This register value specifies the rate at which CPU temperature is compared against the temperature thresholds to determine if an alert event has occurred. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_hitempint (uint8_t soc_die_num, uint8_t *buffer)

  This value specifies the integer portion of the high temperature threshold. The high temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is greater than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_lotempint (uint8_t soc_die_num, uint8_t *buffer)

  This value specifies the integer portion of the low temperature threshold. The low temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is less than or equal to the threshold Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_configwrite (uint8_t soc_die_num, uint8_t *buffer)

  This register provides write access to SBTSI::Config Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t read_sbtsi_cputempdecimal (uint8_t soc_die_num, float *buffer)

  The value returns the decimal portion of the CPU temperature Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbtsi_cputempoffint (uint8_t soc_die_num, uint8_t *temp_int)

  SBTSI::CpuTempOffInt and SBTSI::CpuTempOffDec combine to specify the CPU temperature offset Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbtsi_cputempoffdec (uint8_t soc_die_num, float *temp_dec)

  This value specifies the decimal/fractional portion of the CPU temperature offset added to Tctl to calculate the CPU temperature. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbtsi_hitempdecimal (uint8_t soc_die_num, float *temp_dec)

  This value specifies the decimal portion of the high temperature threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbtsi_lotempdecimal (uint8_t soc_die_num, float *temp_dec)

  value specifies the decimal portion of the low temperature threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbtsi_timeoutconfig (uint8_t soc_die_num, uint8_t *timeout)

  value specifies 0=SMBus defined timeout support disabled. 1=SMBus defined timeout support enabled. SMBus timeout enable. If SB-RMI is in use, SMBus timeouts should be enabled or disabled in a consistent manner on both interfaces. SMBus defined timeouts are not disabled for SB-RMI when this bit is set to 0. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbtsi_alertthreshold (uint8_t soc_die_num, uint8_t *samples)

  Specifies the number of consecutive CPU temperature samples for which a temperature alert condition needs to remain valid before the corresponding alert bit is set. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbtsi_alertconfig (uint8_t soc_die_num, uint8_t *mode)

  Status register is Read-only, volatile field If SBTSI::AlertConfig[AlertCompEn] == 0 , the temperature alert is latched high until the alert is read. If SBTSI::AlertConfig[AlertCompEn] == 1, the alert is cleared when the temperature does not meet the threshold conditions for temperature and number of samples. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbtsi_manufid (uint8_t soc_die_num, uint8_t *man_id)

  Returns the AMD manufacture ID Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbtsi_revision (uint8_t soc_die_num, uint8_t *rivision)

  Specifies the SBI temperature sensor interface revision Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t sbtsi_get_cputemp (uint8_t soc_die_num, float *cpu_temp)

  CPU temperature value The CPU temperature is calculated by adding SBTSI::CpuTempInt and SBTSI::CpuTempDec combine to return the CPU temperature. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t sbtsi_get_temp_status (uint8_t soc_die_num, uint8_t *loalert, uint8_t *hialert)

  Status register is Read-only, volatile field If SBTSI::AlertConfig[AlertCompEn] == 0 , the temperature alert is latched high until the alert is read. If SBTSI::AlertConfig[AlertCompEn] == 1, the alert is cleared when the temperature does not meet the threshold conditions for temperature and number of samples. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t sbtsi_get_config (uint8_t soc_die_num, uint8_t *al_mask, uint8_t *run_stop, uint8_t *read_ord, uint8_t *ara)

  The bits in this register are Read-only and can be written by Writing to the corresponding bits in SBTSI::ConfigWr Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t sbtsi_set_configwr (uint8_t soc_die_num, uint8_t mode, uint8_t config_mask)

  The bits in this register are defined sbtsi_config_write and can be written by writing to the corresponding bits in SBTSI::ConfigWr Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t sbtsi_get_timeout (uint8_t soc_die_num, uint8_t *timeout_en)

To verify if timeout support enabled or disabled Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

- oob_status_t sbtsi_set_timeout_config (uint8_t soc_die_num, uint8_t mode)

  To enable/disable timeout support Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t sbtsi_set_hitemp_threshold (uint8_t soc_die_num, float hitemp_thr)

  This value set the high temperature threshold. The high temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is greater than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t sbtsi_set_lotemp_threshold (uint8_t soc_die_num, float lotemp_thr)

  This value set the low temperature threshold. The low temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is less than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t sbtsi_get_hitemp_threshold (uint8_t soc_die_num, float *hitemp_thr)

  This value specifies the high temperature threshold. The high temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is greater than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t sbtsi_get_lotemp_threshold (uint8_t soc_die_num, float *lotemp_thr)

  This value specifies the low temperature threshold. The low temperature threshold specifies the CPU temperature that causes ALERT_L to assert if the CPU temperature is less than or equal to the threshold. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t read_sbtsi_cputempoffset (uint8_t soc_die_num, float *temp_offset)

  SBTSI::CpuTempOffInt and SBTSI::CpuTempOffDec combine to specify the CPU temperature offset Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t write_sbtsi_cputempoffset (uint8_t soc_die_num, float temp_offset)

  SBTSI::CpuTempOffInt and SBTSI::CpuTempOffDec combine to set the CPU temperature offset Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t sbtsi_set_alert_threshold (uint8_t soc_die_num, uint8_t samples)

  Specifies the number of consecutive CPU temperature samples for which a temperature alert condition needs to remain valid before the corresponding alert bit is set. Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.
- oob_status_t sbtsi_set_alert_config (uint8_t soc_die_num, uint8_t mode)

  Alert comparator mode enable Supported platforms: Fam-19h_Mod-00h-0Fh, Fam-19h_Mod-10h-1Fh, Fam-19h_Mod-90h-9Fh and Fam-1Ah_Mod-00h-0Fh.

7.7.1 Detailed Description

Header file for the APML library for SB-TSI functionality access. All required function, structure, enum, etc. definitions should be defined in this file for SB-TSI Register accessing.
This header file contains the following: APIs prototype of the APIs exported by the APML library. Description of the API, arguments and return values. The Error codes returned by the API.

7.8 rmi_mailbox_mi300.h File Reference

```
#include "apml_common.h"
#include "apml_err.h"
```

Data Structures

- struct max_mem_bw

  Structure for Max DDR/HBM bandwidth and utilization. It contains max bandwidth(16 bit data) in GBps, current utilization bandwidth(Read+Write)(16 bit data) in GBps.
- struct svi_port_domain

  struct containing port and slave address.

- struct freq_limits

  struct containing max frequency and min frequencey limit
- struct mclk_fclk_pstates

  struct containing memory clock and fabric clock pstate mappings.
- struct statistics

  struct containing statistics parameter of interest and output control pstate mappings.
- struct xgmi_speed_rate_n_width

  struct containing xgmi speed rate in MHZ and link width in units of Gpbs. If link_width[0] = 1 then XGMI link X2 is supported. If link_width[1] = 1 then XGMI link X4 is supported. If Link_width[2] = 1 then XGMI link X4 is supported and similarly if link_width[3] = 1 then XGMI link X8 is supported.
- struct hbm_device_info

  struct containing device vendor, part number and total memory size in GBs.
- struct host_status

  struct containing power management controlling status, driving running status.

Macros

- #define MI300A_ENCODING_SIZE 10

  MI300A encoding size //.

Enumerations

- enum esb_mi300_mailbox_commmands {
  SET_MAX_GFX_CORE_CLOCK = 0x81 , **SET_MIN_GFX_CORE_CLOCK** , **SET_MAX_PSTATE** , **GET_PSTATES** ,
  GET_CURR_XGMI_PSTATE , **SET_XGMI_PSTATE** , **UNSET_XGMI_PSTATE** , **GET_XGMI_PSTATES** ,
  GET_XCC_IDLE_RESIDENCY , **GET_ENERGY_ACCUMULATOR** = 0x90 , **GET_RAS_ALARMS** , **GET_PM_ALARMS** ,
  GET_PSN , **GET_LINK_INFO** , **GET_ABS_MAX_MIN_GFX_FREQ** = 0x96 , **GET_SVI_TELEMETRY_BY_RAIL** ,
  GET_DIE_TYPE , **GET_ACT_GFX_FREQ_CAP_SELECTED** = 0x9c , **GET_DIE_HOT_SPOT_INFO** = 0xA0 , **GET_MEM_HOT_SPOT_INFO** ,
  GET_MAX_OP_TEMP , **GET_SLOW_DOWN_TEMP** , **GET_STATUS** = 0xA4 , **GET_MAX_MEM_BW_UTILIZATION** = 0XB0 ,
  GET_HBM_THROTTLE , **SET_HBM_THROTTLE** , **GET_HBM_STACK_TEMP** , **GET_GFX_CLK_FREQ_LIMITS** ,
  GET_FCLK_FREQ_LIMITS , **GET_SOCKETS_IN_SYSTEM** , **GET_HBM_DEVICE_INFO** , **GET_PCIE_STATS** = 0xBA ,
  GET_BIST_RESULTS = 0xBC , **QUERY_STATISTICS** , **CLEAR_STATISTICS** }

  Mailbox message types defined in the APML library.
- enum range_type { **MIN** = 0 , **MAX** }

  APML range type used by GFX core clock frequency. Max, MIN are the values. Min is 0 and Max is 1.
- enum clk_type { **GFX_CLK** = 0 , **F_CLK** }

  APML clock frequency type. GFX_CLK or F_CLK GFX_CLK value is 0 and F_CLK value is 1.
- enum alarms_type { **PM** }

  APML alarms type. PM_ALARMS. PM is 0.

Functions

- oob_status_t set_gfx_core_clock (uint8_t soc_die_num, enum range_type freq_type, uint32_t freq)

  Set maximum/minimum gfx core clock frequency.
- oob_status_t set_mclk_fclk_max_pstate (uint8_t soc_die_num, uint32_t pstate)

  Set maximum mem and fclck pstate.
- oob_status_t get_mclk_fclk_pstates (uint8_t soc_die_num, uint8_t pstate_ind, struct mclk_fclk_pstates *pstate)

Get memory and fabric clock power state mappings.

- oob_status_t set_xgmi_pstate (uint8_t soc_die_num, uint32_t pstate)

  Sets the XGMI Pstate.
- oob_status_t unset_xgmi_pstate (uint8_t soc_die_num)

  Resets the XGMI Pstate.
- oob_status_t get_xgmi_pstates (uint8_t soc_die_num, uint8_t pstate_ind, struct xgmi_speed_rate_n_width *xgmi_pstate)

  Read XGMI power state mappings.
- oob_status_t get_xcc_idle_residency (uint8_t soc_die_num, uint32_t *gfx_cores_idle_res)

  Read xcc idle residency percentage.
- oob_status_t get_energy_accum_with_timestamp (uint8_t soc_die_num, uint64_t *energy, uint64_t *time_stamp)

  Read energy accumulator with time stamp.
- oob_status_t get_alarms (uint8_t soc_die_num, enum alarms_type type, uint32_t *buffer)

  Read PM alarm status based on enumeration type alarms_type.
- oob_status_t get_psn (uint8_t soc_die_num, uint32_t die_index, uint64_t *buffer)

  Reads public serial number (PSN).
- oob_status_t get_link_info (uint8_t soc_die_num, uint8_t *link_config, uint8_t *module_id)

  Read link Info.
- oob_status_t get_max_min_gfx_freq (uint8_t soc_die_num, uint16_t *max_freq, uint16_t *min_freq)

  Read maximum and minimum allowed GFX engine frequency.
- oob_status_t get_act_gfx_freq_cap (uint8_t soc_die_num, uint16_t *freq)

  Read Actual GFX frequency cap selected.
- oob_status_t get_svi_rail_telemetry (uint8_t soc_die_num, struct svi_port_domain port, uint32_t *pow)

  Read SVI based telemetry for individual rails.
- oob_status_t get_die_hotspot_info (uint8_t soc_die_num, uint8_t *die_id, uint16_t *temp)

  Reads local ID of the hottest die and its temperature.
- oob_status_t get_mem_hotspot_info (uint8_t soc_die_num, uint8_t *hbm_stack_id, uint16_t *hbm_temp)

  Reads local ID of the HBM stack and its temperature.
- oob_status_t get_host_status (uint8_t soc_die_num, struct host_status *status)

  Reads the status in a bit vector.
- oob_status_t get_max_mem_bw_util (uint8_t soc_die_num, struct max_mem_bw *bw)

  Reads max memory bandwidth utilization.
- oob_status_t get_hbm_throttle (uint8_t soc_die_num, uint32_t *mem_th)

  Reads HBM throttle.
- oob_status_t set_hbm_throttle (uint8_t soc_die_num, uint32_t mem_th)

  writes HBM throttle.
- oob_status_t get_hbm_temperature (uint8_t soc_die_num, uint32_t index, uint16_t *temp)

  Reads hbm stack temperature.
- oob_status_t get_clk_freq_limits (uint8_t soc_die_num, enum clk_type type, struct freq_limits *limit)

  Reads GFX/F clk frequency limits based on enumeration type clk_type .
- oob_status_t get_sockets_in_system (uint8_t soc_die_num, uint32_t *sockets_count)

  Reads number of sockets in system.
- oob_status_t get_bist_results (uint8_t soc_die_num, uint8_t die_id, uint32_t *bist_result)

  Reads die level bist result status from package.
- oob_status_t get_statistics (uint8_t soc_die_num, struct statistics stat, uint32_t *param)

  Reads statistics for a given parameter.
- oob_status_t clear_statistics (uint8_t soc_die_num)

  Clears statistics.
- oob_status_t get_die_type (uint8_t soc_die_num, uint32_t data_in, uint32_t *data_out)

  Get die type.

- oob_status_t get_curr_xgmi_pstate (uint8_t soc_die_num, uint8_t *xgmi_pstate)

  Get current xgmi pstate Supported platforms: Fam-19h_Mod-90h-9Fh.
- oob_status_t get_max_operating_temp (uint8_t soc_die_num, uint32_t ctf_type, uint16_t *temp)

  Get maximum operating temperature.
- oob_status_t get_slow_down_temp (uint8_t soc_die_num, uint32_t ctf_type, uint16_t *slow_down_temp)

  Get slow down temperature Supported platforms: Fam-19h_Mod-90h-9Fh.
- oob_status_t get_hbm_dev_info (uint8_t soc_die_num, struct hbm_device_info *dev_info)

  Get hbm device information.
- oob_status_t get_pciestats (uint8_t soc_die_num, uint32_t pcie_stat_select, uint32_t *pcie_stats)

  Get PCIe statistics.

7.8.1 Detailed Description

Header file for the MI300 mailbox messages supported by APML library. All required function, structure, enum, etc. definitions should be defined in this file.
This header file contains the following: APIs prototype of the Mailbox messages for MI300 exported by the APML library. Description of the API, arguments and return values. The Error codes returned by the API.

7.8.2 Function Documentation

7.8.2.1 set_gfx_core_clock()

```
oob_status_t set_gfx_core_clock (
            uint8_t soc_die_num,
            enum range_type freq_type,
            uint32_t freq )
```

Set maximum/minimum gfx core clock frequency.
This function sets user provied frequency as MAX GFX/MIN GFX core clock frequency in MHZ based on enumeration type range_type. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*freq_type*	enumeration type range_type containing "MIN" = 0 or "MAX" = 1.
in	*freq*	frequency in MHZ.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.2 set_mclk_fclk_max_pstate()

```
oob_status_t set_mclk_fclk_max_pstate (
            uint8_t soc_die_num,
            uint32_t pstate )
```

Set maximum mem and fclck pstate.
This function sets the maximum memory and fabric clock power state. Mappings from memory and fabric clock pstate to MEMCLK/FCLK frequency can be found by issuing GetPstates command. Supported platforms: Fam-19h_Mod-90h-9Fh.
In case if the in-band has also set the maximum Pstate, then lower of the limits is used.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*pstate*	maximum pstate.Valid pstate range is 0 - 3.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.3 get_mclk_fclk_pstates()

```
oob_status_t get_mclk_fclk_pstates (
            uint8_t soc_die_num,
            uint8_t pstate_ind,
            struct mclk_fclk_pstates * pstate )
```

Get memory and fabric clock power state mappings.

This function returns the memory and fabric clock power state mappings. Returns MEMCLK/FCLK frequency in units of 1MHz for the available clock power states (Pstates). Each MEMCLK/FCLK frequency pair is returned independently for each pstate. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*pstate_ind*	index of the pstate.
out	*pstate*	struct mclk_fclk_pstates containing mem_clk and f_clk frequency in MHz.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.4 set_xgmi_pstate()

```
oob_status_t set_xgmi_pstate (
            uint8_t soc_die_num,
            uint32_t pstate )
```

Sets the XGMI Pstate.

This function sets the specified XGMI Pstate. This disables all active XGMI Pstate management although XGMI power down modes will still be supported. Only 2 XGMI states are supported (0/1). Supported platforms↩ : Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*pstate*	power state. valid values are 0 - 1.

Return values

OOB_SUCCESS	is returned upon successful call.

Return values

Non-zero	is returned upon failure.

7.8.2.5 unset_xgmi_pstate()

```
oob_status_t unset_xgmi_pstate (
            uint8_t soc_die_num )
```

Resets the XGMI Pstate.

This function resets the XGMI Pstate specified in the SetXgmiPstate, causing XGMI link speed/width to be actively managed by the GPU. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.6 get_xgmi_pstates()

```
oob_status_t get_xgmi_pstates (
            uint8_t soc_die_num,
            uint8_t pstate_ind,
            struct xgmi_speed_rate_n_width * xgmi_pstate )
```

Read XGMI power state mappings.

This function reads the XGMI power state mappings.Reads the supported XGMI link speeds and widths available to the SetXgmiPstate message.Link speeds reported in units of 1Gpbs. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*pstate_ind*	xgmi pstate index for speed rate.
out	*xgmi_pstate*	struct xgmi_speed_rate_n_width containing speed rate in MHz and link width

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.7 get_xcc_idle_residency()

```
oob_status_t get_xcc_idle_residency (
            uint8_t soc_die_num,
            uint32_t * gfx_cores_idle_res )
```

Read xcc idle residency percentage.

This function will provide the average xcc idle residency across all GFX cores in the socket.100% specifies that all enabled GFX cores in the socket are running in idle. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*gfx_cores_idle_res*	idle residency in percentage

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.8 get_energy_accum_with_timestamp()

```
oob_status_t get_energy_accum_with_timestamp (
            uint8_t soc_die_num,
            uint64_t * energy,
            uint64_t * time_stamp )
```

Read energy accumulator with time stamp.

This function will read 64 bits energy accumulator and the 56 bit time stamp. Supported platforms : Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*energy*	accumulator 2^ 16 J.
out	*time_stamp*	time stamp (units:10ns).

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.9 get_alarms()

```
oob_status_t get_alarms (
            uint8_t soc_die_num,
            enum alarms_type type,
            uint32_t * buffer )
```

Read PM alarm status based on enumeration type alarms_type.

This function provides PM alarm status if the enumeration type alarms_type is PM = 0 then it will retrieve PM alarm status . If buffer value is 1 the status is VRHOT. If the buffer value is 2 status is die over temp. If the buffer value is 4 status is HBM over temp and if the buffer is 8 then status is PWRBRK. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*type*	enumeration type alarms_type. PM = 0.
out	*buffer*	returns PSP fw return data.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.10 get_psn()

```
oob_status_t get_psn (
            uint8_t soc_die_num,
            uint32_t die_index,
            uint64_t * buffer )
```

Reads public serial number (PSN).

This function will return 64 bit public serial number (PSN) unique to each die. Supported platforms↩ : Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*die_index*	core/die index.
out	*buffer*	returns 64 bit unique public serial number.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.11 get_link_info()

```
oob_status_t get_link_info (
            uint8_t soc_die_num,
            uint8_t * link_config,
            uint8_t * module_id )
```

Read link Info.

This function will read link info. Function will read the module ID and link config reflecting strapping pins. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*link_config*	link configuration.
out	*module_id*	module ID.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.12 get_max_min_gfx_freq()

```
oob_status_t get_max_min_gfx_freq (
            uint8_t soc_die_num,
            uint16_t * max_freq,
            uint16_t * min_freq )
```

Read maximum and minimum allowed GFX engine frequency.

This function will read maximum and minimum allowed GFX engine frequency. Supported platforms↩ : Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*max_freq*	maximum GFX frequency in MHZ.
out	*min_freq*	minimum GFX frequency in MHZ.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.13 get_act_gfx_freq_cap()

```
oob_status_t get_act_gfx_freq_cap (
            uint8_t soc_die_num,
            uint16_t * freq )
```

Read Actual GFX frequency cap selected.

This function will read current seleted GFX engine clock frequency.It reflects minimum of all frequency caps seleted via in-band and out-of-band controls. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*freq*	maximum GFX frequency in MHZ.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.14 get_svi_rail_telemetry()

```
oob_status_t get_svi_rail_telemetry (
            uint8_t soc_die_num,
            struct svi_port_domain port,
            uint32_t * pow )
```

Read SVI based telemetry for individual rails.

This function will read SVI based telemetry for individual rails. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.

Parameters

in	*port*	struct svi_telemetry_domain containing port and slave address.
out	*pow*	power in milliwatts.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.15 get_die_hotspot_info()

```
oob_status_t get_die_hotspot_info (
            uint8_t soc_die_num,
            uint8_t * die_id,
            uint16_t * temp )
```

Reads local ID of the hottest die and its temperature.

This function will read local ID of the hottest die and its corresponding die temperature. Measured in every 1 ms and the most recently measured temperature in ℃ is reported. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*die_id*	Hottest die ID.
out	*temp*	Die hot spot temperature in ℃.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.16 get_mem_hotspot_info()

```
oob_status_t get_mem_hotspot_info (
            uint8_t soc_die_num,
            uint8_t * hbm_stack_id,
            uint16_t * hbm_temp )
```

Reads local ID of the HBM stack and its temperature.

This function will read local ID of the HBM stack and its corresponding HBM stack temperature. Measured in every 1 ms and the most recently measured temperature is reported. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*hbm_stack↩ _id*	Local ID of the hottest HBM stack.
out	*hbm_temp*	temperature in units of 1 ℃.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.17 get_host_status()

```
oob_status_t get_host_status (
            uint8_t soc_die_num,
            struct host_status * status )
```

Reads the status in a bit vector.

This function will read PM controller status and driver running status in a bit vector Supported platforms↩ : Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*status*	struct host_status containing power management controller status and driver running status.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.18 get_max_mem_bw_util()

```
oob_status_t get_max_mem_bw_util (
            uint8_t soc_die_num,
            struct max_mem_bw * bw )
```

Reads max memory bandwidth utilization.

This function will provide theoretic.al maximum HBM/memory bandwidth of the system in GB/s, utilized bandwidth in GB/S. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*bw*	struct max_mem_bw containing max bw, utilized b/w.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.19 get_hbm_throttle()

```
oob_status_t get_hbm_throttle (
            uint8_t soc_die_num,
            uint32_t * mem_th )
```

Reads HBM throttle.
This function will read HBM throttle in percentage. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*mem_th*	hbm throttle in percentage (0 - 100%).

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.20 set_hbm_throttle()

```
oob_status_t set_hbm_throttle (
            uint8_t soc_die_num,
            uint32_t mem_th )
```

writes HBM throttle.
This function will write HBM throttle. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*mem_th*	hbm throttle in percentage (0 - 80%).

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.21 get_hbm_temperature()

```
oob_status_t get_hbm_temperature (
            uint8_t soc_die_num,
            uint32_t index,
            uint16_t * temp )
```

Reads hbm stack temperature.
This function will read particular hbm stack temperature. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*index*	hbm stack index (0 - 7 for MI300).
out	*temp*	temperature in units of 1 ℃.

Return values

OOB_SUCCESS	is returned upon successful call.

Return values

Non-zero	is returned upon failure.

7.8.2.22 get_clk_freq_limits()

```
oob_status_t get_clk_freq_limits (
            uint8_t soc_die_num,
            enum clk_type type,
            struct freq_limits * limit )
```

Reads GFX/F clk frequency limits based on enumeration type clk_type .
This function will provide socket's GFX/F clk max and min frequnecy limits based on enumberation type clk_type . The function reads GFX clk frequency limits if the enumberation type clk_type is GFX_CLK = 0 else it will read F_CLK frequency limits. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*type*	enumeration type clk_type . Values are "GFX_CLK" = 0 or "F_CLK" = 1.
out	*limit*	struct freq_limits containing max and min GFX/F_clk frequency in MHZ.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.23 get_sockets_in_system()

```
oob_status_t get_sockets_in_system (
            uint8_t soc_die_num,
            uint32_t * sockets_count )
```

Reads number of sockets in system.
This function will read number of sockets in system. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*sockets_count*	Numbers of sockets in system

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.24 get_bist_results()

```
oob_status_t get_bist_results (
            uint8_t soc_die_num,
```

```
            uint8_t die_id,
            uint32_t * bist_result )
```

Reads die level bist result status from package.

This function will read die level bist result status from package. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*die_id*	die level id.
out	*bist_result*	constituent bist results depending on MI300X/A/C configuration.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.25 get_statistics()

```
oob_status_t get_statistics (
            uint8_t soc_die_num,
            struct statistics stat,
            uint32_t * param )
```

Reads statistics for a given parameter.

This function will read statistics for a given parameter since the last clear statistics command. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*stat*	struct statistics containing statistics parameter of interest and output control.
out	*param*	parameter or timestamp HI/Lo value.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.26 clear_statistics()

```
oob_status_t clear_statistics (
            uint8_t soc_die_num )
```

Clears statistics.

This function will clear all stored query statistics timestamps and then resumes data collection or aggregation. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.27 get_die_type()

```
oob_status_t get_die_type (
            uint8_t soc_die_num,
            uint32_t data_in,
            uint32_t * data_out )
```

Get die type.

This function will read die-type, counts and AID base die based on die-ID input. If the bit[0] of input/data_in is 1 then it will get maximum die-ID (0 - 255). If the bit[0] of input/data_in is 0 then the data_out[7:0] will be die type i.e. 0 Not available, 1 means AID, 2 means XCD, 3 means CCD and 4 means HBM stack and 5- 255 are reserved. data_out[15:8] means maximum coumt of currently indexed die type. Data_out[19:16] means AID associated with specified die-ID. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*data_in*	input to get the die-type, counts and AID base.
out	*data_out*	maximum die id idnex or the die type based on die-id input.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.28 get_curr_xgmi_pstate()

```
oob_status_t get_curr_xgmi_pstate (
            uint8_t soc_die_num,
            uint8_t * xgmi_pstate )
```

Get current xgmi pstate Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*xgmi_pstate*	current xgmi pstate

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.29 get_max_operating_temp()

```
oob_status_t get_max_operating_temp (
            uint8_t soc_die_num,
            uint32_t ctf_type,
            uint16_t * temp )
```

Get maximum operating temperature.

This function will get critical temperature fault core and HBM temperature. Supported platforms↩ : Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*ctf_type*	critical temperature fault type. 0 for AID, 1 for CCD, 2 for XCD and 3 for HBM.
out	*temp*	Max operating temp in ℃

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.30 get_slow_down_temp()

```
oob_status_t get_slow_down_temp (
            uint8_t soc_die_num,
            uint32_t ctf_type,
            uint16_t * slow_down_temp )
```

Get slow down temperature Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*ctf_type*	critical temperature fault type. 0 for AID, 1 for CCD, 2 for XCD and 3 for HBM.
out	*slow_down_temp*	slow down temperature in ℃

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.31 get_hbm_dev_info()

```
oob_status_t get_hbm_dev_info (
            uint8_t soc_die_num,
            struct hbm_device_info * dev_info )
```

Get hbm device information.

This function will get hbm device information i.e. device vendor, part number and total memory size (GBs). Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*dev_info*	struct hbm_device_info contatining device vendor, part number and total memory size (GBs).

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.8.2.32 get_pciestats()

```
oob_status_t get_pciestats (
            uint8_t soc_die_num,
            uint32_t pcie_stat_select,
            uint32_t * pcie_stats )
```

Get PCIe statistics.

This function will get PCIe statistics including total transitions from L0 to recovery state, total number of replays issued on the PCIe link, total number of NAKs issued on the PCIe link by the device, and total number of NAKs issued on the PCIe link by the receiver. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*pcie_stat_select*	PCIe stat selector. 0 for L0 to recovery count, 1 for replay count, 2 for NAK sent count and 3 for NAK recieved count.
out	*pcie_stats*	pcie statistics

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9 tsi_mi300.h File Reference

```
#include "apml_err.h"
```

Enumerations

- enum sbtsi_mi300_registers {
  SBTSI_HBM_HITEMPINT_LIMIT = 0x40 , **SBTSI_HBM_HITEMPDEC_LIMIT** = 0x44 , **SBTSI_HBM_LOTEMPINT_LIMIT** = 0x48 , **SBTSI_HBM_LOTEMPDEC_LIMIT** = 0x4C ,
  SBTSI_MAX_HBMTEMPINT = 0x50 , **SBTSI_MAX_HBMTEMPDEC** = 0x54 , **SBTSI_HBMTEMPINT** = 0x5C , **SBTSI_HBMTEMPDEC** = 0x60 }

  SB-TSI(Side-Band Temperature Sensor Interface) commands register access.

Functions

- oob_status_t read_sbtsi_hbm_hi_temp_int_th (uint8_t soc_die_num, uint8_t *buffer)
  Get high hbm temperature integer threshold in ℃.
- oob_status_t write_sbtsi_hbm_hi_temp_th (uint8_t soc_die_num, float hi_temp_th)
  Set high hbm temperature integer and decimal threshold in ℃.
- oob_status_t read_sbtsi_hbm_hi_temp_dec_th (uint8_t soc_die_num, float *buffer)
  Get high hbm temperature decimal threshold.
- oob_status_t read_sbtsi_hbm_hi_temp_th (uint8_t soc_die_num, float *buffer)
  Get high hbm temperature threshold.
- oob_status_t read_sbtsi_hbm_lo_temp_int_th (uint8_t soc_die_num, uint8_t *buffer)
  Get low hbm temperature integer threshold.
- oob_status_t read_sbtsi_hbm_lo_temp_dec_th (uint8_t soc_die_num, float *buffer)
  Get low hbm temperature decimal threshold.
- oob_status_t write_sbtsi_hbm_lo_temp_th (uint8_t soc_die_num, float temp_th)
  Set low hbm temperature threshold.
- oob_status_t read_sbtsi_max_hbm_temp_int (uint8_t soc_die_num, uint8_t *buffer)
  Get max hbm integer temperature.
- oob_status_t read_sbtsi_max_hbm_temp_dec (uint8_t soc_die_num, float *buffer)
  Get max hbm decimal temperature.
- oob_status_t read_sbtsi_hbm_temp_int (uint8_t soc_die_num, uint8_t *buffer)
  Get hbm integer temperature.
- oob_status_t read_sbtsi_hbm_temp_dec (uint8_t soc_die_num, float *buffer)
  Get hbm decimal temperature.
- oob_status_t read_sbtsi_hbm_lo_temp_th (uint8_t soc_die_num, float *buffer)
  Get hbm low temperature threshold.
- oob_status_t read_sbtsi_max_hbm_temp (uint8_t soc_die_num, float *buffer)
  Get hbm maximum temperature.
- oob_status_t read_sbtsi_hbm_temp (uint8_t soc_die_num, float *buffer)
  Get hbm temperature.
- oob_status_t read_sbtsi_hbm_alertthreshold (uint8_t soc_die_num, uint8_t *samples)
  Get hbm alert threshold.
- oob_status_t sbtsi_set_hbm_alert_threshold (uint8_t soc_die_num, uint8_t samples)
  Set hbm alert samples.
- oob_status_t get_sbtsi_hbm_alertconfig (uint8_t soc_die_num, uint8_t *mode)
  Get the sbtsi hbm alert config.
- oob_status_t set_sbtsi_hbm_alertconfig (uint8_t soc_die_num, uint8_t mode)
  Set hbm alert config.

7.9.1 Detailed Description

Header file for the APML library for SB-TSI functionality access for MI300. All required function, structure, enum, etc. definitions should be defined in this file for SB-TSI Register accessing.
This header file contains the following: APIs prototype of the APIs exported by the APML library. Description of the API, arguments and return values. The Error codes returned by the API.

7.9.2 Function Documentation

7.9.2.1 read_sbtsi_hbm_hi_temp_int_th()

```
oob_status_t read_sbtsi_hbm_hi_temp_int_th (
            uint8_t soc_die_num,
            uint8_t * buffer )
```

Get high hbm temperature integer threshold in ℃.
This function will read high hbm temperature interger threshold in ℃. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	a pointer to hold the high hbm temperature integer threshold in ℃.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.2 write_sbtsi_hbm_hi_temp_th()

```
oob_status_t write_sbtsi_hbm_hi_temp_th (
            uint8_t soc_die_num,
            float hi_temp_th )
```

Set high hbm temperature integer and decimal threshold in ℃.
This function will set high hbm temperature interger and decimal threshold in ℃. Supported platforms↩ : Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*hi_temp_th*	high hbm temperature threshold containing integer and decimal part in ℃.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.3 read_sbtsi_hbm_hi_temp_dec_th()

```
oob_status_t read_sbtsi_hbm_hi_temp_dec_th (
            uint8_t soc_die_num,
            float * buffer )
```

Get high hbm temperature decimal threshold.
This function will read high hbm temperature decimal threshold in ℃. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	a pointer to hold the high hbm temperature decimal threshold in ℃.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.4 read_sbtsi_hbm_hi_temp_th()

```
oob_status_t read_sbtsi_hbm_hi_temp_th (
            uint8_t soc_die_num,
            float * buffer )
```

Get high hbm temperature threshold.
This function will read high hbm temperature threshold in ℃. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	a pointer to hold the high hbm temperature in ℃

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.5 read_sbtsi_hbm_lo_temp_int_th()

```
oob_status_t read_sbtsi_hbm_lo_temp_int_th (
            uint8_t soc_die_num,
            uint8_t * buffer )
```

Get low hbm temperature integer threshold.
This function will read low hbm temperature interger threshold in ℃. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	a pointer to hold the low hbm temperature integer threshold in ℃.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.6 read_sbtsi_hbm_lo_temp_dec_th()

```
oob_status_t read_sbtsi_hbm_lo_temp_dec_th (
            uint8_t soc_die_num,
            float * buffer )
```

Get low hbm temperature decimal threshold.
This function will low high hbm temperature decimal threshold in ℃. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	a pointer to hold the low hbm temperature decimal threshold in ℃.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.7 write_sbtsi_hbm_lo_temp_th()

```
oob_status_t write_sbtsi_hbm_lo_temp_th (
            uint8_t soc_die_num,
            float temp_th )
```

Set low hbm temperature threshold.

This function will set low hbm temperature threshold in ℃. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*temp_th*	low hbm temperature threshold in ℃.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.8 read_sbtsi_max_hbm_temp_int()

```
oob_status_t read_sbtsi_max_hbm_temp_int (
            uint8_t soc_die_num,
            uint8_t * buffer )
```

Get max hbm integer temperature.

This function will read max hbm interger temperature in ℃ Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	a pointer to hold the max hbm integer temperature in ℃

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.9 read_sbtsi_max_hbm_temp_dec()

```
oob_status_t read_sbtsi_max_hbm_temp_dec (
            uint8_t soc_die_num,
            float * buffer )
```

Get max hbm decimal temperature.
This function will read max hbm decimal temperature in ℃ Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	a pointer to hold the max hbm decimal temperature in ℃.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.10 read_sbtsi_hbm_temp_int()

```
oob_status_t read_sbtsi_hbm_temp_int (
            uint8_t soc_die_num,
            uint8_t * buffer )
```

Get hbm integer temperature.
This function will read hbm integer temperature in ℃. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	a pointer to hold the hbm integer temperature in ℃

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.11 read_sbtsi_hbm_temp_dec()

```
oob_status_t read_sbtsi_hbm_temp_dec (
            uint8_t soc_die_num,
            float * buffer )
```

Get hbm decimal temperature.
This function will read hbm decimal temperature in ℃. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	a pointer to hold the hbm decimal temperature in ℃

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.12 read_sbtsi_hbm_lo_temp_th()

```
oob_status_t read_sbtsi_hbm_lo_temp_th (
            uint8_t soc_die_num,
            float * buffer )
```

Get hbm low temperature threshold.

This function will read hbm low threshold temperature in ℃. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	a pointer to hold the low hbm temperature threshold in ℃.

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.13 read_sbtsi_max_hbm_temp()

```
oob_status_t read_sbtsi_max_hbm_temp (
            uint8_t soc_die_num,
            float * buffer )
```

Get hbm maximum temperature.

This function will read maximum hbm temperature in ℃. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	a pointer to hold the max hbm temperature in ℃

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.14 read_sbtsi_hbm_temp()

```
oob_status_t read_sbtsi_hbm_temp (
            uint8_t soc_die_num,
            float * buffer )
```

Get hbm temperature.

This function will read maximum hbm temperature in ℃. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*buffer*	a pointer to hold the hbm temperature in ℃

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.15 read_sbtsi_hbm_alertthreshold()

```
oob_status_t read_sbtsi_hbm_alertthreshold (
            uint8_t soc_die_num,
            uint8_t * samples )
```

Get hbm alert threshold.

This function will read hbm alert threshold. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
out	*samples*	hbm threshold samples

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.16 sbtsi_set_hbm_alert_threshold()

```
oob_status_t sbtsi_set_hbm_alert_threshold (
            uint8_t soc_die_num,
            uint8_t samples )
```

Set hbm alert samples.

This function will set hbm alert samples. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*samples*	hbm threshold samples

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.17 get_sbtsi_hbm_alertconfig()

```
oob_status_t get_sbtsi_hbm_alertconfig (
            uint8_t soc_die_num,
            uint8_t * mode )
```

Get the sbtsi hbm alert config.

This function will read hbm alert config. 1 indicates Enable hbm high and low temperature alert and 0 indicates disbale hbm high and low temperature alert. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in,out	*mode*	HBM alert enable or disable

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

7.9.2.18 set_sbtsi_hbm_alertconfig()

```
oob_status_t set_sbtsi_hbm_alertconfig (
            uint8_t soc_die_num,
            uint8_t mode )
```

Set hbm alert config.

This function will set hbm alert config. 1 indicates enable hbm high and low temperature alert and 0 indicates disable hbm high and low temperature alert. Supported platforms: Fam-19h_Mod-90h-9Fh.

Parameters

in	*soc_die_num*	[3:0]Socket index, [7:4]Die index.
in	*mode*	1 indicates enable and 0 indicates disable

Return values

OOB_SUCCESS	is returned upon successful call.
Non-zero	is returned upon failure.

Index

apml.h, 83
    esmi_oob_read_byte, 84
    esmi_oob_read_mailbox, 87
    esmi_oob_rmi_read_byte, 85
    esmi_oob_rmi_write_byte, 86
    esmi_oob_tsi_read_byte, 86
    esmi_oob_tsi_write_byte, 87
    esmi_oob_write_byte, 85
    esmi_oob_write_mailbox, 87
    sbrmi_inbnd_msg, 84
    sbrmi_xfer_msg, 88
    sbtsi_xfer_msg, 88
    validate_apml_dependency, 89
    validate_sbrmi_module, 89
    validate_sbtsi_module, 89
apml_encodings, 59
apml_err.h, 91
    OOB_ARG_PTR_NULL, 92
    OOB_CMD_TIMEOUT, 92
    OOB_CPUID_MSR_CMD_ABORTED, 92
    OOB_CPUID_MSR_CMD_EXCESS_DATA_LEN, 92
    OOB_CPUID_MSR_CMD_INVAL_RD_LEN, 92
    OOB_CPUID_MSR_CMD_INVAL_THREAD, 92
    OOB_CPUID_MSR_CMD_TIMEOUT, 92
    OOB_CPUID_MSR_CMD_UNKNOWN_FMT, 92
    OOB_CPUID_MSR_CMD_UNSUPP, 92
    OOB_CPUID_MSR_CMD_WARM_RESET, 92
    OOB_FILE_ERROR, 92
    OOB_FORMATTING_ERROR, 92
    OOB_INTERRUPTED, 92
    OOB_INVALID_INPUT, 92
    OOB_INVALID_MSGSIZE, 92
    OOB_MAILBOX_ADD_ERR_DATA, 92
    OOB_MAILBOX_CMD_ABORTED, 92
    OOB_MAILBOX_CMD_INVAL_CORE, 92
    OOB_MAILBOX_CMD_UNKNOWN, 92
    OOB_MAILBOX_INVALID_INPUT_ARGS, 92
    OOB_MAILBOX_INVALID_OOBRAS_CONFIG, 92
    OOB_MAILBOX_NOT_READY, 92
    OOB_NO_MEMORY, 92
    OOB_NOT_FOUND, 92
    OOB_NOT_INITIALIZED, 92
    OOB_NOT_SUPPORTED, 92
    OOB_PERMISSION, 92
    oob_status_t, 92
    OOB_SUCCESS, 92
    OOB_TRY_AGAIN, 92
    OOB_UNEXPECTED_SIZE, 92
    OOB_UNKNOWN_ERROR, 92
apml_recover_dev
    apml_recovery.h, 93
apml_recovery.h, 92
    apml_recover_dev, 93
Auxiliary functions, 15
    errno_to_oob_status, 15
    esmi_get_err_msg, 15

bmc_ras_runtime_din, 59
    err_category, 60
    number_of_instance, 60
    number_of_offsets, 60
buffer_data
    ras_runtime_response, 77
buffer_size_per_instance
    ras_runtime_response, 77

clear_MP0_alert_status
    SB-RMI Register Read Byte Protocol, 39
clear_sbrmi_ras_status
    SB-RMI Register Read Byte Protocol, 38
clear_shut_down_err
    SB-RMI Register Read Byte Protocol, 41
clear_statistics
    rmi_mailbox_mi300.h, 160
clear_sw_async_alert_status
    SB-RMI Register Read Byte Protocol, 39
core_floor_limit_din, 60
core_mca_err_reporting_en
    oob_config_d_in, 71
cpuid_reg
    esmi_cpuid_msr.h, 94
Current, Min, Max TDP, 21
    read_max_tdp, 22
    read_min_tdp, 22
    read_tdp, 21

dimm_pow_din, 61
dimm_pow_dout, 61
dimm_power, 62
dimm_sb_reg_d_in, 62
dimm_sb_reg_write, 62
dimm_spd_d_in, 63
dimm_spd_write, 63
dimm_thermal, 64
dimm_thermal_din, 64
dpm_level, 65
Dram and other features Query, 24
    read_ccd_bist_result, 27

read_cclk_freq_limit, 28
read_ccx_bist_result, 28
read_ddr_bandwidth, 29
read_dram_throttle, 24
read_iod_bist, 27
read_nbio_error_logging_register, 26
read_socket_c0_residency, 29
read_vddio_mem_power, 26
write_dram_throttle, 26
dram_cecc_leak_rate
oob_config_d_in, 71
dram_cecc_oob_ec_mode
oob_config_d_in, 71
dram_thr_with_status, 65

err_category
bmc_ras_runtime_din, 60
err_size
oob_err_inj_types, 72
err_type
run_time_threshold, 79
err_types_vendor
oob_err_inj_types, 72
errno_to_oob_status
Auxiliary functions, 15
esb_mailbox_commmands
esmi_mailbox.h, 103
esmi_cpuid_msr.h, 93
cpuid_reg, 94
esmi_get_err_msg
Auxiliary functions, 15
esmi_get_logical_cores_per_socket
using CPUID Register Access, 31
esmi_get_processor_info
using CPUID Register Access, 30
esmi_get_threads_per_core
using CPUID Register Access, 31
esmi_get_threads_per_socket
SB-RMI Register Read Byte Protocol, 39
esmi_get_vendor_id
using CPUID Register Access, 30
esmi_mailbox.h, 94
esb_mailbox_commmands, 103
get_apb_state, 131
get_avg_dram_thr_with_status, 127
get_avg_dram_throttle, 127
get_bmc_ras_dram_ecc_run_time_error_info, 135
get_bmc_ras_mca_run_time_error_info, 134
get_bmc_ras_oob_config, 121
get_bmc_ras_pcie_run_time_error_info, 135
get_bmc_ras_run_time_err_validity_ck, 119
get_bmc_ras_run_time_error_info, 120
get_cc6_enable, 129
get_ccd_power_consumption, 132
get_ch_dram_thr_with_status, 127
get_ch_dram_throttle, 128
get_cpu_rail_iso_freq_policy, 125
get_df_pstate_range, 131
get_dfc_enable, 126
get_dimm_pow_data, 140
get_dimm_sb_register, 122
get_dimm_serial_num, 123
GET_DIMM_SPD, 104
get_dimm_thermal_sensor_data, 140
get_effective_floor_freq_per_core, 138
get_enabled_hsmp_commands, 139
get_floor_core_limit, 136
get_hottest_dimm_temp_range_ref_rate, 141
get_pc6_enable, 128
get_post_code, 119
get_power_efficiency_mode_selection, 137
get_supporting_error_types, 134
get_svi3_vr_controller_temp_by_rail, 133
get_tdelta, 133
get_xgmi_link_width_range, 130
get_xgmi_pstate_range, 132
override_delay_reset_on_sync_flood, 118
read_bmc_control_pcie_gen5_rate, 114
read_bmc_cpu_base_frequency, 113
read_bmc_rapl_units, 113
read_bmc_ras_fch_reset_reason, 106
read_bmc_ras_mca_msr_dump, 106
read_bmc_ras_mca_validity_check, 105
read_bmc_ras_pcie_config_access, 105
read_current_dfpstate_frequency, 112
read_current_io_bandwidth, 109
read_current_xgmi_bandwidth, 110
read_dimm_power_consumption, 107
read_dimm_spd_register, 122
read_dimm_temp_range_and_refresh_rate, 106
read_dimm_thermal_sensor, 107
read_lclk_dpm_level_range, 116
read_ppin_fuse, 122
read_pwr_current_active_freq_limit_core, 108
read_pwr_current_active_freq_limit_socket, 108
read_pwr_svi_telemetry_all_rails, 109
read_rapl_core_energy_counters, 114
read_rapl_pckg_energy_counters, 115
read_ras_df_err_dump, 117
read_ras_df_err_validity_check, 117
read_rtc, 122
read_smu_fw_ver, 124
read_socket_freq_range, 109
read_ucode_revision, 116
reset_on_sync_flood, 118
set_bmc_ras_err_threshold, 120
set_bmc_ras_oob_config, 121
set_cc6_enable, 130
set_core_floor_limit, 136
set_cpu_rail_iso_freq_policy, 125
set_dfc_enable, 126
set_dimm_sb_register_data, 123
SET_DIMM_SPD, 104
set_enabled_hsmp_commands, 139
set_floor_limit_for_all_cores, 136
set_pc6_enable, 129
set_pcie_link_training, 131

set_power_efficiency_mode_selection, 137
set_xgmi_pstate_range, 124
soc_dimm_power_limit, 138
write_apb_disable, 111
write_apb_enable, 112
write_bmc_pcie_config, 124
write_bmc_report_dimm_power, 104
write_bmc_report_dimm_thermal_sensor, 104
write_df_pstate_range, 116
write_dimm_spd_register, 123
write_gmi3_link_width_range, 110
write_lclk_dpm_level_range, 113
WRITE_PCIE_LINK_CONTROL, 104
write_pcie_link_control, 141
write_pwr_efficiency_mode, 115
write_xgmi_link_width_range, 111
esmi_oob_cpuid
SB-RMI CPUID Register Access, 33
esmi_oob_cpuid_eax
SB-RMI CPUID Register Access, 33
esmi_oob_cpuid_ebx
SB-RMI CPUID Register Access, 34
esmi_oob_cpuid_ecx
SB-RMI CPUID Register Access, 34
esmi_oob_cpuid_edx
SB-RMI CPUID Register Access, 35
esmi_oob_read_byte
apml.h, 84
esmi_oob_read_mailbox
apml.h, 87
esmi_oob_read_msr
SB_RMI Read Processor Register Access, 32
esmi_oob_rmi_read_byte
apml.h, 85
esmi_oob_rmi_write_byte
apml.h, 86
esmi_oob_tsi_read_byte
apml.h, 86
esmi_oob_tsi_write_byte
apml.h, 87
esmi_oob_write_byte
apml.h, 85
esmi_oob_write_mailbox
apml.h, 87
esmi_rmi.h, 142
esmi_tsi.h, 144

freq_limits, 66

get_act_gfx_freq_cap
rmi_mailbox_mi300.h, 155
get_alarms
rmi_mailbox_mi300.h, 153
get_apb_state
esmi_mailbox.h, 131
get_avg_dram_thr_with_status
esmi_mailbox.h, 127
get_avg_dram_throttle
esmi_mailbox.h, 127
get_bist_results
rmi_mailbox_mi300.h, 159
get_bmc_ras_dram_ecc_run_time_error_info
esmi_mailbox.h, 135
get_bmc_ras_mca_run_time_error_info
esmi_mailbox.h, 134
get_bmc_ras_oob_config
esmi_mailbox.h, 121
get_bmc_ras_pcie_run_time_error_info
esmi_mailbox.h, 135
get_bmc_ras_run_time_err_validity_ck
esmi_mailbox.h, 119
get_bmc_ras_run_time_error_info
esmi_mailbox.h, 120
get_cc6_enable
esmi_mailbox.h, 129
get_ccd_power_consumption
esmi_mailbox.h, 132
get_ch_dram_thr_with_status
esmi_mailbox.h, 127
get_ch_dram_throttle
esmi_mailbox.h, 128
get_clk_freq_limits
rmi_mailbox_mi300.h, 159
get_cpu_rail_iso_freq_policy
esmi_mailbox.h, 125
get_curr_xgmi_pstate
rmi_mailbox_mi300.h, 161
get_df_pstate_range
esmi_mailbox.h, 131
get_dfc_enable
esmi_mailbox.h, 126
get_die_hotspot_info
rmi_mailbox_mi300.h, 156
get_die_type
rmi_mailbox_mi300.h, 161
get_dimm_pow_data
esmi_mailbox.h, 140
get_dimm_sb_register
esmi_mailbox.h, 122
get_dimm_serial_num
esmi_mailbox.h, 123
GET_DIMM_SPD
esmi_mailbox.h, 104
get_dimm_thermal_sensor_data
esmi_mailbox.h, 140
get_effective_floor_freq_per_core
esmi_mailbox.h, 138
get_enabled_hsmp_commands
esmi_mailbox.h, 139
get_energy_accum_with_timestamp
rmi_mailbox_mi300.h, 153
get_floor_core_limit
esmi_mailbox.h, 136
get_hbm_dev_info
rmi_mailbox_mi300.h, 162
get_hbm_temperature
rmi_mailbox_mi300.h, 158

get_hbm_throttle
rmi_mailbox_mi300.h, 157
get_host_status
rmi_mailbox_mi300.h, 157
get_hottest_dimm_temp_range_ref_rate
esmi_mailbox.h, 141
get_hsmp_cmds_din, 66
get_hsmp_cmds_dout, 66
get_link_info
rmi_mailbox_mi300.h, 154
get_max_mem_bw_util
rmi_mailbox_mi300.h, 157
get_max_min_gfx_freq
rmi_mailbox_mi300.h, 154
get_max_operating_temp
rmi_mailbox_mi300.h, 161
get_mclk_fclk_pstates
rmi_mailbox_mi300.h, 151
get_mem_hotspot_info
rmi_mailbox_mi300.h, 156
get_pc6_enable
esmi_mailbox.h, 128
get_pciestats
rmi_mailbox_mi300.h, 163
get_post_code
esmi_mailbox.h, 119
get_power_efficiency_mode_selection
esmi_mailbox.h, 137
get_psn
rmi_mailbox_mi300.h, 154
get_sbtsi_hbm_alertconfig
tsi_mi300.h, 170
get_slow_down_temp
rmi_mailbox_mi300.h, 162
get_sockets_in_system
rmi_mailbox_mi300.h, 159
get_statistics
rmi_mailbox_mi300.h, 160
get_supporting_error_types
esmi_mailbox.h, 134
get_svi3_vr_controller_temp_by_rail
esmi_mailbox.h, 133
get_svi_rail_telemetry
rmi_mailbox_mi300.h, 155
get_tdelta
esmi_mailbox.h, 133
get_xcc_idle_residency
rmi_mailbox_mi300.h, 152
get_xgmi_link_width_range
esmi_mailbox.h, 130
get_xgmi_pstate_range
esmi_mailbox.h, 132
get_xgmi_pstates
rmi_mailbox_mi300.h, 152
gpio_assertion_for_async_alerts
SB-RMI Register Read Byte Protocol, 41
gpio_assertion_on_mailbox
SB-RMI Register Read Byte Protocol, 41
hbm_device_info, 67
host_status, 67
hottest_dimm_temp_refresh_rate, 68

input
ras_df_err_dump, 75

lclk_dpm_level_range, 68
link_id
link_id_bw_type, 69
link_id_bw_type, 68
link_id, 69

max_ddr_bw, 69
max_mem_bw, 69
mca_bank, 70
mca_oob_misc0_ec_enable
oob_config_d_in, 71
mclk_fclk_pstates, 70

nbio_err_log, 70
num_of_instances
ras_runtime_response, 77
number_of_instance
bmc_ras_runtime_din, 60
number_of_offsets
bmc_ras_runtime_din, 60

OOB_ARG_PTR_NULL
apml_err.h, 92
OOB_CMD_TIMEOUT
apml_err.h, 92
oob_config_d_in, 71
core_mca_err_reporting_en, 71
dram_cecc_leak_rate, 71
dram_cecc_oob_ec_mode, 71
mca_oob_misc0_ec_enable, 71
pcie_err_reporting_en, 71
OOB_CPUID_MSR_CMD_ABORTED
apml_err.h, 92
OOB_CPUID_MSR_CMD_EXCESS_DATA_LEN
apml_err.h, 92
OOB_CPUID_MSR_CMD_INVAL_RD_LEN
apml_err.h, 92
OOB_CPUID_MSR_CMD_INVAL_THREAD
apml_err.h, 92
OOB_CPUID_MSR_CMD_TIMEOUT
apml_err.h, 92
OOB_CPUID_MSR_CMD_UNKNOWN_FMT
apml_err.h, 92
OOB_CPUID_MSR_CMD_UNSUPP
apml_err.h, 92
OOB_CPUID_MSR_CMD_WARM_RESET
apml_err.h, 92
oob_err_inj_types, 72
err_size, 72
err_types_vendor, 72
OOB_FILE_ERROR
apml_err.h, 92

OOB_FORMATTING_ERROR
apml_err.h, 92
OOB_INTERRUPTED
apml_err.h, 92
OOB_INVALID_INPUT
apml_err.h, 92
OOB_INVALID_MSGSIZE
apml_err.h, 92
OOB_MAILBOX_ADD_ERR_DATA
apml_err.h, 92
OOB_MAILBOX_CMD_ABORTED
apml_err.h, 92
OOB_MAILBOX_CMD_INVAL_CORE
apml_err.h, 92
OOB_MAILBOX_CMD_UNKNOWN
apml_err.h, 92
OOB_MAILBOX_INVALID_INPUT_ARGS
apml_err.h, 92
OOB_MAILBOX_INVALID_OOBRAS_CONFIG
apml_err.h, 92
OOB_MAILBOX_NOT_READY
apml_err.h, 92
OOB_NO_MEMORY
apml_err.h, 92
OOB_NOT_FOUND
apml_err.h, 92
OOB_NOT_INITIALIZED
apml_err.h, 92
OOB_NOT_SUPPORTED
apml_err.h, 92
OOB_PERMISSION
apml_err.h, 92
oob_status_t
apml_err.h, 92
OOB_SUCCESS
apml_err.h, 92
OOB_TRY_AGAIN
apml_err.h, 92
OOB_UNEXPECTED_SIZE
apml_err.h, 92
OOB_UNKNOWN_ERROR
apml_err.h, 92
op_buffer_size
ras_runtime_response, 77
Out-of-band Performance (Boost limit) Control, 20
write_esb_boost_limit, 20
write_esb_boost_limit_allcores, 21
override_delay_reset_on_sync_flood
esmi_mailbox.h, 118

pci_address, 72
pcie_err_reporting_en
oob_config_d_in, 71
pcie_link_training, 73
Performance (Boost limit) Monitor, 19
read_bios_boost_fmax, 19
read_esb_boost_limit, 19
Platforms, 18
pow_eff_mode, 73
Power Control, 18
write_socket_power_limit, 18
Power Monitor, 16
read_max_socket_power_limit, 17
read_socket_power, 16
read_socket_power_limit, 17
processor_info, 74
Prochot, 23
read_prochot_residency, 23
read_prochot_status, 23
pstate_freq, 74

rail_mode
svi3_vr_cont_data_in, 80
ras_df_err_chk, 74
ras_df_err_dump, 75
input, 75
ras_override_delay, 75
ras_rt_err_req_type, 76
ras_rt_valid_err_inst, 76
ras_runtime_response, 77
buffer_data, 77
buffer_size_per_instance, 77
num_of_instances, 77
op_buffer_size, 77
read_bios_boost_fmax
Performance (Boost limit) Monitor, 19
read_bmc_control_pcie_gen5_rate
esmi_mailbox.h, 114
read_bmc_cpu_base_frequency
esmi_mailbox.h, 113
read_bmc_rapl_units
esmi_mailbox.h, 113
read_bmc_ras_fch_reset_reason
esmi_mailbox.h, 106
read_bmc_ras_mca_msr_dump
esmi_mailbox.h, 106
read_bmc_ras_mca_validity_check
esmi_mailbox.h, 105
read_bmc_ras_pcie_config_access
esmi_mailbox.h, 105
read_ccd_bist_result
Dram and other features Query, 27
read_cclk_freq_limit
Dram and other features Query, 28
read_ccx_bist_result
Dram and other features Query, 28
read_current_dfpstate_frequency
esmi_mailbox.h, 112
read_current_io_bandwidth
esmi_mailbox.h, 109
read_current_xgmi_bandwidth
esmi_mailbox.h, 110
read_ddr_bandwidth
Dram and other features Query, 29
read_dimm_power_consumption
esmi_mailbox.h, 107
read_dimm_spd_register
esmi_mailbox.h, 122

read_dimm_temp_range_and_refresh_rate
esmi_mailbox.h, 106
read_dimm_thermal_sensor
esmi_mailbox.h, 107
read_dram_throttle
Dram and other features Query, 24
read_esb_boost_limit
Performance (Boost limit) Monitor, 19
read_iod_bist
Dram and other features Query, 27
read_lclk_dpm_level_range
esmi_mailbox.h, 116
read_max_socket_power_limit
Power Monitor, 17
read_max_tdp
Current, Min, Max TDP, 22
read_max_threads_per_l3
SB-RMI CPUID Register Access, 35
read_min_tdp
Current, Min, Max TDP, 22
read_nbio_error_logging_register
Dram and other features Query, 26
read_ppin_fuse
esmi_mailbox.h, 122
read_prochot_residency
Prochot, 23
read_prochot_status
Prochot, 23
read_pwr_current_active_freq_limit_core
esmi_mailbox.h, 108
read_pwr_current_active_freq_limit_socket
esmi_mailbox.h, 108
read_pwr_svi_telemetry_all_rails
esmi_mailbox.h, 109
read_rapl_core_energy_counters
esmi_mailbox.h, 114
read_rapl_pckg_energy_counters
esmi_mailbox.h, 115
read_ras_df_err_dump
esmi_mailbox.h, 117
read_ras_df_err_validity_check
esmi_mailbox.h, 117
read_rtc
esmi_mailbox.h, 122
read_sbrmi_alert_mask
SB-RMI Register Read Byte Protocol, 38
read_sbrmi_alert_status
SB-RMI Register Read Byte Protocol, 38
read_sbrmi_ready_status
SB-RMI Register Read Byte Protocol, 40
read_sbtsi_alertconfig
SBTSI Register Read Byte Protocol, 51
read_sbtsi_alertthreshold
SBTSI Register Read Byte Protocol, 50
read_sbtsi_config
SBTSI Register Read Byte Protocol, 46
read_sbtsi_configwrite
SBTSI Register Read Byte Protocol, 48
read_sbtsi_cpuinttemp
SBTSI Register Read Byte Protocol, 45
read_sbtsi_cputempdecimal
SBTSI Register Read Byte Protocol, 48
read_sbtsi_cputempoffdec
SBTSI Register Read Byte Protocol, 49
read_sbtsi_cputempoffint
SBTSI Register Read Byte Protocol, 48
read_sbtsi_cputempoffset
SBTSI Register Read Byte Protocol, 56
read_sbtsi_hbm_alertthreshold
tsi_mi300.h, 170
read_sbtsi_hbm_hi_temp_dec_th
tsi_mi300.h, 165
read_sbtsi_hbm_hi_temp_int_th
tsi_mi300.h, 164
read_sbtsi_hbm_hi_temp_th
tsi_mi300.h, 166
read_sbtsi_hbm_lo_temp_dec_th
tsi_mi300.h, 166
read_sbtsi_hbm_lo_temp_int_th
tsi_mi300.h, 166
read_sbtsi_hbm_lo_temp_th
tsi_mi300.h, 169
read_sbtsi_hbm_temp
tsi_mi300.h, 169
read_sbtsi_hbm_temp_dec
tsi_mi300.h, 168
read_sbtsi_hbm_temp_int
tsi_mi300.h, 168
read_sbtsi_hitempdecimal
SBTSI Register Read Byte Protocol, 49
read_sbtsi_hitempint
SBTSI Register Read Byte Protocol, 47
read_sbtsi_lotempdecimal
SBTSI Register Read Byte Protocol, 49
read_sbtsi_lotempint
SBTSI Register Read Byte Protocol, 47
read_sbtsi_manufid
SBTSI Register Read Byte Protocol, 51
read_sbtsi_max_hbm_temp
tsi_mi300.h, 169
read_sbtsi_max_hbm_temp_dec
tsi_mi300.h, 167
read_sbtsi_max_hbm_temp_int
tsi_mi300.h, 167
read_sbtsi_revision
SBTSI Register Read Byte Protocol, 51
read_sbtsi_status
SBTSI Register Read Byte Protocol, 45
read_sbtsi_timeoutconfig
SBTSI Register Read Byte Protocol, 50
read_sbtsi_updaterate
SBTSI Register Read Byte Protocol, 46
read_smu_fw_ver
esmi_mailbox.h, 124
read_socket_c0_residency
Dram and other features Query, 29

read_socket_freq_range
esmi_mailbox.h, 109
read_socket_power
Power Monitor, 16
read_socket_power_limit
Power Monitor, 17
read_tdp
Current, Min, Max TDP, 21
read_ucode_revision
esmi_mailbox.h, 116
read_vddio_mem_power
Dram and other features Query, 26
reset_on_sync_flood
esmi_mailbox.h, 118
rmi_mailbox_mi300.h, 147
clear_statistics, 160
get_act_gfx_freq_cap, 155
get_alarms, 153
get_bist_results, 159
get_clk_freq_limits, 159
get_curr_xgmi_pstate, 161
get_die_hotspot_info, 156
get_die_type, 161
get_energy_accum_with_timestamp, 153
get_hbm_dev_info, 162
get_hbm_temperature, 158
get_hbm_throttle, 157
get_host_status, 157
get_link_info, 154
get_max_mem_bw_util, 157
get_max_min_gfx_freq, 154
get_max_operating_temp, 161
get_mclk_fclk_pstates, 151
get_mem_hotspot_info, 156
get_pciestats, 163
get_psn, 154
get_slow_down_temp, 162
get_sockets_in_system, 159
get_statistics, 160
get_svi_rail_telemetry, 155
get_xcc_idle_residency, 152
get_xgmi_pstates, 152
set_gfx_core_clock, 150
set_hbm_throttle, 158
set_mclk_fclk_max_pstate, 150
set_xgmi_pstate, 151
unset_xgmi_pstate, 152
run_time_err_d_in, 78
run_time_threshold, 78
err_type, 79

SB-RMI CPUID Register Access, 32
esmi_oob_cpuid, 33
esmi_oob_cpuid_eax, 33
esmi_oob_cpuid_ebx, 34
esmi_oob_cpuid_ecx, 34
esmi_oob_cpuid_edx, 35
read_max_threads_per_l3, 35
SB-RMI Mailbox Service, 16
SB-RMI Register Read Byte Protocol, 36
clear_MP0_alert_status, 39
clear_sbrmi_ras_status, 38
clear_shut_down_err, 41
clear_sw_async_alert_status, 39
esmi_get_threads_per_socket, 39
gpio_assertion_for_async_alerts, 41
gpio_assertion_on_mailbox, 41
read_sbrmi_alert_mask, 38
read_sbrmi_alert_status, 38
read_sbrmi_ready_status, 40
set_sw_async_alert_mask, 40
thread_en_reg_v10, 42
SB_RMI Read Processor Register Access, 31
esmi_oob_read_msr, 32
sbrmi_inbnd_msg
apml.h, 84
sbrmi_xfer_msg
apml.h, 88
SBTSI Register Read Byte Protocol, 42
read_sbtsi_alertconfig, 51
read_sbtsi_alertthreshold, 50
read_sbtsi_config, 46
read_sbtsi_configwrite, 48
read_sbtsi_cpuinttemp, 45
read_sbtsi_cputempdecimal, 48
read_sbtsi_cputempoffdec, 49
read_sbtsi_cputempoffint, 48
read_sbtsi_cputempoffset, 56
read_sbtsi_hitempdecimal, 49
read_sbtsi_hitempint, 47
read_sbtsi_lotempdecimal, 49
read_sbtsi_lotempint, 47
read_sbtsi_manufid, 51
read_sbtsi_revision, 51
read_sbtsi_status, 45
read_sbtsi_timeoutconfig, 50
read_sbtsi_updaterate, 46
sbtsi_get_config, 53
sbtsi_get_cputemp, 52
sbtsi_get_hitemp_threshold, 55
sbtsi_get_lotemp_threshold, 56
sbtsi_get_temp_status, 52
sbtsi_get_timeout, 54
sbtsi_set_alert_config, 57
sbtsi_set_alert_threshold, 57
sbtsi_set_configwr, 53
sbtsi_set_hitemp_threshold, 54
sbtsi_set_lotemp_threshold, 55
sbtsi_set_timeout_config, 54
write_sbtsi_cputempoffset, 56
write_sbtsi_updaterate, 46
sbtsi_get_config
SBTSI Register Read Byte Protocol, 53
sbtsi_get_cputemp
SBTSI Register Read Byte Protocol, 52
sbtsi_get_hitemp_threshold
SBTSI Register Read Byte Protocol, 55

sbtsi_get_lotemp_threshold
SBTSI Register Read Byte Protocol, 56
sbtsi_get_temp_status
SBTSI Register Read Byte Protocol, 52
sbtsi_get_timeout
SBTSI Register Read Byte Protocol, 54
sbtsi_set_alert_config
SBTSI Register Read Byte Protocol, 57
sbtsi_set_alert_threshold
SBTSI Register Read Byte Protocol, 57
sbtsi_set_configwr
SBTSI Register Read Byte Protocol, 53
sbtsi_set_hbm_alert_threshold
tsi_mi300.h, 170
sbtsi_set_hitemp_threshold
SBTSI Register Read Byte Protocol, 54
sbtsi_set_lotemp_threshold
SBTSI Register Read Byte Protocol, 55
sbtsi_set_timeout_config
SBTSI Register Read Byte Protocol, 54
sbtsi_xfer_msg
apml.h, 88
set_bmc_ras_err_threshold
esmi_mailbox.h, 120
set_bmc_ras_oob_config
esmi_mailbox.h, 121
set_cc6_enable
esmi_mailbox.h, 130
set_core_floor_limit
esmi_mailbox.h, 136
set_cpu_rail_iso_freq_policy
esmi_mailbox.h, 125
set_dfc_enable
esmi_mailbox.h, 126
set_dimm_sb_register_data
esmi_mailbox.h, 123
SET_DIMM_SPD
esmi_mailbox.h, 104
set_enabled_hsmp_commands
esmi_mailbox.h, 139
set_floor_limit_for_all_cores
esmi_mailbox.h, 136
set_gfx_core_clock
rmi_mailbox_mi300.h, 150
set_hbm_throttle
rmi_mailbox_mi300.h, 158
set_hsmp_cmds_din, 79
set_mclk_fclk_max_pstate
rmi_mailbox_mi300.h, 150
set_pc6_enable
esmi_mailbox.h, 129
set_pcie_link_training
esmi_mailbox.h, 131
set_power_efficiency_mode_selection
esmi_mailbox.h, 137
set_sbtsi_hbm_alertconfig
tsi_mi300.h, 171
set_sw_async_alert_mask
SB-RMI Register Read Byte Protocol, 40
set_xgmi_pstate
rmi_mailbox_mi300.h, 151
set_xgmi_pstate_range
esmi_mailbox.h, 124
soc_dimm_power_limit
esmi_mailbox.h, 138
statistics, 79
svi3_vr_cont_data_in, 80
rail_mode, 80
svi3_vr_cont_data_out, 80
svi_port_domain, 80

temp_refresh_rate, 81
thread_en_reg_v10
SB-RMI Register Read Byte Protocol, 42
tsi_mi300.h, 163
get_sbtsi_hbm_alertconfig, 170
read_sbtsi_hbm_alertthreshold, 170
read_sbtsi_hbm_hi_temp_dec_th, 165
read_sbtsi_hbm_hi_temp_int_th, 164
read_sbtsi_hbm_hi_temp_th, 166
read_sbtsi_hbm_lo_temp_dec_th, 166
read_sbtsi_hbm_lo_temp_int_th, 166
read_sbtsi_hbm_lo_temp_th, 169
read_sbtsi_hbm_temp, 169
read_sbtsi_hbm_temp_dec, 168
read_sbtsi_hbm_temp_int, 168
read_sbtsi_max_hbm_temp, 169
read_sbtsi_max_hbm_temp_dec, 167
read_sbtsi_max_hbm_temp_int, 167
sbtsi_set_hbm_alert_threshold, 170
set_sbtsi_hbm_alertconfig, 171
write_sbtsi_hbm_hi_temp_th, 165
write_sbtsi_hbm_lo_temp_th, 167

unset_xgmi_pstate
rmi_mailbox_mi300.h, 152
using CPUID Register Access, 30
esmi_get_logical_cores_per_socket, 31
esmi_get_processor_info, 30
esmi_get_threads_per_core, 31
esmi_get_vendor_id, 30

validate_apml_dependency
apml.h, 89
validate_sbrmi_module
apml.h, 89
validate_sbtsi_module
apml.h, 89

write_apb_disable
esmi_mailbox.h, 111
write_apb_enable
esmi_mailbox.h, 112
write_bmc_pcie_config
esmi_mailbox.h, 124
write_bmc_report_dimm_power
esmi_mailbox.h, 104

write_bmc_report_dimm_thermal_sensor
esmi_mailbox.h, 104
write_df_pstate_range
esmi_mailbox.h, 116
write_dimm_spd_register
esmi_mailbox.h, 123
write_dram_throttle
Dram and other features Query, 26
write_esb_boost_limit
Out-of-band Performance (Boost limit) Control, 20
write_esb_boost_limit_allcores
Out-of-band Performance (Boost limit) Control, 21
write_gmi3_link_width_range
esmi_mailbox.h, 110
write_lclk_dpm_level_range
esmi_mailbox.h, 113
WRITE_PCIE_LINK_CONTROL
esmi_mailbox.h, 104
write_pcie_link_control
esmi_mailbox.h, 141
write_pwr_efficiency_mode
esmi_mailbox.h, 115
write_sbtsi_cputempoffset
SBTSI Register Read Byte Protocol, 56
write_sbtsi_hbm_hi_temp_th
tsi_mi300.h, 165
write_sbtsi_hbm_lo_temp_th
tsi_mi300.h, 167
write_sbtsi_updaterate
SBTSI Register Read Byte Protocol, 46
write_socket_power_limit
Power Control, 18
write_xgmi_link_width_range
esmi_mailbox.h, 111

xgmi_speed_rate_n_width, 81